Generic and Algebraic Computation Models: When AGM Proofs Transfer to the GGM

Joseph Jaeger and Deep Inder Mohan

School of Cybersecurity and Privacy
Georgia Institute of Technology
Atlanta, Georgia, USA
{josephjaeger, dmohan}@gatech.edu
https://cc.gatech.edu/~josephjaeger/
https://deep-inder.github.io/

Abstract. The Fuchsbauer, Kiltz, and Loss (CRYPTO 2018) claim that (some) hardness results in the algebraic group model imply the same hardness results in the generic group model was recently called into question by Katz, Zhang, and Zhou (ASIACRYPT 2022). The latter gave an interpretation of the claim under which it is incorrect. We give an alternate interpretation under which it is correct, using natural frameworks for capturing generic and algebraic models for arbitrary algebraic structures. Most algebraic analyses in the literature can be captured by our frameworks, making the claim correct for them.

Contents

A preliminary version of this work appears in the CRYPTO 2024 proceedings [JM24], ©IACR 2024.
This is the full version.

1 Introduction

Since the earliest days of modern cryptography, assumptions about the difficulty of computational tasks on groups have been central to its study [DH76]. Resultantly, the relationships between the security of cryptographic schemes and underlying hard problems for cryptographic groups have been studied in a variety of ways. In this work, we focus on non-standard models of group-based schemes, such as the generic group models (GGMs) as introduced by Maurer [Mau05] and Shoup [Sho97],[1] as well as the more recent algebraic group model (AGM) of Fuchsbauer, Kiltz, and Loss [FKL18] (FKL).

In different ways, these capture the idea that in a sufficiently strong group, real-world algorithms performing relevant computations about the group are likely to be honestly executing the intended operations of the group. In the AGM, one tracks the group elements $X_1, \ldots X_n$ given as input to an algorithm. When the algorithm outputs a group element Y it must additionally provide an "explanation" explaining how Y was derived from the input elements. The explanation takes the form of a vector $(e_1, \ldots, e_n)$ such that $Y = X_1^{e_1} \times X_2^{e_2} \times \cdots \times X_n^{e_n}$. A reduction can arrange to pass group elements as inputs to the underlying algorithm such that success in its goal implies the explanation satisfies a polynomial equation, from which the reduction extracts the solution to its own problem. This polynomial-based methodology is similar to the analysis that underlies proofs in the GGMs.

The AGM has gained widespread popularity (e.g., the original paper currently has over 300 citations) and has been used to analyze theoretically interesting and practically relevant protocols. FKL motivated its use as follows.

> It is the first restricted model of computation covering group-specific algorithms yet allowing to derive simple and meaningful security statements. [FKL18, Abstract]

This simplicity is core to our current work. Proofs in the GGM often begin with somewhat boilerplate steps to replace the "honest" generic group with one where group elements are represented by polynomials. The proofs eventually end with another somewhat boilerplate argument about the probability of the polynomials colliding on random inputs. In some sense, the heart of the proof lies in showing the desired security reduces to collisions in these polynomials. The AGM allows one to focus on this heart. The first boilerplate step is removed as the given algorithm's explanation vector tells us immediately what polynomial corresponds to its output group elements. The second boilerplate step is removed because the AGM focuses on modular analysis. These low-level details are instead covered by assuming and reducing to some underlying security property. So the AGM allows a simple modular proof. Additionally, FKL showed a "lifting" result that (under minor conditions) AGM security automatically carries over to the GGM for free!

Katz, Zhang, and Zhou (KZZ) [ZZK22][2] recently called that claim into question. They observe ambiguities in the claims of FKL and provide one possible interpretation, which focuses on Shoup's GGM. They define a game that is AGM secure (via a Shoup GGM reduction) but Shoup GGM insecure, so the lifting result must fail for this interpretation. We emphasize that this result only applies to one specific interpretation of what FKL meant in their work.

Zhandry [Zha22] argued that the model envisioned by FKL is best captured by restricting analysis to security gamesthath exist in Maurer's GGM. He observes that this restriction circumvents

[1] Recall that Shoup's GGM treats each group element as uniquely represented by a string (chosen either at random or in the "worst-case"). Maurer's GGM has no unique representations; group elements are instead referenced using (non-unique) pointers into a table. In either case, oracles are provided for performing group operations.

[2] The author order differs between the published and ePrint versions of [ZZK22]. When asked, KZZ indicated that alphabetical order was intended.

the KZZ counterexample, as the underlying game is not within Maurer's GGM. This circumvention leaves open whether the lifting result holds under this interpretation or further counterexamples may be possible.

Our main contribution is a new interpretation under which the lifting result *provably* holds. The interpretation is both natural (e.g., matches the informal language used when discussing the AGM) and useful (many uses of the AGM are captured by it). It affirms the underlying ideas of the claim by FKL while complementing the interpretations of KZZ by helping refine our understanding of when the claim holds. We lift from the AGM to Maurer's GGM, assuming the security reduction exists in the Maurer GGM. This supports Zhandry's interpretation, as there is a natural correlation between when security games and reductions (which typically need to emulate the security games) can be expressed in the Maurer GGM. Our lifting result will additionally apply if only the reduction and not the security game can be expressed in the Maurer GGM, we are only aware of contrived examples of this. Along the way, we also consider and categorize other criticisms and limitations of the AGM discussed, e.g., by FKL, KZZ, and Zhandry.

1.1 Our Contributions

Three frameworks of computation. Our primary contribution is defining three different frameworks to express algorithms that compute on groups, which allow us to prove the lifting result.[3] These frameworks act as three lenses for analyzing existing group-based algorithms. We ultimately establish an equivalence between these frameworks. By being able to freely switch between our three frameworks, we can appropriately choose the best-suited framework to analyze any given question.

We briefly intuit one of our three frameworks inspired by programming language concepts—the pseudocode framework (Sec. 3). Algorithms are expressed in this framework as pseudocode for a "typed" programming language. We assume the existence of standard data types like $\langle\textbf{int}\rangle$ and $\langle\textbf{bits}\rangle$ in the language, which may be freely operated on. To impose a restriction for groups, we define a special data type $\langle\textbf{elem}\rangle$ for group elements. Variables of this type may only be used as either inputs to or outputs from a predefined set of functions.

For cyclic groups, think of functions `gen`, `op`, `eq`, encode_σ, and decode_σ. Function `gen`() outputs the group generator of type $\langle\textbf{elem}\rangle$. Function $\texttt{op}(X, Y)$ takes two $\langle\textbf{elem}\rangle$ inputs and outputs the $\langle\textbf{elem}\rangle$ variable XY obtained by applying the group operation. Relation $\texttt{eq}(X, Y)$ returns a $\langle\textbf{bit}\rangle$ indicating whether the input $\langle\textbf{elem}\rangle$ variables are equal. The encoding function $\text{encode}_\sigma(X)$ maps a $\langle\textbf{elem}\rangle$ variable to its representation (according to encoding function σ) as a bitstring $\langle\textbf{bits}\rangle$ s. The decoding function $\text{decode}_\sigma(s)$ maps from a $\langle\textbf{bits}\rangle$ variable to the $\langle\textbf{elem}\rangle$ value it represents (according to σ).

Within this framework, we create a natural "hierarchy" of types of algorithms defined by *syntactically* restricting what functions an algorithm has access to. At the lowest level are type-safe algorithms which can use the `gen`, `op`, and `eq` operations on $\langle\textbf{elem}\rangle$ variables. The next, algebraic, level adds the use of encode_σ to typecast from $\langle\textbf{elem}\rangle$ to a $\langle\textbf{bits}\rangle$. All new $\langle\textbf{elem}\rangle$ variables produced would be derived from the outputs of `gen` and `op`, but the algorithms can "see" the group elements to decide *how* to produce them. Finally, the highest "unrestricted" level adds decode_σ to typecast $\langle\textbf{bits}\rangle$ variables to $\langle\textbf{elem}\rangle$. This level allows an algorithm to encode all group elements as bits, perform whatever computations it desires, and then decode the resulting elements back into the desired format.

While it is not immediately obvious, this hierarchy of three different syntactic restrictions to the same computational model captures exactly the algorithms expected for the Maurer-style "type-safe"

[3] In fact, following Maurer [Mau05], we express them more generally such that our frameworks could capture other computational models of interest. For simplicity in the introduction, we focus only on the traditional group models.

generic group model, FKL-style algebraic group model, and the standard model of computation. Because `gen`, `op`, and `eq` are independent of the choice of the encoding function, being restricted to their use gives a Maurer-style version of being generic. Then access to encode_σ allows an algorithm to "see" what each group element is, but they are still restricted from generating $\langle \textbf{elem} \rangle$ outputs that are unrelated to their $\langle \textbf{elem} \rangle$ inputs by the behavior of `gen` and `op`, which gives them the algebraic flavor. Finally, $\text{decode}_\sigma()$ effectively removes this restriction, thus allowing arbitrary standard model computation when σ is known.

Armed with this framework, we map our interpretation of the algebraic group model to those of prior works (Sec. 5.4), finding:

1. The three sets of algorithms are strict subsets of each other, making it immediate what their relative computational power is and allowing security arguments to directly apply from higher to lower levels in the hierarchy.
2. There is a well-defined notion of when variables "are" group elements, avoiding ambiguities in FKL's AGM framework arising from different interpretations of this notion.
3. Where an algorithm falls in the GGM/AGM/SM hierarchy is implicitly determined once it's written as per any of our frameworks.
4. The KZZ counterexamples do not apply to our interpretation of the lifting result (stated in terms of type-safe generic algorithms).

The way that our interpretation satisfies the second and fourth points naturally mirrors Zhandry's interpretation of the AGM in terms of Maurer's GGM and his observation that Maurer's GGM can be (and often implicitly is) thought of in terms of type safety.

Shoup-style generic group models are captured instead by a *semantic* change. In Shoup-style models, algorithms are fully allowed to view and operate on bitstrings representing the group elements just as in the standard model. So we consider the same syntactic set of "unrestricted" algorithms but now quantify over a random or worst-case selection of the group encoding function, giving "random-representation" and "all-representation" models.[4]

We believe there is a certain philosophic niceness to the fact that all of these existing models for group-based computation can be captured so cleanly by simple syntactic distinctions within a shared computation model (plus a clean semantic distinction for the Shoup-style models). Perhaps this indicates a deeper truth that they are in some sense "natural" models to study.

Our other frameworks (Sec. 5) swap out the underlying model of computation but provide similar levels. They are more directly inspired by the "type-safe" generic group models as formalized by Maurer [Mau05] (using oracles that perform group operations) and Zhandry [Zha22] (using circuits with special group element wires and gates). Algorithms expressed in the oracle framework are restricted to operating on group elements via invoking predefined oracles that match the special $\langle \textbf{elem} \rangle$ functions from the pseudocode framework. Group elements are stored in a hidden table by the oracle and algorithms are given labels of these objects (i.e., their indices in the table). Algorithms in the circuit framework combine traditional boolean circuits with special wires for holding group elements. These wires can only be used with special ga,tes which again match the special $\langle \textbf{elem} \rangle$ functions. In either case, we obtain the same type-safe, algebraic, unrestricted hierarchy.

(In)equivalence of the frameworks. An intended benefit of introducing these three different frameworks (beyond comparison to past representations of generic group computation) is to be able

[4] The standard (and algebraic) model quantifies the encoding function before algorithms are chosen and restricts attention to encodings from which group operations can be efficiently computed. The type-safe model is independent of the encoding.

to hop between them when performing different sorts of analysis, choosing whichever model is most suited to the job. For example, it is easiest to explicitly express security games, adversaries, and reductions in the pseudocode framework. The oracle framework is excellent for meta-analysis reliant on tracking information about the group operations being performed. We can transform oracle-based algorithms by creating new algorithms that emulate their oracles while tracking whatever information is of interest.[5] However, before we can rigorously hop between frameworks we must establish whether the three frameworks are as equivalent as they appear (Sec. 5.3).

Surprisingly, they are not! The inequivalence arises due to circuits not being able to choose between two different execution paths, which is trivial in the pseudocode or oracle-based frameworks (from if-then statements). We show this explicitly by observing that a circuit cannot compute $\texttt{mux}(X_0, X_1, b) = X_b$ where the X_i are group elements and b is a bit. Thinking of the circuit as a sort of graph, there will never be any paths from bit wires to element wires as the two gates producing element outputs do not take bit inputs. However, such dependence is clearly necessary to compute `mux`. Zhandry happened to consider a stronger gate set (with a combined group operation and exponentiation gate) that can compute `mux`, but this indicates that the circuit-based framework is more brittle than desired. Building from this, we establish that `mux` is the only difference between the frameworks; they are equivalent if `mux` is computable.

AGM to GGM lifting result. Having shown that the frameworks are essentially equivalent we then move to their analysis. The hierarchical nature of the frameworks makes the FKL lifting result natural (Sec. 6). There is, in fact, little to say about it as almost all of the properties one needs can be readily extracted from the presumed existing algebraic group model proof.

Let A be an algebraic algorithm and R a reduction that treats A as a black box. If R never uses encode or decode (i.e., is type-safe/generic), then the composition of R and A will be algebraic (resp. type-safe) if A is. Note that this is a purely syntactic claim about what the code of the two algorithms includes. Consequently, showing R works for all algebraic A implies it also necessarily does so for generic A because in our frameworks all generic algorithms are algebraic.

This framing closely matches how FKL presented their lemma originally. A crucial fork in the road leading to our different interpretation and result than KZZ is that we have well-defined notions of what variables are group elements and thus how a type-safe reduction can pass them as input to a not-necessarily-type-safe adversary. The KZZ result provided a reduction which is generic in the style of the Shoup GGM while our positive result considers reductions which are generic in the style of the Maurer GGM. We survey a number of papers that worked in the AGM and observe that their security games and reductions could fit within our frameworks and hence the lemma applies. It is straightforward to check if they do using our pseudocode framework as one simply imposes types on the variables of the existing algorithms and verifies whether encode or decode are ever being used to cast between element and non-element values.

Takeaways. One of the main takeaways from the combination of our work, Zhandry [Zha22], and KZZ [ZZK22] is that the lifting result does hold if the reduction is specifically Maurer-generic. Other than the contrived counterexamples from KZZ, we are not aware of any existing AGM reductions that aren't Maurer-generic. This supports Zhandry's proposal that the AGM should be restricted to security games that are Maurer-generic, because typically reductions have to internally execute the game code and thus will be Maurer-generic when the game is.

[5] We use this for relating our notion of algebraic with that of FKL by emulating the standard GGM proof technique of tracking how new group elements were derived from earlier ones.

1.2 Metacryptography

This work is a focusing on a sort of "metacryptography". Like KZZ [ZZK22], the focus is not on introducing new protocols, security proofs, or security definitions. Rather, we study the models through which cryptographic analyses are performed. Metacryptography, in this sense, is of course part of cryptography and is inherently a fuzzy concept. We nonetheless find it a useful concept to mention explicitly as it helps frame the discussion and brings into scope questions and ideas that otherwise remain invisible.

How models fall short. When analyzing cryptographic models we might broadly categorize the desiderata based on whether they are technical or non-technical. Non-technical desiderata include how easy are they to understand, how easy they are they to write proofs in (and how likely are those proofs to be incomplete or erroneous), and how likely are papers using them to get accepted.[6]

On the technical side, most discussions will fall under *soundness* and *completeness*. As stated in the contrapositive, an issue with completeness is one where there is some scheme that "should" be secure but cannot be shown to be so in our model. Examples here include the well-known [FKL18, Zha22, ZZK22] observation that the algebraic group model adds nothing over the standard model for security games with no group element outputs and Zhandry's observation [Zha22] that various types of primitives cannot be proven secure in the type-safe GGM.

An issue with soundness is one where a scheme should be considered insecure (e.g., there are known attacks in the standard model) but is deemed secure in an idealized model. Consider a contrived security notion where an algebraic algorithm is implicitly given the bit representation of a group element but not explicitly given the element. Then the adversary is artificially restricted because the element can *only* be output if the adversary can determine an explanation for it, despite already knowing its bit representation. A standard model adversary could output it trivially and perhaps use that to win its game.

A related concept of note is what we will call scope creep. Analysis of a model is performed in a specific setting. If the setting changes sufficiently without being re-evaluated, the model may break in unexpected ways. An example here is the equivalence between the type-safe and random-representation models shown by Jager and Schwenk [JS08]. It was proven in a restricted setting and is often interpreted as "these models are equivalent". However such equivalence can fail when the setting of analysis changes to one not originally covered, e.g., multi-stage games [Zha22] or when inputs depend on the encoding [SGS20]. It is important to understand the scope in which a result was proven and the underlying proof technique, to be able to identify when it no longer applies.

Ambiguous and illusory bugs. Let us introduce (again fuzzy) terminology for two sorts of "bugs".[7] An *ambiguous* bug arises when there are multiple ways to interpret a claim and the veracity thereof varies between the possible interpretations. The combination of our work and KZZ establishes that FKL's lifting result claim is an ambiguous bug. There is a valid interpretation of it (as identified by our work), but there are also interpretations by which it fails.

An *illusory* bug is one which in some sense goes away as soon as it is identified and understood. An example here is how the circuit-based model *is not* equivalent to the other models if `mux` is not computable. Zhandry's [Zha22] model happens to be able to compute `mux`, but there is no indication that anyone realized this was even a potential gap. It is common for models of oracle-based type-safe models to give only `op` and `gen` as the operation. Were these ported to the circuit-based setting, `mux` would be uncomputable.

[6] The last is perhaps not as philosophically relevant, but in practice certainly plays a role in which models are used.
[7] We loosely think here of "bug" as a catchall term for any sort of gap, error, omission, imprecision, etc. in analysis.

One may argue that any given ambiguous or illusory bug is not a bug at all. Perhaps the author had the right idea in mind all along and it was merely the reader who misinterpreted. Even if identified and new, such bugs will often be easily fixable. There is still value in finding and discussing them. The ambiguous bug at the intersection of our and KZZ's work leads to a more detailed study of precisely in which conditions can algebraic security results be lifted to the generic setting and a more general exploration of what generic algorithms mean. Hypothetically, follow-up work to Zhandry could have used his circuit-based model with only `op` and `gen` and thus been unable to compute very simple functions. Most likely, they'd work at a high level of abstraction implicitly adding `mux` without having realized they did so. However, there is always the possibility that they would perform a low-level proof that gives the "wrong" result because `mux` is excluded but in a way that this `mux` distinction remained hidden.

2 Notation and Preliminaries

If $\mathbf{l}$ is a vector, we let $\mathbf{l}_i$ or $\mathbf{l}[i]$ denote its i-th component. If X is a list, we overload notation by letting X also denote the set of elements contained in the list. For some $i, j \in \mathbb{Z}, i \leq j$, we denote the set of all values in $\mathbb{Z}$ between i and j inclusive as $[i, j]$, i.e., $[i, j] = \{i, i+1, \cdots, j\}$. We let $[j] = [1, j]$. If S is a set, then S^* is the set of all finite tuples of elements in S.

If G is a game that outputs a bit (typically representing a boolean), then $\Pr[\mathsf{G}]$ denotes the probability its output is 1. We identify $\{\mathsf{false}, \mathsf{true}\}$ with $\{0, 1\}$. We use $\leftarrow\!\!\$$ for randomized assignments (e.g., sampling uniformly from a set) and $\leftarrow$ for deterministic assignments.

Groups and encodings. A group $\mathbb{G}$ specifies a set of elements $\mathbb{G}.S$ and a group operation $\mathbb{G}.\mathtt{op} : \mathbb{G}.S \times \mathbb{G}.S \to \mathbb{G}.S$. It is required that the operation is associative, there exists an identity element, and every element has an inverse. When $\mathbb{G}$ is fixed, we use multiplicative notation so $AB = \mathbb{G}.\mathtt{op}(A, B)$, $A^3 = \mathtt{op}(A, \mathtt{op}(A, A))$, A^{-1} is the inverse of A, and A^0 is the identity element.

We will restrict our attention to cyclic groups with a known generator $\mathbb{G}.g$, often written as $g_{\mathbb{G}}$, of known order p. In this case, the group is necessarily isomorphic to the group $\mathbb{Z}_p^+$ (integers modulo p with the addition operation) under the isomorphism $I_{\mathbb{G}} : \mathbb{Z}_p \to \mathbb{G}.S$ defined by $a \mapsto g_{\mathbb{G}}^a$. The discrete logarithm of an element A, written $\mathtt{dlog}_{\mathbb{G}}(A)$ is the pre-image of A according to this map. We generally will use capital letters to denote a group element other than the generator and the corresponding lowercase letter to denote its discrete log.

For computation on a group, we sometimes encode the group elements as bitstrings. We call an injective function $\sigma_{\mathbb{G}} : \mathbb{G}.S \to \{0,1\}^{\sigma.n}$ an encoding. We let $\sigma_{\mathbb{G}}^{-1}$ denote its inverse and define $\sigma_{\mathbb{G}}^{-1}(\alpha) = \perp$ if α is not in the range of $\sigma_{\mathbb{G}}$. Then the bitstring representation of A is $\sigma_{\mathbb{G}}(A)$. We generally use Greek letters to denote the bitstring representation of a group element.[8]

For encodings used in practice, given $\sigma_{\mathbb{G}}(A)$ and $\sigma_{\mathbb{G}}(B)$ it is possible to efficiently compute $\sigma_{\mathbb{G}}(AB)$ and $\sigma_{\mathbb{G}}(A^{-1})$. Additionally, we typically desire that verifying whether there exists an $A \in \mathbb{G}.S$ such that $\alpha = \sigma_{\mathbb{G}}(A)$ should be efficient. We refer to such encodings as efficient encodings.

Algebraic group model. Given $\mathcal{G} = (\mathbb{G}, g, p)$ as the description of the underlying group, FKL provide the following definition.

Definition 1 (Algebraic algorithm [FKL18]). *An algorithm* $\mathsf{A}_{\mathsf{alg}}$ *executed in an algebraic game* $\mathbf{G}_{\mathcal{G}}$ *is called* algebraic *if for all group elements* $\mathbf{Z}$ *that* $\mathsf{A}_{\mathsf{alg}}$ *outputs (i.e. the elements in bold uppercase*

[8] We sometimes discuss examples from previous works that do not distinguish between group elements and the bitstring representation. We then follow the notational convention of the work.

letters), it additionally provides the representation of $\mathbf{Z}$ *relative to all previously recorded group elements. That is, if* $\overrightarrow{\mathbf{L}}$ *is the list of group elements* $\mathbf{L}_0, \ldots, \mathbf{L}_m \in \mathbb{G}$ *that* $\mathsf{A}_{\mathsf{alg}}$ *has received so far (w.l.o.g* $\mathbf{L}_0 = g$*), then* $\mathsf{A}_{\mathsf{alg}}$ *must also provide a vector* $\overrightarrow{z}$ *such that* $\mathbf{Z} = \prod_i \mathbf{L}_i^{z_i}$*. We denote such an output as* $[\mathbf{Z}_{\overrightarrow{z}}]$*.*

Here the group elements input and output to $\mathsf{A}_{\mathsf{alg}}$ are "syntactically distinguished" from other inputs and outputs.[9] The group elements are thought of as represented by their bitstring representation according to $\sigma_{\mathbb{G}}$.

3 Pseudocode Framework for Idealized Computation

Our goal is to define a framework for (Maurer) generic, algebraic, and standard models of computations as a hierarchy where each model is a literal subset of the next. We will eventually define and study the relationship between three such frameworks (corresponding to three ways prior work has conceptualized Maurer-style generic groups). In this section, we start with a pseudocode framework because it is the easiest to write security games and algorithms with. This allows us to efficiently compare our interpretation of FKL [FKL18] to that of KZZ [ZZK22].

In Sec. 3.1, we start by defining a notion of the "universe of computation" which allows our analysis to broadly apply to idealized models of computation beyond cyclic groups. Then in Sec. 3.2 we formalize our pseudocode framework and give the intuition for why it captures the desired models of computation (to be expanded on after we define the other frameworks in Sec. 5).

3.1 Idealized Models of Computation

We will specify a hierarchy of models of computations based on algebraic structures, building on the abstract models of computation introduced by Maurer [Mau05]. We fix a set S, a set Π of operations on S (functions of the form $f : S_\perp \times S_\perp \times \cdots \times S_\perp \times \{0,1\}^{f.n} \to S_\perp$), a set Σ of relations on S (functions of the form $\rho : S_\perp \times S_\perp \times \cdots \times S_\perp \times \{0,1\}^{\rho.n} \to \{0,1\}$), and an injection $\sigma : S_\perp \to \{0,1\}^{\sigma.n}$. Here we use $S_\perp$ to denote $S \cup \{\perp\}$ where $\perp \notin S$ is a new special symbol used to indicate some form of rejection or failure.

At the lowest level of our hierarchy (formalized precisely later) we consider the "type-safe generic" model wherein an algorithm can compute on bitstrings as normal, but can only compute on the elements of S by using the functions in Π and relations in Σ. The middle level of the hierarchy is the "algebraic" model that also depends on σ which we think of as specifying how the elements of S are encoded as bitstrings. At this level, algorithms may also apply σ to elements of S. However, algebraic algorithms still cannot apply σ^{-1}, so the only way to construct elements of S is via operations in Π. The highest level of our hierarchy is the standard model, wherein an algorithm is also allowed to use the decoding function σ^{-1}. The requirement that functions accept $\perp$ as input is necessary because we will assume that σ^{-1} outputs $\perp$ when given input not in σ's range.

For compactness of notation we combine S, Π, Σ, and σ as follows.

Definition 2 (Universe of Computation). *A* universe (of computation) U *specifies* $U.S$*,* $U.\Pi$*,* $U.\Sigma$*, and* $U.\sigma$ *with the above syntax and semantics. A* partial universe of computation U *leaves* $U.\sigma$ *unspecified.*

Consider defining a universe U for performing computations on a cyclic group $\mathbb{G}$. We would define $U.S = \mathbb{G}.S$ and $U.\sigma = \sigma_{\mathbb{G}}$. For predicates, it is typical to give only the equality predicate, so $U.\Sigma = \{\mathtt{eq}\}$ where $\mathtt{eq}(A \in S, B \in S)$ returns a bit indicating whether $A = B$. There are a

[9] Here we are paraphrasing FKL and make no claims about the precise intended interpretation of this phrase.

number of conventions used for what functions $U.\Pi$ should contain. The original type-safe generic group model of Maurer [Mau05] considered $U.\Pi = \{\mathtt{op}, \mathtt{const}\}$ where $\mathtt{op}(A \in S, B \in S) = AB$ and $\mathtt{const}(a \in \{0,1\}^n) = g_{\mathbb{G}}^a$, interpreting a as an integer in $\mathbb{Z}_p$ ($g_{\mathbb{G}}$ denotes the generator). He noted that in his analysis it often suffices to consider the "closure" $\overline{\Pi}$ of Π consisting of all functions that can be computed by Π on some fixed list of inputs in S. He always gave the generator as input, which we will instead model as the function $\mathtt{gen}() = g_{\mathbb{G}}$. One expects any Π with the same closure to be basically equivalent. Due to this, other functions have been used, including $\mathtt{inv}(A \in S) = A^{-1}$, $\mathtt{invop}(A \in S, B \in S, b \in \{0,1\}) = \mathtt{op}(A, B^{2b-1})$,[10] $\mathtt{exp}(A \in S, a \in \{0,1\}^n) = A^a$, $\mathtt{expop}(A \in S, B \in S, a \| b \in \{0,1\}^{2n}) = A^a B^b$, $\mathtt{multi\text{-}expop}(A_1 \in S, \ldots, A_m \in S, a_1 \| \ldots \| a_m \in \{0,1\}^{mn}) = \prod_i A_i^{a_i}$.

In the description above, we left unspecified how the functions and relations act on input $\perp$. In general, we only ever care about $\perp$ when it is obtained as the output of $\sigma_{\mathbb{G}}^{-1}$. Moreover, when $\sigma_{\mathbb{G}}$ is being thought of as a practically relevant encoding of a group, one can refrain from computing $\sigma_{\mathbb{G}}^{-1}$ on inputs not in the range of $\sigma_{\mathbb{G}}$ because there will be an efficient algorithm for checking if a given string is in that range which can be used first. When it is relevant, we think of $\mathtt{eq}$ as correctly checking equality with $\perp$. All functions discussed are defined to output $\perp$ if one or more of their inputs are $\perp$. It is broadly desirable for it to be possible to compute the relation $\mathtt{is\text{-}bot}$ defined by $\mathtt{is\text{-}bot}(A \in S_\perp) = 1$ iff $A = \perp$. One could add $\mathtt{is\text{-}bot}$ in Σ or emulate it using $\mathtt{eq}$ and functions in Π such as by noting that $\mathtt{is\text{-}bot}(A) = \mathtt{eq}(A, \mathtt{op}(A, \mathtt{gen}()))$ as long as $|\mathbb{G}.S| \neq 1$.

Appropriate other choices of U allow capturing generic/algebraic group models for non-cyclic groups [Mau05], generic/algebraic group action models [DHK+23], and generic/algebraic bilinear group models [MTT19, CH20] (among others).

It is not essential that the operations in $U.\Pi$ and $U.\Sigma$ are deterministic and only output a single elements/bit. Lipmaa, Parisella, and Siim [LPS25] cast the generic/algebraic group model with oblivious sampling into our framework by allowing randomized operations that output an element and multiple bits. Our results generalize naturally to cover this.

3.2 Pseudocode Computation Framework

For this framework, assume the existence of a typed programming (pseudocode) language $\mathcal{P}$ with pre-existing types including (but not necessarily limited to) integer $\langle\mathbf{int}\rangle$ and bit $\langle\mathbf{bit}\rangle$. Where useful, we assume the existence of "syntactic sugar" in $\mathcal{P}$ to make it easier to express complex algorithms. For instance, we use the convention that making any type plural (e.g., changing $\langle\mathbf{bit}\rangle \to \langle\mathbf{bits}\rangle$) indicates a vector of elements with that type. We also use a simplified syntax to represent data structures, control flow statements, and loops.

Our pseudocode-based framework for group computation extends this programming language by adding an additional restricted type $\langle\mathbf{elem}\rangle$ for elements of $U.S$. We introduce functions func_f for each $f \in \Pi$ and the relations rel_ρ for each $\rho \in \Sigma$ that can operate on $\langle\mathbf{elem}\rangle$ variables. In addition, there are also special functions for mapping our restricted $\langle\mathbf{elem}\rangle$ type to the unrestricted $\langle\mathbf{bits}\rangle$ (via the encoding function σ), and vice-versa. No other operations on $\langle\mathbf{elem}\rangle$ are allowed. These operations are shown in Fig. 1. We will use func_Π and rel_Σ as shorthand for all the collection of all operation func_f or rel_ρ.

Our pseudocode framework is as follows.

Definition 3 (Pseudocode Framework). *Fix universe U. Then*

- $\mathrm{TSA}_{\mathrm{P}}[U]$ *is the set of pseudocode algorithms that do not use* $\mathrm{encode}_{U.\sigma}$ *or* $\mathrm{decode}_{U.\sigma}$*;*
- $\mathrm{AA}_{\mathrm{P}}[U]$ *is the set of pseudocode algorithms that do not use* $\mathrm{decode}_{U.\sigma}$*; and*

[10] Note that $B^{2b-1} \in \{B, B^{-1}\}$.

Operation $\langle\textbf{elem}\rangle$ $\text{func}_f(\langle\textbf{elems}\rangle\ \mathbf{A}, \langle\textbf{bits}\rangle\ \mathbf{x})$:

return $f(\mathbf{A}, \mathbf{x})$

Operation $\langle\textbf{bit}\rangle$ $\text{rel}_\rho(\langle\textbf{elems}\rangle\ \mathbf{A}, \langle\textbf{bits}\rangle\ \mathbf{x})$:

return $\rho(\mathbf{A}, \mathbf{x})$

Operation $\langle\textbf{bits}\rangle$ $\text{encode}_\sigma(\langle\textbf{elem}\rangle\ A)$:

return $\sigma(A)$

Operation $\langle\textbf{elem}\rangle$ $\text{decode}_\sigma(\langle\textbf{bits}\rangle\ \alpha)$:

return $\sigma^{-1}(\alpha)$

Fig. 1: Pseudocode operations for type $\langle\textbf{elem}\rangle$. Here $f \in U.\Pi$, $\rho \in U.\Sigma$, and $\sigma = U.\sigma$.

– $\text{SA}_\text{P}[U]$ *is the set of all pseudocode algorithms.*

Algorithms in the three sets are, respectively, called type-safe*,* algebraic*, and* standard/unrestricted *algorithms.*

Note that the input-output syntax of functions in func_f and relations in rel_ρ are independent of the choice of σ. So an algorithm in any of the sets can syntactically be defined given a partial universe of computation. The partial universe is all that is required for the semantics of a type-safe algorithm to be defined. We use the term "type-safe (group) model" to refer to computation by a type-safe algorithm. Precise measurements of efficiency (e.g., should equality checks have unit cost or zero cost [Mau05, Zha22]) are inessential to our results, so we treat it informally.

The semantics of algebraic and standard algorithms require $U.\sigma$ to be fixed. We use the term "algebraic (group) model" or "standard (group) model" to refer to computation by an algebraic or standard algorithm, respectively, that was fixed after an efficient $U.\sigma$ was quantified.[11] Note that $\text{TSA}_\text{P}[U] \subset \text{AA}_\text{P}[U] \subset \text{SA}_\text{P}[U]$—they form a hierarchy. This literal hierarchy (more than the particular decision to describe things in terms of pseudocode) is what allows us to simply formalize the lifting result in Sec. 6.

Power of the language. So far, we have not nailed down any specifics of the computational power assumed of the programming language. We could formalize this by assuming that $\mathcal{P}$ is functionally equivalent to the Bellare-Rogaway example programming language for code-based games [BR06, Appendix B, ePrint]. Then $\mathcal{P}$ will be able to express any finite-time computable function.

For example, the Bellare-Rogaway context-free grammar has a "expressions" non-terminal *exp* (with rule $exp \to bool \mid int \mid str \mid set \mid array \mid call$) that defines the "types" and a non-terminal *identifier* with a rule for what variable names are allowed. Suppose we have fixed a universe of computation for cyclic groups with $U.\Sigma = \{\texttt{eq}\}$ and $U.\Pi = \{\texttt{op}, \texttt{gen}\}$. Our extensions to $\mathcal{P}$ can be formally modeled by adding the following rules to the Bellare-Rogaway context-free grammar.

$$
\begin{aligned}
exp &\longrightarrow elem \\
elem &\longrightarrow identifier \mid identifier[str] \mid \texttt{gen}() \mid \texttt{op}(elem, elem) \mid \text{decode}_\sigma(str) \\
str &\longrightarrow \texttt{eq}(elem, elem) \mid \text{encode}_\sigma(elem)
\end{aligned}
$$

With appropriately defined semantics for this added syntax, this will allow elements to be assigned to variables (named by identifiers), stored in arrays indexed by (bit)strings, taken as input by functions, and returned by functions.

[11] Shoup-style generic models will change this assumption of how $U.\sigma$ is quantified, either at random or in the worst case. In this setting, it is less confusing to refer to unrestricted, rather than standard algorithms

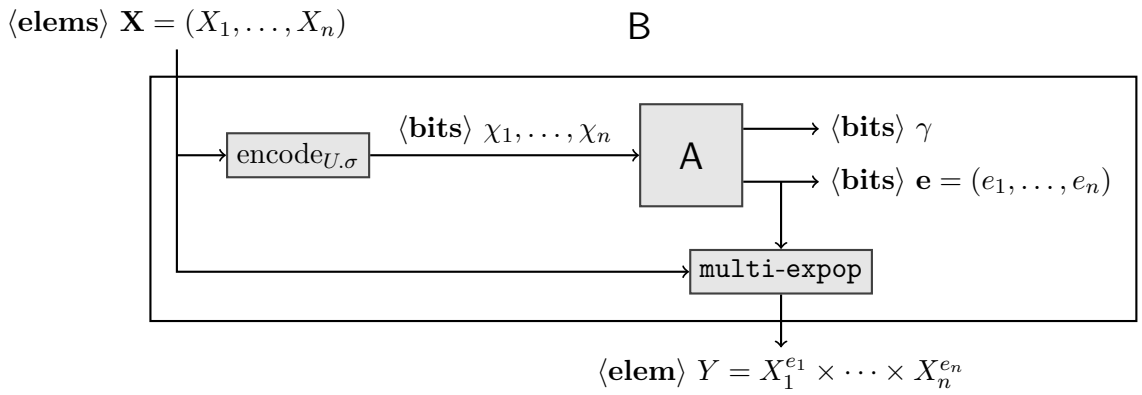


Fig. 2: Algorithm $\mathsf{B} \in \mathrm{AA}_{\mathrm{P}}[U]$ constructed from FKL-style AGM algorithm A

Intuition behind models. Let us give intuition for why these sets correspond to the existing notions of the Maurer generic group model, FKL algebraic group model, and standard model (when U is defined for a cyclic group).

We think this notion of the type-safe generic algorithms nicely captures how some previous work intuitively thinks about Maurer's GGM. Consider, for example, how some works [DHH+21, FKL18, RSS20, SGS20, SGS21] make implicit or syntactic distinctions between values that are group elements or strings, as observed by Zhandry [Zha22] when introducing his circuit-based version of Maurer's GGM. The fact that Zhandry called his circuit-based model "The Type-Safe Model" evokes direct comparison to concepts from programming languages. Meier [Mei23] and Ducas [Duc09] explicitly think about generic models in programming language terms.

The FKL [FKL18] definition of the AGM requires that when an algorithm A outputs a group element, it must also provide an exponent vector $\mathbf{e}$ explaining how the element could be computed from the group elements given as input. This algorithm is thought of as being specified in a typical model of computation (e.g., Turing machine or circuit). Its input group elements are represented as bitstrings, so X would be represented as $\sigma_{\mathbb{G}}(X)$.

Given such an A we can construct an "equivalent" $\mathsf{B} \in \mathrm{AA}_{\mathrm{P}}[U]$ (shown pictorially in Fig. 2). It uses $\mathrm{encode}_{U.\sigma}$ on each of the inputs in $\langle\mathbf{elem}\rangle\ \mathbf{X}$ to obtain their $\langle\mathbf{bits}\rangle$ representation, then it internally runs A on these inputs. We are assuming that A acts only on bitstrings and that our programming language is powerful enough to perform this internal emulation. When A outputs an explanation vector $\mathbf{e}$ (for an element represented by the bits γ), then B uses $\langle\mathbf{elem}\rangle$ operations on $\mathbf{X}$ (say, a single `multi-expop`) to obtain its output $\langle\mathbf{elem}\rangle\ Y = \prod_i X_i^{e_i}$.

In the other direction, suppose we are given $\mathsf{B} \in \mathrm{AA}_{\mathrm{P}}[U]$ and want to build an FKL-style B. It could use bitstring operations on the bitstring representations of group elements to emulate all of B's $\langle\mathbf{elem}\rangle$ operations. Note that we are using our assumption that $U.\sigma$ was fixed before A was defined and is efficient. While doing so, it can track the explanations of all of B's type $\langle\mathbf{elem}\rangle$ variables. For example, the operation $\mathrm{func}_{\mathtt{op}}(X, Y)$ results in an element whose explanation is the component-wise sum of X and Y's explanations (assuming $X, Y \neq \perp$).

The standard model example is analogous. Suppose A is a standard algorithm operating on and outputting bitstring representations of elements. Then $\mathsf{B} \in \mathrm{SA}_{\mathrm{P}}[U]$ uses $\mathrm{encode}_{U.\sigma}$ to represent input elements as bitstrings, runs A internally as an algorithm on these bitstrings, then uses $\mathrm{decode}_{U.\sigma}$ to cast the results back into type $\langle\mathbf{elem}\rangle$. In the other direction, a standard algorithm A (knowing the efficient $U.\sigma$) can emulate all of B's computation.

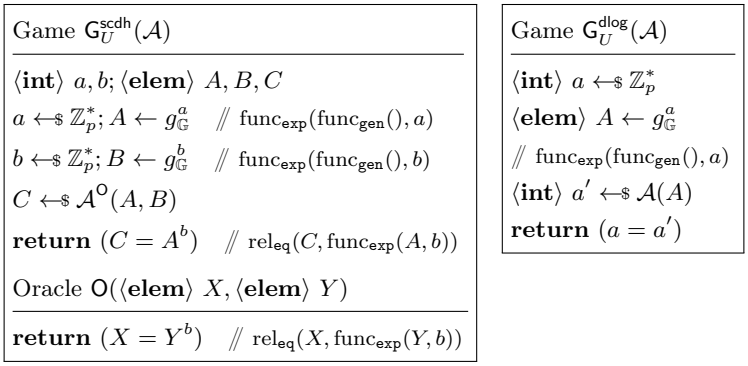


Fig. 3: Strong CDH and discrete log games expressed in the pseudocode framework

Examples. The primary benefit of the pseudocode framework (when compared to the frameworks we will give in Sec. 5) is that it most easily allows specifying algorithms using easy-to-parse pseudocode. As an example, let U be a universe of computation for a group $\mathbb{G}$ with $U.\Pi \supseteq \{\mathtt{gen}, \mathtt{exp}\}$ and $U.\Sigma \supseteq \{\mathtt{eq}\}$. Then we define Strong Computational Diffie Hellman (CDH) and Discrete Logarithm security via the games $\mathsf{G}^{\mathsf{scdh}}_U$ and $\mathsf{G}^{\mathsf{dlog}}_U$ shown in Fig. 3. We use notation like $g_{\mathbb{G}}$, $g^a_{\mathbb{G}}$, and $=$ as shorthand for specific U operations (defined in the comments).

Note that syntactically defining the games only requires U to be a partial universe of computation. Defining $U.\sigma$ adds semantics to $\mathcal{A}$'s behavior. Implicitly, $\mathcal{A}$ is run using the same U as the game. For a given universe U and adversary $\mathcal{A}$ we define the advantages $\mathsf{Adv}^{\mathrm{scdh}}_U(\mathcal{A}) = \Pr[\mathsf{G}^{\mathsf{scdh}}_U(\mathcal{A})]$ and $\mathsf{Adv}^{\mathrm{dlog}}_U(\mathcal{A}) = \Pr[\mathsf{G}^{\mathsf{dlog}}_U(\mathcal{A})]$. Normal CDH security is captured by considering $\mathcal{A}$ playing Strong CDH that never queries its oracle. By techniques of FKL, (Strong) CDH security against algebraic algorithms reduces to discrete logarithm security.

3.3 Shoup Generic Algorithms

We describe a way to view Shoup generic models when considering our framework. Note that we will have to describe both *syntactically* what actions an algorithm can take and *semantically* how these actions are interpreted.

Typical descriptions of Shoup [Sho97] generic algorithms provide them access to an oracle for group operations with respect to an encoding function σ. The oracle takes in two bitstrings α and β then returns $\sigma(\mathtt{op}(\sigma^{-1}(\alpha), \sigma^{-1}(\beta)))$. Consequently, a Shoup generic algorithm is given direct access to the bitstring representations of group elements and may apply whatever bit operations it desires to them, as well as interpret arbitrary strings as being group elements. This matches what an unrestricted algorithm is allowed to do. Using encode_σ and decode_σ, unrestricted algorithms can always map between $\langle\mathbf{elem}\rangle$ variables for computing the group operation and $\langle\mathbf{bits}\rangle$ variables for applying the bit operations of its choice. Indeed an unrestricted model algorithm can simulate the described oracle as $\mathrm{encode}_\sigma(\mathrm{func}_{\mathtt{op}}(\mathrm{decode}_\sigma(x), \mathrm{decode}_\sigma(y))$. An algorithm on bits with access to the oracle could similarly emulate the view of an unrestricted algorithm.

Rather than introducing a separate oracle, we will treat Shoup generic-ness as syntactically captured by unrestricted algorithms,[12] meaning we allow all algorithms in $\mathrm{SA}_{\mathrm{P}}[U]$. The operations on $\langle\mathbf{elem}\rangle$ play the role of the oracles. Shoup models fix different semantics of how $U.\sigma$ is picked.

Definition 4 (Informal). *Fix partial universe U and let $\mathsf{A} \in \mathrm{SA}_{\mathrm{P}}[U]$. In the* fixed-representation model *(i.e., standard model), we measure the success of A with respect to an a priori known efficient*

[12] We refer to them both as "standard" and "unrestricted" to avoid Shoup generic models using "standard" algorithms.

$U.\sigma$. In the random-representation model*, we measure the success of* A *with respect to a randomly chosen $U.\sigma$ (over some reasonable distribution). In the* all-representation model*, we measure the success of* A *with respect to the $U.\sigma$ for which it performs worst.*

The phrases "measure how well" and "performs worst" are intentionally vague, as the success of algorithms is measured differently for different types of algorithms. Consider when the algorithm is an adversary, a reduction, or a simulator; each would define success differently. There would inevitably be scope creep beyond any particular definition of success we provide. Typically, the random distribution is uniform over all $\sigma : \mathbb{G}.S \to \{0,1\}^l$ for a fixed l.

Example 1. For Strong CDH, in the fixed-representation model, we use a known $U.\sigma$ and measure $\mathsf{Adv}^{\text{scdh}}_U(\mathcal{A})$. In the random-representation model we measure $\mathbb{E}_{U.\sigma}[\mathsf{Adv}^{\text{scdh}}_U(\mathcal{A})]$. In the all-representation model we measure $\min_{U.\sigma} \mathsf{Adv}^{\text{scdh}}_U(\mathcal{A})$.

Shoup [Sho97] used both the random- and all-representation models. Successful algorithms (i.e., algorithms with non-negligible advantage) were stated in terms of the all-representation model while the random-representation model was used to bound how well an algorithm can perform.

4 AGM Interpretations and Metacryptography

With our pseudocode framework in place, we can compare the resulting interpretation of the AGM to that discussed in other works. Along the way, we apply the metacryptographic terminology (e.g., soundness, completeness, ambiguous, illusory) from Sec. 1.2 to help frame precisely what is being discussed.

No output group elements. It's well known [FKL18, Zha22, ZZK22] that FKL's AGM is equivalent to the standard model for algorithms that do not output group elements. The same holds for our framework.[13] Suppose $\mathsf{A} \in \mathrm{SA}_{\mathrm{P}}[U]$ has inputs of both type $\langle\textbf{elem}\rangle$ and $\langle\textbf{bits}\rangle$ but only produces outputs of type $\langle\textbf{bits}\rangle$. Internally, A may obtain new $\langle\textbf{elem}\rangle$ variables from its $\langle\textbf{bits}\rangle$ inputs using $\mathrm{decode}_{U.\sigma}$ and subsequently apply operations from $U.\Pi$ on these.

Assuming a fixed, efficient U, σ, we can build an equivalent $\mathsf{B} \in \mathrm{AA}_{\mathrm{P}}[U]$. Note that B can perform any of the computation steps of A, except $\mathrm{decode}_{U.\sigma}$. To avoid this, we replace all of A's computations on $\langle\textbf{elem}\rangle$ variables with the corresponding computation using their bitstring representations. In particular, B first applies $\mathrm{encode}_{U.\sigma}$ to all input group elements. When A performs any of the operations in $U.\Pi$ or $U.\Sigma$, we do the same operation using the bitstring representations. When A performs $\mathrm{encode}_{U.\sigma}$ or $\mathrm{decode}_{U.\sigma}$ we leave it internally represented as a bitstring.

This limitation of algebraic group models indicates that it is less *complete* than generic group models. For example, an encryption scheme may be (correctly) proven secure in generic models but for which no typical standard model (and hence no algebraic model) hardness assumption is known to suffice.

Deriving new group elements. Before formally defining their AGM, FKL [FKL18, p.6, ePrint] say "Intuitively, the only way for an algebraic algorithm to output a new group element Z is to derive it via group [operations] from known group elements." KZZ give an example algorithm that is algebraic according to FKL but seemingly does not match this intuition. For their example, we fix

[13] It is worth noting that we are currently focusing on "single-shot" algorithms that are run once and produce output based on the given input. If instead, e.g., an algorithm is run multiple times to interact with a security game, this equivalence only holds if *all* outputs (including intermediate queries to the security game) are not group elements.

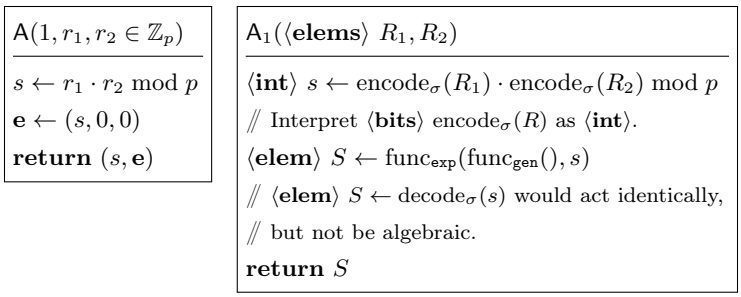


Fig. 4: An example algorithm introduced by KZZ which is algebraic per FKL but "intuitively" non-algebraic (left) and the natural, "intuitively" algebraic pseudocode interpretation of it (right)

U to be a universe of computation for the cyclic group $\mathbb{Z}_p$. In particular let $U.S = \mathbb{Z}_p$, $U.\Sigma = \{\texttt{eq}\}$, $U.\Pi = \{\texttt{gen}, \texttt{op}, \texttt{exp}\}$, and $U.\sigma$ which maps a number $r \in \mathbb{Z}$ to its $\lceil \log(p) \rceil$-bit binary representation. Note that $\texttt{gen}() = 1 \in \mathbb{Z}_p$, $\texttt{op}(r_1, r_2) = r_1 + r_2 \bmod p$ and $\texttt{exp}(r, a) = a \cdot r \bmod p$.

The KZZ example A is shown on the left of Fig. 4 (the original algorithm picked r_1, r_2 at random, but this distinction is inessential). Note that this algorithm is not in our pseudocode framework; it works in a more standard notation where all code represents appropriately defined computation on bitstrings. The algorithm is given the generator of the group $\mathbb{Z}_p$ as well as two elements of the group. These elements are then *multiplied* (recall the group operation is addition) to obtain $s \in \mathbb{Z}_p$, which is output together with a valid explanation for it. Note that when using $\mathbb{Z}_p$ if 1 is the first group element input to an algorithm, then $(s, 0, 0, \ldots, 0)$ will always be a valid explanation for the group element s.

In the interpretation of KZZ, this algorithm does not match the intuition for an algebraic algorithm as it derives the new group element s via multiplication, which is not the group operation in the group. Let us provide an alternate interpretation of the intuition for algebraic algorithms [FKL18, PV05].

Remark 1. We think of an algorithm as "intuitively" algebraic if all new group elements are obtained via the group operation, *but* arbitrary computations on group elements and their bitstring encodings can be used to decide what group operations to perform.

This is captured by A_1 in Fig. 4, which gives our interpretation of A as an algorithm in $\text{AA}_\text{P}[U]$, which follows the general approach sketched for converting FKL algebraic algorithms into ones algebraic per our framework.

Note that when we impose a type system on algorithms written without one, there are multiple ways to do so which produce "valid" instantiations. As an example, one of the comments in A_1 shows how it could instead have been defined as a non-algebraic algorithm in $\text{SA}_\text{P}[U] \setminus \text{AA}_\text{P}[U]$. Alternatively, it could have been interpreted as an operation purely on type $\langle\textbf{int}\rangle$ variables. There should be no expectation that *all* instantiations of a non-typed algorithm will satisfy some property; the question is whether there *exists* one that does.

As observed by KZZ, if discrete logarithms are easy in a group then any algorithm can be interpreted as algebraic for that group by these conventions. This is true and not an issue. The AGM is used for analyzing relationships between the hardness of problems with respect to some group. An AGM reduction, say, from CDH to discrete log is thus *correctly* identifying that CDH is no harder than discrete log in such groups. Given that, we consider this an illusory issue.

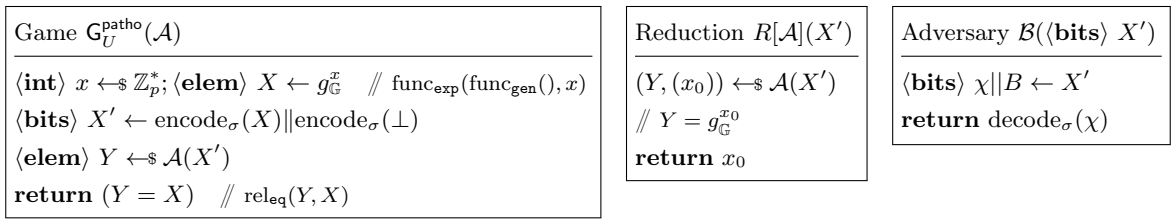


Fig. 5: Pathological (non-type-safe) game, reduction R (for FKL-style algebraic $\mathcal{A}$), and $\mathcal{B} \in \mathrm{SA}_{\mathrm{P}}[U]$ establishing universal unsoundness of the algebraic group model

Index calculus. KZZ also question the FKL interpretation that index-calculus algorithms are algebraic. The above discussion gives a natural interpretation of this idea. Consider a universe of computation where $U.S$ is a cyclic subgroup of $\mathbb{Z}_q^*$, $U.\Sigma = \{\mathtt{eq}\}$, $U.\Pi = \{\mathtt{gen}, \mathtt{op}, \mathtt{exp}\}$, and $U.\sigma$ maps a number $r \in \mathbb{Z}$ to its $\lceil\log(r)\rceil$-bit binary representation. Here $\mathtt{gen}()$ outputs a generator g for $U.S$ and $\mathtt{op}, \mathtt{exp}$ perform multiplication and exponentiation modulo q. Let IC be an algorithm which on input the bitstring representations of g, q, and a group element A uses index-calculus methods to calculate $\mathtt{dlog}_{\mathbb{G}}(A)$.

Then $\mathcal{A}_{\mathsf{cdh}}$ which on input $\langle\mathbf{elems}\rangle\ A, B$ computes $a \leftarrow\!\$\ \mathsf{IC}(\mathrm{encode}_\sigma(\mathtt{gen}()), q, \mathrm{encode}_\sigma(A))$ and returns $\mathrm{func}_{\mathtt{exp}}(B, a)$ is an algorithm in $\mathrm{AA}_{\mathrm{P}}[U]$. The fact that its success can be bounded by that of a related discrete log attacker is a natural result captured by the AGM [FKL18].

Lack of formalism of group element inputs. FKL gave an example of an algorithm $\mathsf{A}_{\mathsf{alg}}$ which will always output $\mathbf{X}$ when given $X' = \mathbf{X}\|\bot$ where $\mathbf{X}$ is (the bitstring representation of) a group element. If $\mathsf{A}_{\mathsf{alg}}$ is FKL algebraic, then it must also be outputting an explanation of $\mathbf{X}$ in terms of $g_{\mathbb{G}}$. Note that input X' is a bitstring, not a group element, and so is not part of the explanation. This explanation will give the discrete log of $\mathbf{X}$. FKL say the following.

> Allowing inputs of form X' while requiring algorithms to be algebraic leads to contradictions. (E.g., one could use $\mathsf{A}_{\mathsf{alg}}$ to compute discrete logarithms: given a challenge $\mathbf{X} = g^x$, run $[\mathbf{X}]_x \leftarrow\!\$\ \mathsf{A}_{\mathsf{alg}}(\mathbf{X}||\bot)$ and return x.) We therefore demand that non-group-elements inputs must not depend on group elements. [FKL18, p.7, ePrint]

From the metacryptographic perspective, let us note that the parenthetical example is not by itself a "contradiction" here. Indeed, one can define such an algebraic $\mathsf{A}_{\mathsf{alg}}$ for a given U if and only if discrete logarithms are easy to compute in the group. This can be nicely phrased as an issue of soundness (i.e., *contradicting* an implicit assumption of soundness).

Consider the game $\mathsf{G}^{\mathsf{patho}}_U$ defined in Fig. 5 for which we define $\mathsf{Adv}^{\mathrm{patho}}_U(\mathcal{A}) = \Pr[\mathsf{G}^{\mathsf{patho}}_U(\mathcal{A})]$. Fix a partial U for cyclic group computations. Then it is clear that (via our equivalence with the FKL AGM) we can define an efficient reduction $R : \mathrm{AA}_{\mathrm{P}}[U] \to \mathrm{AA}_{\mathrm{P}}[U]$ such that for all adversaries $\mathcal{A} \in \mathrm{AA}_{\mathrm{P}}[U]$ and encodings $U.\sigma$ we have $\mathsf{Adv}^{\mathrm{patho}}_U(\mathcal{A}) \le \mathsf{Adv}^{\mathrm{dlog}}_U(R[\mathcal{A}])$.[14] We've shown this for an FKL-style algebraic $\mathcal{A}$. On the other hand, the standard adversary $\mathcal{B} \in \mathrm{SA}_{\mathrm{P}}[U]$ defined in the same figure clearly achieves $\mathsf{Adv}^{\mathrm{patho}}_U(\mathcal{B}) = 1$.[15] Hence the algebraic group model is misclassifying the security of the game for all $U.\sigma$ for which discrete logarithms are hard (i.e., all $U.\sigma$ for which we hope the model gives us a meaningful result). Consequently, the model is universally unsound with respect to $\mathsf{Adv}^{\mathrm{patho}}_U$.

[14] We typically think of $U.\sigma$ quantified before algebraic algorithms. Quantifying the reduction first makes for a stronger statement.

[15] Indeed, this is the case for all $U.\sigma$ so $\mathcal{B}$ is a successful all-representation generic adversary.

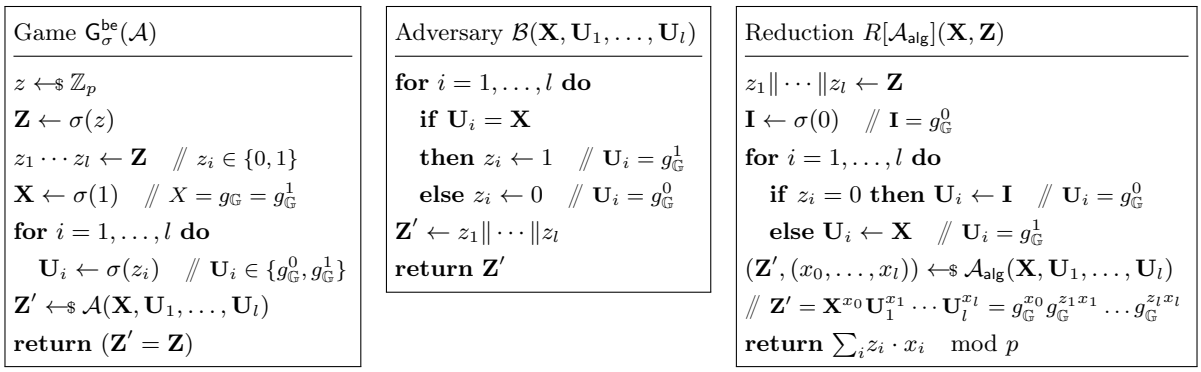


Fig. 6: The KZZ binary encoding game, generic adversary, and generic AGM reduction

In our interpretation, it is natural to formalize that "non-group-element inputs must not depend on group elements" by restricting attention to security games whose "challenger" can be expressed as an algorithm in $\mathrm{TSA}_{\mathrm{P}}[U]$. This will exclude $\mathsf{G}^{\mathsf{patho}}_U$ as it runs encode_σ. This us precisely Zhandry's proposal that the AGM be restricted to type-safe games. Even with this restriction, Zhandry [Zha22] is able to show there still exist instances of universal unsoundness by giving a one-time MAC whose AGM security reduces to discrete log, but which is insecure in the standard model for any $U.\sigma$ for which `expop` is efficiently computable on the bitstring representations of elements. We do not care about $U.\sigma$ for which this is not the case. This universal unsoundness is similar to examples of universal unsoundness in the random oracle [CGH98] and generic group models [Den02] in that it involves the game interpreting and running attacker-chosen input as code. With the duality between code and data, we cannot hope to formally specify a division between games that will or will not allow such behavior. We instead (as with the random oracle and generic group models) use informal interpretation to disallow such games.

Algebraic to generic lifting result. KZZ's main result was a counter-example security game showing that security proven in the AGM via "generic" reductions do not necessarily imply hardness in the Shoup GGM in their interpretation. The binary encoding game $\mathsf{G}^{\mathsf{be}}_{\mathbb{G}}$ is shown in Fig. 6. Define $\mathsf{Adv}^{\mathrm{be}}_{\sigma}(\mathcal{A}) = \Pr[\mathsf{G}^{\mathsf{be}}_{\sigma}(\mathcal{A})]$.

Following KZZ, the code is written in non-typed pseudocode representing computation on bitstrings. It uses the underlying group $\mathbb{Z}_p$ with bitstring representations defined by the encoding function σ. The game encodes a random group element $\mathbf{Z}$ as bits z_i, then further encodes those bits as $\mathbf{U}_i = g^{z_i}_{\mathbb{G}}$. The adversary $\mathcal{B}$ checks the value of each $\mathbf{U}_i$ to determine z_i, then concatenates these bits to obtain its output $\mathbf{Z}'$, which will definitely match the original $\mathbf{Z}$, so $\min_\sigma \mathsf{Adv}^{\mathrm{be}}_{\sigma}(\mathcal{B}) = 1$ and KZZ consider this a successful (all-representation) generic attack. Next to $\mathcal{B}$ is a reduction R which, given a FKL-style algebraic $\mathcal{A}_{\mathsf{alg}}$, uses its input $\mathbf{Z}$ from the discrete log game (here $\mathbf{X} = g_{\mathbb{G}}$) to simulate the view of $\mathcal{A}_{\mathsf{alg}}$. To correctly output $\mathbf{Z}$, $\mathcal{A}_{\mathsf{alg}}$ must also output its explanation. This can be used to calculate the discrete log of $\mathbf{Z}$. For each σ, $\mathsf{Adv}^{\mathrm{be}}_{\sigma}(\mathcal{A}_{\mathsf{alg}}) \leq \mathsf{Adv}^{\mathrm{dlog}}_{U}(R[\mathcal{A}_{\mathsf{alg}}])$ so KZZ consider this a successful (all-representation) generic reduction. Because there exists both a generic reduction to discrete log and a successful generic attack, the FKL lifting result does not hold for this interpretation.

To be a counterexample for the type-safe model, we would need to assign types to the variables in G^{be} and have both $\mathcal{B}$ and R be generic. Consider the variable $\mathbf{Z}'$. If its type is $\langle\mathbf{elem}\rangle$, then $\mathcal{B}$'s creation of it will require the use of decode_σ—making $\mathcal{B}$ not generic. If its type is $\langle\mathbf{bits}\rangle$, then $\mathcal{A}$ will be completely unconstrained in how it outputs the bits (see the first discussion in this section)

and thus it is of no use to a reduction R. With this view, either $\mathcal{B}$ or R must fail and hence a type-safe lifting result is not ruled out. We provide one in Sec. 6. So the issue argued by KZZ can be considered an ambiguous bug; it depends on one's interpretation.

Separately, we could rule out this example by being strict about the convention that security games should be type-safe; as discussed above, this is the approach taken by Zhandry [Zha22]. In that case, the operations $z_1 \cdots z_l \leftarrow \mathbf{Z}$ and $(\mathbf{Z}' = \mathbf{Z})$ together require that either $\mathbf{Z}'$ is type $\langle \mathbf{bits} \rangle$ (meaning $\mathcal{A}$ has no group element outputs) or it is type $\langle \mathbf{elem} \rangle$ and an encode_σ or decode_σ operation is required for $\mathbf{Z}$. This non-type-safe game would give an example of universal unsoundness à la $\mathsf{G}^{\mathsf{patho}}_U$. The fact that this game is non-type-safe underlies why the KZZ reduction needs to be non-type-safe; it needs to simulate the game for the adversary that it runs internally.

5 Alternate Frameworks and Framework Analysis

Here we describe in detail our other frameworks. In Sec. 5.1, we describe a framework inspired by Maurer's GGM formalism [Mau05] that accesses and operates on elements of $\mathbb{G}.S$ via queries to special oracles. In Sec. 5.2 we describe a framework for representing algorithms as circuits having special types of wires for elements of $\mathbb{G}.S$ with special gates for each function in Π and Σ. This framework emulates and extends the formalism defined by Zhandry in his Type-safe GGM instantiation [Zha22].

5.1 Oracle Computation Framework

We start by describing a framework using special oracles that perform restricted computations. This framework can be thought of as an extended variant of Maurer's formalism of abstract models of computation [Mau05].

In this framework, we assume the existence of some underlying notion of Turing machines $\mathcal{M}$, which operate on the binary alphabet and can make oracle queries. The oracles will store elements of S in a table V indexed by labels of the algorithm's choice. When it wishes a function to be computed on elements of S it specifies the labels of the elements to be used. Formally, we define oracles for each function $f \in \Pi$ or $\rho \in \Sigma$ and for σ as shown in Fig. 7.

Oracles $\mathsf{O}_f(\ell_1, \ldots, \ell_n, x, \ell_{\mathrm{out}})$	Oracles $\mathsf{O}_\rho(\ell_1, \ldots, \ell_n, x)$	Oracle $\mathsf{Encode}_\sigma(\ell_1)$	Oracle $\mathsf{Decode}_\sigma(\alpha, \ell_{\mathrm{out}})$
$V[\ell_{\mathrm{out}}] \leftarrow f(V[\ell_1], \ldots, V[\ell_n], x)$	**return** $\rho(V[\ell_1], \ldots, V[\ell_n], x)$	**return** $\sigma(V[\ell_1])$	$V[\ell_{\mathrm{out}}] \leftarrow \sigma^{-1}(\alpha)$
return			**return**

Fig. 7: Oracles that act on elements of S. Elements are stored in the table V. Every variable given as input to an oracle is a bitstring. Here $f \in U.\Pi$, $\rho \in U.\Sigma$, and $\sigma = U.\sigma$.

Here we have used the convention of having ℓ_1 through ℓ_n denote the labels for input elements of S, having x denote the input bitstring, and having ℓ_{out} denote the label for the output element. We emphasize that the algorithm calling these oracles is required to be specified in $\mathcal{M}$, but we are agnostic to the details of how the oracles are implemented.

We write O_Π to denote all oracles O_f and O_Σ to denote all oracle O_ρ. Now we define our hierarchy of algorithms for this framework.

Definition 5 (Oracle Framework). *Fix a universe of computation U. Then*

- $\mathrm{TSA}_{\mathrm{O}}[U]$ *is the set of all oracle algorithms (in* $\mathcal{M}$*) that expect access to the oracles* $\mathsf{O}_{U.\Pi}$ *and* $\mathsf{O}_{U.\Sigma}$*;*
- $\mathrm{AA}_{\mathrm{O}}[U]$ *is the set of all oracle algorithms that expect access to the oracles* $\mathsf{O}_{U.\Pi}$*,* $\mathsf{O}_{U.\Sigma}$*, and* $\mathsf{Encode}_{U.\sigma}$*; and*
- $\mathrm{SA}_{\mathrm{O}}[U]$ *is the set of all oracle algorithms that expect access to the oracles* $\mathsf{O}_{U.\Pi}$*,* $\mathsf{O}_{U.\Sigma}$*,* $\mathsf{Encode}_{U.\sigma}$*, and* $\mathsf{Decode}_{U.\sigma}$*.*

Algorithms in these three sets are, respectively, called type-safe*,* algebraic *and* standard/unrestricted *algorithms.*

Similar to $\mathrm{TSA}_{\mathrm{P}}$, the input-output syntax of O_{Σ} and O_{Π} are independent of the choice of σ so it suffices to consider a partial universe of computation for $\mathrm{TSA}_{\mathrm{O}}$.

This framework is essentially equivalent to the abstract model of computation introduced by Maurer [Mau05] with the oracle playing the role of the black box $\mathbf{B}$ and the table V playing the role of the variables $V_1, \ldots, V_m$. One benefit of this formalism is that the oracles allow explicitly simulating the view of an adversary with another algorithm that computes the required functions while tracking something about the queries that are made.

Algorithm inputs and outputs. With this framework, one needs to provide conventions for what it means for an algorithm to be given elements inputs/output elements and, more generally, how group elements are passed between multiple interacting algorithms. We will assume input elements A_i are written into known, fixed entries of the table corresponding to the label $\ell_{\mathrm{input},i}$, and that when there are multiple oracle algorithms, they each have independent tables V. The execution environment copies elements between tables when one algorithm's output is another's input.

Consider an algorithm that is supposed to output $(A_1, \ldots A_n, y)$ where the A_i are elements and y is a bitstring. We will consider two conventions. In the first convention, (the "permissive" convention) we have the adversary output a tuple $(\ell_1, \ldots, \ell_n, y)$ of bitstrings, where each ℓ_i is a label indicating which entry of V stores the desired output elements. In the second convention, (the "strict" convention) there are some a priori fixed labels $\ell_{\mathrm{output},i}$, one for each output, and the algorithm's i-th output is considered to be $V[\ell_{\mathrm{output},i}]$.

We think of the permissive convention as the "correct" convention because (for subtle reasons that will be discussed later) it turns out that for some U the strict convention can result in a weaker model of computation. We will give an example U later and then prove some conditions on U under which the two conventions are ensured to be equivalent. When we want to discuss strict convention we will write $\mathrm{XA}_{\mathrm{SO}}[U]$ for $\mathrm{X} \in \{\mathrm{TS}, \mathrm{A}, \mathrm{S}\}$.

Maurer [Mau05] considers three specific types of problems called "extraction", "computation", and "distinction". Of these, only computation requires outputting an element and Maurer uses the strict output convention.

5.2 Circuit Computation Framework

Next, we present a circuit-based framework that is inspired by Zhandry's Type-Safe formalism of the generic group model [Zha22]. In this framework, we assume some underlying notion of circuit-based computation $\mathcal{C}$, wherein the circuits compute on wires that store bits using a universal gate set (say, `nand`). Then our framework adds an additional type of wire that stores elements of S (or $\perp$). Functions in Π or Σ as well as σ are computed by special gates. Altogether, the gate types allowed in our framework are defined as shown in Fig. 8.

We use $\mathtt{G}_{\Pi}$ and $\mathtt{G}_{\Sigma}$ as shorthand for all gates $\mathtt{G}_f$ and $\mathtt{G}_{\rho}$. Now we can formally define our hierarchy of circuits (algorithms) for this framework.

Gates $\mathtt{G}_f$	Gate $\mathtt{encode}_\sigma$	Gate $\mathtt{nand}$
$\mathtt{G}_f : S \times \cdots \times S \times \{0,1\}^{f.n} \to S$	$\mathtt{encode}_\sigma : S \to \{0,1\}^{\sigma.n}$	$\mathtt{nand}(a,b) : \{0,1\} \times \{0,1\} \to \{0,1\}$
$\mathtt{G}_f(A_1, \ldots, A_n, x) = f(A_1, \ldots, A_n, x)$	$\mathtt{encode}_\sigma(A) = \sigma(A)$	$\mathtt{nand}(a,b) = \neg(a \wedge b)$
Gates $\mathtt{G}_\rho$	**Gate $\mathtt{decode}_\sigma$**	
$\mathtt{G}_\rho : S \times \cdots \times S \times \{0,1\}^{\rho.n} \to \{0,1\}$	$\mathtt{decode}_\sigma : \{0,1\}^{\sigma.n} \to S$	
$\mathtt{G}_\rho(A_1, \ldots, A_n, x) = \rho(A_1, \ldots, A_n, x)$	$\mathtt{decode}_\sigma(\alpha) = \sigma^{-1}(\alpha)$	

Fig. 8: Gates acting on bits or element wires. Here $f \in U.\Pi$, $\rho \in U.\Sigma$, and $\sigma = U.\sigma$.

Definition 6 (Circuit Framework). *Fix a universe of computation U. Then*

- $\mathrm{TSA}_\mathrm{C}[U]$ *is the set of all $\mathcal{C}$ circuits that additionally use gates $\mathtt{G}_{U.\Sigma}$ and $\mathtt{G}_{U.\Pi}$;*
- $\mathrm{AA}_\mathrm{C}[U]$ *is the set of all $\mathcal{C}$ circuits that additionally use gates $\mathtt{G}_{U.\Sigma}$, $\mathtt{G}_{U.\Pi}$, and $\mathtt{encode}_{U.\sigma}$; and*
- $\mathrm{SA}_\mathrm{C}[U]$ *is the set of all $\mathcal{C}$ circuits that additionally use gates $\mathtt{G}_{U.\Sigma}$, $\mathtt{G}_{U.\Pi}$, $\mathtt{encode}_{U.\sigma}$, and $\mathtt{decode}_{U.\sigma}$.*

Algorithms represented by circuits in these three sets are, respectively, called type-safe, algebraic *and* standard/unrestricted *algorithms.*

5.3 Inter-Framework Equivalence

We now analyze and compare our frameworks at an algorithmic level. We ask whether the sets of algorithms expressible in the different frameworks are "equivalent" to each other and to prior notions of the generic group model, algebraic group model, and standard model.

We will first formally specify our notion of equivalence between frameworks. For that, it will be useful to introduce some terminology. If $\mathcal{S}[U]$ is a set of algorithms for universe of computation U and $f : U.S^{f.m} \times \{0,1\}^{f.n} \to U.S^{f.o} \times \{0,1\}^{f.p}$ is a function, we say $\mathcal{S}[U]$ computes f if there exists $\mathsf{A} \in \mathcal{S}[U]$ which on input $(\mathbf{X}, s)$ outputs $f(\mathbf{X}, s)$.

Definition 7 (Equivalence). *Consider two sets of algorithms $\mathcal{S}[U]$ and $\mathcal{S}'[U]$ for universe of computation U. We say $\mathcal{S}$ and $\mathcal{S}'$ are* equivalent *iff the set of functions $\mathcal{S}[U]$ computes is the same as the set of functions $\mathcal{S}'[U]$ computes.*

A natural approach is to ask whether every algorithm $\mathsf{A} \in \mathrm{XA}_\mathrm{Y}[U]$ can be converted into an algorithm $\mathsf{B} \in \mathrm{XA}_\mathrm{Z}[U]$ that produces the same outputs when given the same inputs. Here, $\mathrm{X} \in \{\mathrm{TS}, \mathrm{A}, \mathrm{S}\}$ specifies whether both A and B are type-safe generic algorithms, algebraic algorithms, or unrestricted algorithms and $\mathrm{Y}, \mathrm{Z} \in \{\mathrm{P}, \mathrm{O}, \mathrm{C}, \mathrm{SO}\}$ specifies the underlying framework. Ideally, the resource usage of B will be similar to that of A (e.g., polynomially related) but we do not focus on this as we have not fixed a notion of efficiency. This property holds for reasonable notions of efficiency.

We show that under mild assumptions on U as well as the underlying computation notions $\mathcal{M}, \mathcal{C}$, and $\mathcal{P}$, we can convert between the different frameworks with at most a polynomial loss in efficiency. The need for these mild assumptions arises for the circuit-based and "strict" oracle-based frameworks since, surprisingly, we cannot always compute the function $\mathtt{mux}(\langle\mathbf{elems}\rangle\ X_0, X_1, \langle\mathbf{bit}\rangle\ b) = X_b$ with algorithms in $\mathrm{TSA}_\mathrm{C}[U]$ and $\mathrm{TSA}_\mathrm{SO}[U]$.

Claim. Fix U with $\mathtt{mux} \in U.\Pi$ and $\mathrm{X} \in \{\mathrm{TS}, \mathrm{A}, \mathrm{S}\}$. Then $\mathrm{XA}_\mathrm{P}[U]$, $\mathrm{XA}_\mathrm{O}[U]$, $\mathrm{XA}_\mathrm{C}[U]$, and $\mathrm{XA}_\mathrm{SO}[U]$ are equivalent.

In Appendix A, we show the non-equivalence without `mux` and show the inter-framework equivalence by converting algorithms from one framework to another.

5.4 Comparing to Prior Definitions

Intuitively, since algorithms in $\mathrm{TSA}_{\mathrm{P}}[U]$ are independent of the choice of σ, this can be thought of as these algorithms being generic with respect to the underlying group $U.\mathbb{G}$. Then by giving algorithms access to $\mathrm{encode}_{\sigma}()$, we allow them to "see" what each group element is. However, due to the inherent type safety, they are still restricted from generating $\langle\textbf{elem}\rangle$ outputs that are unrelated to their $\langle\textbf{elem}\rangle$ inputs, which gives them the algebraic flavor. Then $\mathrm{decode}_{\sigma}()$ effectively removes this type safety, thus becoming standard model compliant.

A (non-)counter example. Our modeling of algebraic algorithms by their execution behavior rather than their inputs and outputs may initially seem overly restrictive. Consider the following algorithm (paraphrased from a EUROCRYPT 2024 reviewer) that is clearly algebraic as per the FKL framework, but for which it is not directly evident where it falls in our framework.

Example 2. Let A be an algorithm on group $\mathbb{G}$ that executes another algorithm B internally. First A forwards its inputs of group elements and non-group elements to B as bits. Then B internally executes "non-algebraic" operations (e.g., B may use a hash-to-group circuit to derive group elements from its inputs). Finally, B outputs a bitstring with the value $x \in \mathbb{Z}_p$. Then A submits the outputs $(g_{\mathbb{G}}^{x}, (x))$, where g is the generator of $\mathbb{G}$ and (x) is the vector of explanations.

As per the FKL framework, algorithm A is algebraic, as it outputs a group element $g_{\mathbb{G}}^{x}$, as well as its explanation x. However, it is perhaps not immediately clear where A lies as per our framework since B's execution behavior is "non-algebraic". We can create A_{P}, an algebraic instantiation of A in our pseudocode framework. First A_{P} accepts its group element inputs as $\langle\textbf{elem}\rangle$ variables, and other inputs as $\langle\textbf{bits}\rangle$. Internally, A_{P} executes $\mathrm{encode}_{U.\sigma}$ on all group elements to produce the required inputs for B. Then A_{P} then internally emulates B's execution on these bits to obtain the output $\langle\textbf{int}\rangle$ x. It can then output $\langle\textbf{elem}\rangle$ $g_{\mathbb{G}}^{x}$ by performing group operations to compute the exponentiation.

Algebraic models with explanations. AGM reductions, e.g., from CDH to discrete log, require the FKL-style output of explanation vectors. To capture this we design an "extractor" $\mathcal{X}[\cdot] : \mathrm{AA}_{\mathrm{O}}[U_{\mathbb{G}}] \to \mathrm{AA}_{\mathrm{O}}[U_{\mathbb{G}}]$ that compiles any algebraic algorithm in our framework into an algorithm that does output explanations corresponding to each group element output.

Fix an algebraic oracle algorithm A and consider $\mathcal{X}[\mathsf{A}]$ defined in Fig. 9. Throughout the computation, it uses table E to store explanations for each group element in V. Here E is a doubly-indexed table such that each $E[\ell]$ is a vector $(E_0[\ell], E_1[\ell], \ldots, E_{|X|}[\ell])$ of the explanation for the element $V[\ell]$ (when non-$\perp$). Each oracle updates E to maintain that it explains the relevant elements.

Theorem 1. *Fix a cyclic group $\mathbb{G}$ and define a universe for it with $U_{\mathbb{G}}.S = \mathbb{G}.S$, $U_{\mathbb{G}}.\sigma = \sigma_{\mathbb{G}}$, $U_{\mathbb{G}}.\Sigma = \{\mathtt{eq}\}$, and $U_{\mathbb{G}}.\Pi = \{\mathtt{gen}, \mathtt{op}, \mathtt{mux}\}$. Define $\mathcal{X}[\mathsf{A}]$ as in Fig. 9 from A. Then for any inputs x, X at the end of $\mathcal{X}[\mathsf{A}]$'s execution it will hold that*

$$V[\ell_j] = g_{\mathbb{G}}^{E_0[\ell_j]} \prod_{i=1}^{|X|} X_i^{E_i[\ell_j]}$$

for all $j \in [n]$ with $V[\ell_j] \neq \perp$. If $\mathsf{A} \in \mathrm{TSA}_{\mathrm{O}}[U_{\mathbb{G}}]$, then $\mathcal{X}[\mathsf{A}] \in \mathrm{TSA}_{\mathrm{P}}[U_{\mathbb{G}}]$.

Algorithm $\mathcal{X}[\mathsf{A}](\langle\textbf{bits}\rangle\ x, \langle\textbf{elems}\rangle\ X)$

$\langle\textbf{elems}\rangle\ V$
$\langle\textbf{ints}\rangle\ E \leftarrow 0$ // initialized 0 everywhere
$\langle\textbf{bits}\rangle\ \ell, \overrightarrow{\ell_{\text{input}}}$
for $i \in [|X|]$ **do** $V[\ell_{\text{input},i}] \leftarrow X[i]$
// $\ell_{\text{input},i}$ is the label for the i^{th} input
for $i \in [|X|]$ **do** $E_i[\ell_{\text{input,i}}] \leftarrow 1$
// Set explanation of each input as itself
$(\ell_1, \ldots, \ell_n, x) \leftarrow\!\$\ \mathsf{A}^{\mathsf{O}_{\mathtt{gen}}, \mathsf{O}_{\mathtt{op}}, \mathsf{O}_{\mathtt{eq}}, \mathsf{Encode}_\sigma}(x, \overrightarrow{\ell_{\text{input}}})$
return $(E[\ell_1], \ldots, E[\ell_n], x)$

Oracle $\mathsf{O}_{\mathtt{eq}}(\ell_i, \ell_j)$

return $\text{rel}_{\mathtt{eq}}(V[\ell_i], V[\ell_j])$

Oracle $\mathsf{Encode}_\sigma(\ell)$

return $\text{encode}_\sigma(V[\ell])$

Oracles $\mathsf{O}_{\mathtt{gen}}(\ell_{\text{out}})$

$V[\ell_{\text{out}}] \leftarrow \text{func}_{\mathtt{gen}}()$
$E[\ell_{\text{out}}] \leftarrow (0, \ldots, 0)$
// Initialize to 0 everywhere
$E_0[\ell_{\text{out}}] \leftarrow 1$ // except $g^1_{\mathbb{G}} = g_{\mathbb{G}}$
return

Oracles $\mathsf{O}_{\mathtt{op}}(\ell_i, \ell_j, \ell_{\text{out}})$

$\langle\textbf{bit}\rangle\ \text{bot} \leftarrow (\perp \in \{V[\ell_i], V[\ell_j]\})$
$V[\ell_{\text{out}}] \leftarrow \text{func}_{\mathtt{op}}(V[\ell_i], V[\ell_j])$
$E[\ell_{\text{out}}] \leftarrow E[\ell_i] + E[\ell_j]$
if bot **do** $E[\ell_{\text{out}}] \leftarrow (0, \ldots, 0)$
return

Oracles $\mathsf{O}_{\mathtt{mux}}(\ell_i, \ell_j, b, \ell_{\text{out}})$

if $b = 0$ **do** $k \leftarrow i$ **else** $k \leftarrow j$
$(V[\ell_{\text{out}}], E[\ell_{\text{out}}]) \leftarrow (V[\ell_k], E[\ell_k])$
return

Fig. 9: Extractor algorithm $\mathcal{X}[\mathsf{A}]$ for oracle algorithm $\mathsf{A} \in \text{AA}_{\text{O}}[U]$

The result will generalize easily to other reasonable choices of $U_{\mathbb{G}}.\Pi$. The core requirement is that for each $f \in U_{\mathbb{G}}.\Pi$ we can identify an f' which computes "the same" function on explanations. By this we mean that if $f(A_1, \ldots, A_n, x) = B$ then $B = \prod_i X_i^{f'(\mathbf{e}_1, \ldots, \mathbf{e}_n, x)_i}$ if each $\mathbf{e}_j$ is an explanation of A_j in terms of the X's (meaning $A_j = \prod_i X_i^{\mathbf{e}_{ji}}$).

Proof (Sketch). We can, in fact, prove something stronger, namely that the given relationship holds at all times during the execution of $\mathcal{X}[\mathsf{A}]$ (after the initial setup) and for all choices of ℓ with $V[\ell] \neq \perp$. The base case holds from the for loops setting $E_i[\ell_{\text{input},i}] \leftarrow 1$.

For the induction step, we consider what happens in each oracle. In $\mathtt{gen}$, the explanations E are set to the appropriate hardcoded values for the generator. In oracle $\mathsf{O}_{\mathtt{op}}$, adding the explanations (unless one of the inputs is $\perp$), gives the desired behavior as $(\prod_i X_i^{a_i})(\prod_i X_i^{b_i}) = \prod_i X_i^{a_i + b_i}$ holds. In oracle $\mathsf{O}_{\mathtt{mux}}$, the explanation is copied from the same label as the group element. □

Using our equivalences between frameworks, we could restate this result for A being an algebraic group algorithm in any of the frameworks. We will henceforth assume the existence of an algorithm $\mathcal{X}[\mathsf{A}]$ for any algebraic A which runs in essentially the same amount of time as A and produces the same output, except the group elements are replaced with vectors over $\mathbb{Z}_p$ which explain how these outputs could have been derived from the inputs.

FKL AGM and standard model. By defining $\mathcal{X}^*[\mathsf{A}]$ to be identical to $\mathcal{X}[\mathsf{A}]$, except it also outputs each $\text{encode}_\sigma(V[\ell_i])$, the above result proves that any algebraic algorithm in our frameworks can be converted to an FKL-style equivalent. We also desire the other direction—any algorithm that is algebraic as per FKL's framework is also algebraic in ours. Additionally, we also want to show that our framework provides sufficient tools to capture any standard model instantiations. These are informally captured by the following claim.

Claim (Informal). For every algebraic algorithm in FKL's formalism, there exists an algebraic algorithm in our frameworks that produces the same outputs on the same inputs. Furthermore, any standard model algorithm can be expressed as an unrestricted algorithm in our framework.

The main ideas for this were discussed in Sec. 3.2.

Scope creep. Our framework equivalence, extractor, and comparison to FKL's AGM or the standard model all described "single-shot" algorithms that simply produce output based on the given input. If we are not careful, there is a risk of errors due to scope creep in our work when we consider adversaries that query oracles provided by a game. This could be modelled by viewing the adversary as a sequence of algorithms that output what oracle should be queried next, with what input, and what state should be passed back to the adversary next time it is invoked.

The ideas underlying our results so far port over easily to this setting, *assuming no additional restrictions are placed on the adversary.* However, several of these comparisons assumed the ability to store arbitrary additional state so the ideas may not work for "multi-stage" games [RSS11] which restrict what state the adversary is allowed to store between invocations. This is analogous to Zhandry's [Zha22] observation that the "single-shot" equivalence between Maurer-style and Shoup-style GGMs shown by Jager and Schwenk [JS08] can fail for multi-stage games but not for type-safe "single-stage" games. Care is needed.

5.5 Other Universes (of Computation)

So far we've considered when U models a cyclic group. Now we will discuss how the relevant ideas extend to algebraic/generic modes for bilinear groups or group actions.

Bilinear group models. Let $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ be groups of prime order p and $e : \mathbb{G}_1.S \times \mathbb{G}_2.S \to \mathbb{G}_T.S$ be a non-trivial bilinear map (meaning $e(g_{\mathbb{G}_1}^a, g_{\mathbb{G}_2}^b) = e(g_{\mathbb{G}_1}, g_{\mathbb{G}_1})^{ab}$ and $e(g_{\mathbb{G}_1}, g_{\mathbb{G}_2}) \neq e(g_{\mathbb{G}_1}, g_{\mathbb{G}_1})^0$).

Lipmaa, Parisella, and Siim [LPS25] used a version of our framework for bilinear groups subsequent to the original publication of this work. Consider defining a universe of computation U for these groups. We would define $U.S = \mathbb{G}_1.S \sqcup \mathbb{G}_2.S \sqcup \mathbb{G}_T.S$ and $U.\sigma$ by $U.\sigma(A) = \sigma_{\mathbb{G}_i}(A)$ for $A \in \mathbb{G}_i.S$.[16] Here $\sqcup$ indicates an assumption that this is a disjoint union. We set $U.\Sigma = \{\texttt{eq}\}$ where $\texttt{eq}(A \in S, B \in S)$ returns a bit indicating whether $A = B$. As with non-bilinear group models, there is flexibility in defining $U.\Pi$. We could for example define $U.\Pi = \{\texttt{gen}, \texttt{op}, \texttt{pair}, \texttt{mux}\}$ where $\texttt{gen}(i \in \{0,1\}^2) = g_{\mathbb{G}_i}$, $\texttt{op}(A \in S, B \in S) = AB$ if $A, B \in \mathbb{G}_i.S$ for the same i and $\texttt{op}(A, B) = \perp$ otherwise, and $\texttt{pair}(A \in S, B \in S) \to e(A, B)$ if $A \in \mathbb{G}_1.S$ and $B \in \mathbb{G}_2.S$ and $\texttt{pair}(A \in S, B \in S) = \perp$ otherwise.

The above captures Type 3 bilinear group models. In Type 2 models there is an efficient isomorphism $\psi : \mathbb{G}_2.S \to \mathbb{G}_1.S$ and so $U.\Pi$ should also include $\texttt{iso}$ defined by $\texttt{iso}(A \in S) = \psi(A)$ for $A \in \mathbb{G}_2.S$ and $\texttt{iso}(A) = \perp$ otherwise. In Type 1 (also known as symmetric) models ψ^{-1} is also efficient so we should similarly include $\texttt{iso}$ but define $\texttt{iso}(A \in S) = \psi^{-1}(A)$ for $A \in \mathbb{G}_1.S$.

Algebraic bilinear group models were defined, e.g., in [BFL20, CH20, MTT19]. Let $\mathbf{A}$, $\mathbf{B}$, and $\mathbf{C}$ denote respectively the inputs an algorithm has received so far from $\mathbb{G}_1$, $\mathbb{G}_2$, and $\mathbb{G}_T$ with

[16] This would formally be more general if we defined universes of computation to specify multiple S sets. Then we could allow collisions in the output of $U.\sigma$ across the different groups or even $\mathbb{G}_1.S, \mathbb{G}_2.S, \mathbb{G}_T.S$ being non-disjoint. Such a modification would not change our conclusions.

$(A_0, B_0, C_0) = (g_{\mathbb{G}_1}, g_{\mathbb{G}_2}, g_{\mathbb{G}_T})$. When outputting an element D, an algebraic algorithm must include vectors $\mathbf{a}$, $\mathbf{b}$, and $\mathbf{c}$ and matrices $\mathbf{x}$, $\mathbf{y}$, and $\mathbf{z}$. If $D \in \mathbb{G}_1.S$, they must satisfy

$$D = \prod_i A_i^{a_i} \quad /\!/ \text{ Type 3}$$
$$D = \prod_i A_i^{a_i} \cdot \prod_i \psi(B_i)^{b_i}. \quad /\!/ \text{ Type 1 or 2}$$

If $D \in \mathbb{G}_2.S$, they must satisfy

$$D = \prod_i B_i^{b_i} \quad /\!/ \text{ Type 2 or 3}$$
$$D = \prod_i \psi^{-1}(A_i)^{a_i} \cdot \prod_i B_i^{b_i}. \quad /\!/ \text{ Type 1}$$

If $D \in \mathbb{G}_T.S$, they must satisfy

$$D = \prod_i C_i^{c_i} \cdot \prod_{ij} e(A_i, B_j)^{z_{ij}} \quad /\!/ \text{ Type 3}$$
$$D = \prod_i C_i^{c_i} \cdot \prod_{ij} e(A_i, B_j)^{z_{ij}} \cdot \prod_{ij} e(\psi(B_i), B_j)^{y_{ij}} \quad /\!/ \text{ Type 2}$$
$$D = \prod_i C_i^{c_i} \cdot \prod_{ij} e(A_i, B_j)^{z_{ij}} \cdot \prod_{ij} e(\psi(B_i), B_j)^{y_{ij}} \cdot \prod_{ij} e(A_i, \psi^{-1}(A_j))^{x_{ij}}. \quad /\!/ \text{ Type 1}$$

The equivalence between prior standard, algebraic, or generic models for bilinear groups follows analogously to our results for plain cyclic groups. For algebraic model equivalence, we need only verify that explanation vectors can be tracked across $U.\Pi$ operations so that an appropriate extractor χ can be constructed.

Input group elements can be given explanations where $\mathbf{a}$, $\mathbf{b}$, or $\mathbf{c}$ (depending on the group) is set to the appropriate standard basis vector, and all other parts of the explanation are all zeros. Suppose X has explanation $v_X = (\mathbf{a}_X, \mathbf{b}_X, \mathbf{c}_X, \mathbf{x}_X, \mathbf{y}_X, \mathbf{z}_X)$ and Y has explanation $v_Y = (\mathbf{a}_Y, \mathbf{b}_Y, \mathbf{c}_Y, \mathbf{x}_Y, \mathbf{y}_Y, \mathbf{z}_Y)$.

If $Z = \texttt{gen}(i)$, then it is explain by $\mathbf{a}_Z$, $\mathbf{b}_Z$, or $\mathbf{c}_Z$ equaling $(1, 0, 0, \dots)$ as appropriate and everything else being all zeros. If $Z = \texttt{op}(X, Y)$, then it is explained by $v_Z = v_X + v_Y$. If $Z = \texttt{mux}(X, Y, b)$, then it is explained by $v_Z = (1 - b)v_X + bv_Y$. If $Z = \texttt{pair}(X, Y)$, then it is explained by $\mathbf{x}_Z = \mathbf{a}_X \otimes \mathbf{a}_Y$, $\mathbf{y}_Z = \mathbf{b}_X \otimes \mathbf{b}_Y$, and $\mathbf{z}_Z = \mathbf{a}_X \otimes \mathbf{b}_Y + \mathbf{a}_Y \otimes \mathbf{b}_X$[17]. If $Z = \texttt{iso}(X)$, then $v_Z = v_X$. The above assumes $Z \neq \perp$; if $Z = \perp$, then it doesn't need an explanation.

Group action models. Let $\mathbb{G}$ be a group of order N, $\mathcal{X}$ be a set, and $\star : \mathbb{G}.S \times \mathcal{X} \to \mathcal{X}$ be a regular group action (meaning $g_{\mathbb{G}} \star x = x$, $(AB) \star x = A \star (B \star x)$, and $y = A \star x$ holds for precisely one A). Fix $x_{\mathcal{X}} \in \mathcal{X}$, which we call the origin of $\mathcal{X}$. Here we are, in particular, thinking of modelling "Known-order Cyclic Effective Group Actions" [ADMP20, DHK+23]. The twist of $x = A \star x_{\mathcal{X}}$ is $x^t = A^{-1} \star x_{\mathcal{X}}$.

Consider defining a universe of computation U. We could define $U.S = \mathbb{G}.S \sqcup \mathcal{X}$ and $U.\sigma$ by $U.\sigma(A) = \sigma_{\mathbb{G}}(A)$ for $A \in \mathbb{G}.S$ and $U.\sigma(A) = \sigma_{\mathcal{X}}(x)$ for $x \in \mathcal{X}$. Here $\sigma_{\mathcal{X}} : \mathcal{X} \to \{0,1\}^{\sigma_{\mathcal{X}}.n}$ is a fixed way of encoding $\mathcal{X}$ elements as bitstrings. We set $U.\Sigma = \{\texttt{eq}\}$ where $\texttt{eq}(A \in S, B \in S)$ returns a bit indicating whether $A = B$. As with other group models, there is flexibility in defining $U.\Pi$. We could

[17] $\otimes$ is the inner product.

for example define $U.\Pi = \{\texttt{gen}, \texttt{op}, \texttt{orig}, \texttt{action}, \texttt{mux}\}$ where $\texttt{gen}() = g_{\mathbb{G}}$, $\texttt{op}(A \in S, B \in S) = AB$ if $A, B \in \mathbb{G}.S$, $\texttt{orig}() = x_{\mathcal{X}}$, and $\texttt{action}(A \in S, x \in S) = A \star x$ if $A \in \mathbb{G}.S$ and $x \in \mathcal{X}$. If an efficient twist is assumed, we additionally add a twist function defined by $\texttt{twist}(x \in S) = x^t$ if $x \in \mathcal{X}$.

Montgomery and Zhandry [MZ22] and Duman, Hartmann, Kiltz, Kunzweiler, Lehmann, Riepel [DHK+23] introduced random-representation-style generic models for group actions. Our universe above corresponds roughly to the Montgomery and Zhandry model. Duman, et al. argue that group action assumptions are intended to be based only on the hardness of the action itself, not of group operations. This could be captured by adding a discrete log function to $U.\Pi$ defined by $\texttt{dlog}(A \in S) = \texttt{dlog}_{\mathbb{G}}(A)$ if $A \in \mathbb{G}.S$. A more literal capturing of the Duman, et al. model would have $U.S = \mathcal{X}$, omit $\texttt{gen}$ and $\texttt{op}$ from $U.\Pi$, and add $\texttt{action}(x \in S, a \in \{0,1\}^n) = g_{\mathbb{G}}^a \star x$ to $U.\Pi$.

Duman, et al. define an algebraic group action model. If $\mathbf{x}$ denotes the inputs the algorithm has received so far from $\mathcal{X}$ (with $x_0 = x_{\mathcal{X}}$) then the algorithm is required to explain an output x with (A, i, b) satisfying

$$y = \begin{cases} A \star x_i, & \text{if } b = 0 \\ A \star x_i^t, & \text{if } b = 1. \end{cases}$$

To prove the equivalence of our induced algebraic model with their model, we must verify that we can track explanations across $U.\Pi$ operations. Input elements x_i are given explanation $(\texttt{gen}_{\mathbb{G}}^0, i, 0)$. Suppose x has explanation (A_x, i_x, b_x). If y is generated by acting on x with B, then it is given explanation (BA_x, i_x, b_x). If y is generated by twisting x, then it is given explanation $(A_x, i_x, 1 - b_x)$.

6 AGM to GGM Lifting Result

The goal of this section is to understand when this equivalence at the algorithmic level can be used to infer that the security of a cryptosystem against algorithms within different frameworks holds. In this section, we formalize a general notion of what is meant by a cryptographic security goal and use that framework to study the question of what conditions cause the inference to hold. We affirmatively establish criteria under which the FKL lifting result necessarily holds.

6.1 Example: CDH and Discrete Log

We start with a concrete example of lifting a result from the algebraic group model to the generic group model. Then we will abstract out the core properties we used to state a general lifting result.

Security definitions. Consider the games defined in Fig. 10 which recall CDH and discrete log security. They are parameterized by a universe U for a group $\mathbb{G}$. We define advantage functions $\mathsf{Adv}_U^{\mathrm{cdh}}(\mathcal{A}) = \Pr[\mathsf{G}_U^{\mathsf{cdh}}(\mathcal{A})]$ and $\mathsf{Adv}_U^{\mathrm{dlog}}(\mathcal{A}) = \Pr[\mathsf{G}_U^{\mathsf{dlog}}(\mathcal{A})]$.

AGM security reduction. Now consider the reduction R shown on the right, which we view as a map $R[\cdot] : \mathcal{A} \mapsto R[\mathcal{A}]$. It internally runs the extractor derived from $\mathcal{A}$, as defined in Sec. 5.4. Thus, R can only be applied to algebraic $\mathcal{A}$. It reduces CDH security to discrete logarithm security. The algorithm $\mathsf{Solve}_{\mathbf{a}}(e)$ take a polynomial $e \in \mathbb{Z}_p[\mathbf{a}]$ and returns a list of solutions to it.[18] The view simulated to $\mathcal{X}^*[\mathcal{A}]$ is perfectly correct (both A and B are uniform in $\mathbb{G}.S \setminus \{g_{\mathbb{G}}^0\}$). If $\mathcal{A}$ would output C and succeed in CDH, then $\texttt{dlog}_{\mathbb{G}}(C) = \texttt{dlog}_{\mathbb{G}}(A) \cdot \texttt{dlog}_{\mathbb{G}}(B)$. By construction, $\texttt{dlog}_{\mathbb{G}}(B) = b \cdot \texttt{dlog}_{\mathbb{G}}(A)$ and the explanation (x_0, x_1, x_2) tells us that $\texttt{dlog}_{\mathbb{G}}(C) = x_0 + x_1 \cdot \texttt{dlog}_{\mathbb{G}}(A) + x_2 \cdot \texttt{dlog}_{\mathbb{G}}(B)$. Putting together and using formal variable $\mathbf{a}$ for $\texttt{dlog}_{\mathbb{G}}(A)$, we have the

[18] Assume it sets $a_2 = 0$ if there is only one solution and also $a_1 = 0$ if there are none.

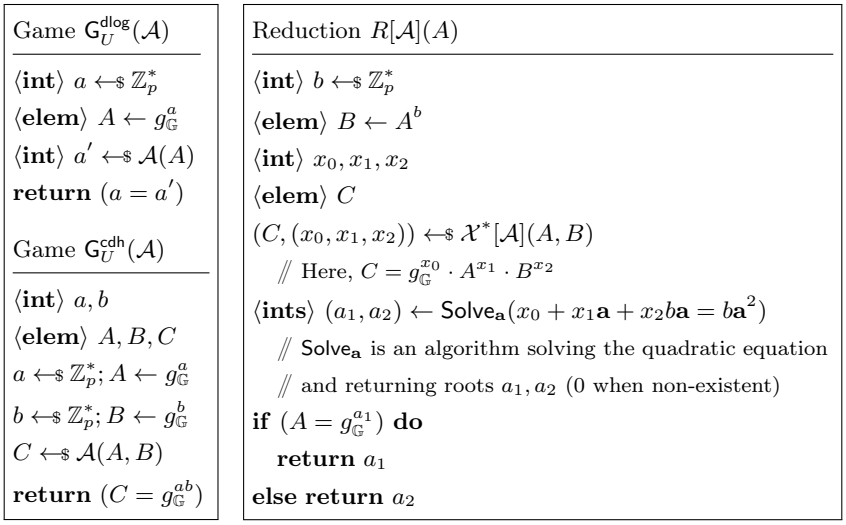


Fig. 10: Games defining computational Diffie-Hellman (CDH) and discrete log security, as well as a reduction showing security of discrete log implies security of CDH against algebraic algorithms

equation $x_0 + x_1\mathbf{a} + x_2 b\mathbf{a} = \mathbf{a}b\mathbf{a}$. As $b \neq 0$, this is a degree two polynomial with at most two zeros. The adversary $\mathcal{A}$ being correct implies that $\mathtt{dlog}_{\mathbb{G}}(A)$ one of the two, so R uses Solve to find it.

Thus $R[\mathcal{A}]$ succeeds whenever $\mathcal{A}$ would, giving $\mathsf{Adv}^{\mathrm{cdh}}_U(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{dlog}}_U(R[\mathcal{A}])$.

Lifting to the GGM. Taking stock of where we are so far, we have used a reduction to the discrete logarithm problem to show that CDH security holds for algebraic adversaries assuming the discrete logarithm problem is hard in the group. Suppose now we want to state this as a security result for type-safe adversaries. First, let us recall that $\mathsf{Adv}^{\mathrm{dlog}}_U(\mathcal{B})$ is at most $O(q^2/p)$ for all type-safe adversaries where q is the number of operation queries $\mathcal{B}$ performs [Mau05]. If we can then show that $\mathcal{B} = R[\mathcal{A}]$ is type safe whenever $\mathcal{A}$ is, we are done.

This is basically immediate. Examining the code of $R[\mathcal{A}]$ we see that encode_σ and decode_σ are clearly never used unless $\mathcal{X}^*[\mathcal{A}]$ uses it. So $R[\mathcal{A}]$ is type safe if $\mathcal{X}^*[\mathcal{A}]$ is. We previously noted $\mathcal{X}^*[\mathcal{A}]$ is type safe if $\mathcal{A}$ is. Hence $R[\mathcal{A}]$ is type safe and we have the bound $\mathsf{Adv}^{\mathrm{cdh}}_U(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{dlog}}_U(R[\mathcal{A}]) \in O(q^2/p)$ meaning security has been lifted to the type-safe group model.

The lifting step required that our proof satisfy these three properties:

1. The reduction established security in the algebraic group model, showing that $\mathsf{Adv}^{\mathrm{cdh}}_U(\mathcal{A}) \leq \mathsf{Adv}^{\mathrm{dlog}}_U(R[\mathcal{A}])$ held for algebraic $\mathcal{A}$.
2. Whenever $\mathcal{A}$ was type safe, so was $R[\mathcal{A}]$.
3. Discrete logarithms are known to be hard for type-safe adversaries.

All three properties are readily extractable from the proof. Property (1) was the main content of the proof. Property (2) was inferred by simply looking at the code of R. Had R been written in a typical pseudocode notation (perhaps with an $\mathcal{A}$ written in the style of FKL's AGM) without types, we would have to first induce a typing onto all of R's variables and deduce from that whether (2) was satisfied, interpreting their $\mathcal{A}$ as our $\mathcal{X}^*[\mathcal{A}]$. This is typically straightforward. Finally (3) is an assumption we bring with us when initiating the attempt to lift the result.

6.2 Inferring GGM Security from AGM Security

We now abstract out the necessary components of an algebraic reduction to lift it to the generic model. It may seem that our framing is too abstract to the point of essentially being vacuous, but this is a positive attribute because the necessary properties are typically straightforward to extract from an existing algebraic group model proof.

First, we define advantage functions abstractly as a map from algorithms to real numbers. Then the notion of hardness for generic algorithms means the output of this function is small.

Definition 8 (Advantage). *Fix U. An advantage metric is a function $\mathsf{Adv}^{\mathrm{met}}_U : \mathrm{SA}_{\mathrm{P}}[U] \to \mathbb{R}$. It is* generically hard *if $\mathsf{Adv}^{\mathrm{met}}_U(\mathcal{A})$ is negligible for all efficient $\mathcal{A} \in \mathrm{TSA}_{\mathrm{P}}[U]$.*

The fact that we stated this in terms of the pseudocode framework is inconsequential. Any of the frameworks suffices for our equivalence. We are stating things non-asymptotically, so formally we are not assigning precise meaning to the terms "negligible" and "efficient". If we define a notion of running time for $\mathrm{TSA}_{\mathrm{P}}[U]$ algorithms (e.g., in type-safe models it is common to measure only the number of $U.\Pi$ operations, treating bitstring computation and $\mathtt{eq} \in U.\Sigma$ as free operations) and state everything asymptotically in terms of security parameter n we could use the usual interpretations of $n^{-\omega(1)}$ and PPT.

Next, we define a general notion of an algebraic reduction being a map between algorithms and that a reduction is model-preserving if the output algorithm is generic whenever the input is.

Definition 9 (Reduction). *Fix U. An* algebraic reduction *is a map $R[\cdot] : \mathrm{AA}_{\mathrm{P}}[U] \to \mathrm{AA}_{\mathrm{P}}[U]$. It is* model-preserving *if $R[\mathcal{A}] \in \mathrm{TSA}_{\mathrm{P}}[U]$ whenever $\mathcal{A} \in \mathrm{TSA}_{\mathrm{P}}[U]$. It is* efficiency preserving *if $R[\mathcal{A}]$ is efficient whenever $\mathcal{A}$ is.*

Finally, we require that R actually was proven to imply security in the AGM.

Definition 10 (AGM Reduction). *Fix U and two advantage metrics, $\mathsf{Adv}^{\mathrm{met1}}_U$ and $\mathsf{Adv}^{\mathrm{met2}}_U$. We say algebraic reduction R is* a Δ-AGM reduction *for these metrics if for all $\mathcal{A} \in \mathrm{AA}_{\mathrm{P}}[U]$ we have*

$$\mathsf{Adv}^{\mathrm{met1}}_U(\mathcal{A}) \leq \Delta \cdot \mathsf{Adv}^{\mathrm{met2}}_U(R[\mathcal{A}]).$$

For our claims, this is only required to hold for a fixed U, but for a given partially specified U the relationship would typically hold for any choice of $U.\sigma$. Putting these two definitions together gives our lifting result.

Theorem 2 (Lifting Result). *Fix U and two advantage metrics, $\mathsf{Adv}^{\mathrm{met1}}_U$ and $\mathsf{Adv}^{\mathrm{met2}}_U$ where the latter is generically hard. Let R be a Δ-AGM, model-preserving, and efficiency-preserving reduction for these metrics. Then $\mathsf{Adv}^{\mathrm{met1}}_U$ is generically hard.*

Proof. This follows naturally from the assumptions. Let efficient $\mathcal{A} \in \mathrm{TSA}_{\mathrm{P}}[U]$ be given. Because R is a Δ-AGM reduction we have that $\mathsf{Adv}^{\mathrm{met1}}_U(\mathcal{A}) \leq \Delta \cdot \mathsf{Adv}^{\mathrm{met2}}_U(R[\mathcal{A}])$. Since R is model and efficiency preserving, we have that $R[\mathcal{A}] \in \mathrm{TSA}_{\mathrm{P}}[U]$ and $R[\mathcal{A}]$ is efficient. Then by the generic hardness of $\mathsf{Adv}^{\mathrm{met2}}_U$ we infer that $\Delta \cdot \mathsf{Adv}^{\mathrm{met2}}_U(R[\mathcal{A}])$ and hence $\mathsf{Adv}^{\mathrm{met1}}_U(\mathcal{A})$ is small. □

The abstract framing of the result makes its proof straightforward, but leaves the natural question of whether cryptographers are writing AGM proofs via reductions that satisfy our conditions. In short, yes. First, we note that by the very nature of our goal and starting point, we expect these conditions to hold for advantage metrics and reductions used in an algebraic group model proof. Whether R is model preserving will need checking—is it actually the case that $R[\mathcal{A}]$ is generic whenever $\mathcal{A}$ is for some given proof?

Reduction $\mathsf{R}[\mathcal{A}]^{\mathsf{O}}$	Oracle $\mathsf{SimOrac}(\langle\mathbf{bits}\rangle\ x, \langle\mathbf{elems}\rangle\ X, \langle\mathbf{ints}\rangle\ \mathbf{e})$
$\langle\mathbf{elems}\rangle\ X, ST$	$(x, X, st, ST) \leftarrow\!\$\ \mathsf{R.Orac}^{\mathsf{O}}(x, X, \mathbf{e}, st, ST)$
$\langle\mathbf{bits}\rangle\ x, st$	**return** (x, X)
$\langle\mathbf{ints}\rangle\ \mathbf{e}$	
$(st, ST) \leftarrow\!\$\ \mathsf{R.Init}^{\mathsf{O}}$	
$(x, X, \mathbf{e}) \leftarrow\!\$\ \mathcal{X}[\mathcal{A}]^{\mathsf{SimOrac}}$	
$(x, X) \leftarrow\!\$\ \mathsf{R.Fin}^{\mathsf{O}}(x, X, \mathbf{e}, st, ST)$	
return (x, X)	

Fig. 11: A blackbox reduction R defined by algorithms $\mathsf{R.Init}$, $\mathsf{R.Orac}$, and $\mathsf{R.Fin}$. It maps adversary $\mathcal{A}$ to the adversary $\mathsf{R}[\mathcal{A}]$ which expects access to oracle O.

Consider the general framing of a black-box reduction R shown in Fig. 11. It assumes the reduction is of a very natural straight-line form used by the majority of AGM proofs. The reduction specifies three algorithms to run before $\mathcal{A}$ ($\mathsf{R.Init}$), after $\mathcal{A}$ ($\mathsf{R.Fin}$), and whenever $\mathcal{A}$ makes any oracle queries ($\mathsf{R.Orac}$). The extracted version of the algorithm is used so that the explanation vector $\mathbf{e}$ for the $\langle\mathbf{elem}\rangle$ variables X is included as part of the output for use by the reductions finalize procedure and oracle procedure. Note that st and ST are simply the internal state (bits and elements) the reduction stores out of view of $\mathcal{A}$ while x, X are used to store any communication between R and $\mathcal{A}$ as well as R's final output.

The following gives a natural condition when such a R is model preserving.

Theorem 3. *Let* R *be a reduction of the form shown in Fig. 11. Suppose that* $\mathsf{R.Init}, \mathsf{R.Orac}, \mathsf{R.Fin} \in \mathrm{TSA}_{\mathrm{P}}[U]$. *Then* R *is a model-preserving reduction.*

Proof. By construction of $\mathcal{X}[\mathcal{A}]$, if $\mathcal{A} \in \mathrm{TSA}_{\mathrm{P}}[U]$ then $\mathcal{X}[\mathcal{A}] \in \mathrm{TSA}_{\mathrm{P}}[U]$. Thus the code in R consist entirely of type-safe algorithms being executed, so $\mathsf{R}[\mathcal{A}] \in \mathrm{TSA}_{\mathrm{P}}[U]$. □

The point is that when an existing AGM proof is read, it is straightforward to verify that the reduction is of this form and that all of its constituent lines of code are themselves type safe. Consider that our CDH to discrete log reduction can indeed be written in this form. Fundamental to the ease is that type safety is simply a syntactic property that can be checked, perhaps after imposing a type system on a reduction that was written assuming all variables are bits.

The result would, of course, naturally extend if R treated $\mathcal{A}$ in other, more complicated ways, such as running it multiple times or rewinding it, as these do not change the general black-box nature of most cryptographic reductions. Of course, if the reduction somehow actually relied on encoding-based properties of the group, then we cannot claim any sort of lifting result.

We view this as a strong indication that FKL had the correct understanding when making their claim that AGM analysis carries over to the GGM. Our work complements that of FKL by giving a more rigorous framing on which to understand this result and that of KZZ by helping to draw more precise boundaries of what sorts of "generic reductions" the FKL claim holds for.

Scope creep. As discussed, our syntax for the reduction in Fig. 11 could have been extended to cover a more general notion of black-box reductions. Similarly, we could have considered more complicated advantage relationships, e.g.,

$$\mathsf{Adv}_U^{\mathrm{met1}}(\mathcal{A}) \le \Delta_2(\mathcal{A}) \cdot \mathsf{Adv}_U^{\mathrm{met2}}(R_1[\mathcal{A}]) + \Delta_3(\mathcal{A}) \cdot \mathsf{Adv}_U^{\mathrm{met3}}(R_2[\mathcal{A}]) + \varepsilon(\mathcal{A})$$

Table 1: Summary of some AGM reductions in literature. Each reduction R is model- and efficiency-preserving by examination.

Security property	**Reduced to**	**Is R generic?**
Decisional Diffie-Hellman, Square Diffie-Hellman [AHK20, FKL18]	Discrete Log	Yes
EUF-CMA Security of Schnorr Signatures [BVHKX23, FPS20]	Discrete Log	Yes
IND-CCA1 Security of ElGamal Encryption [FKL18]	q DDH	Yes
IND-CCA2 Security of Signed ElGamal PKE [FPS20]	Discrete Log and DDH	Yes
Linear Combination Diffie Hellman [FKL18]	Discrete Log	Yes
LRSW Assumption [BFL20]	Discrete Log	Yes
Non-malleability of ECVRF [BVHKX23]	Discrete Log	Yes
UF-CMA Security of BLS [FKL18]	Discrete Log	Yes
UF-CMA Security of Threshold BLS Signatures [BL22]	One More Discrete Log	Yes
Unforgeability of Clause Blind Schnorr Signatures [FPS20]	One More Discrete Log and Modified ROS	Yes
One More Sequential Unforgeability of Abe's/Schnorr Blind Signatures [KLX22]	Discrete Log	Yes

where R_1, R_2 are reductions and $\Delta_1, \Delta_2, \varepsilon$ are sufficiently small for efficient $\mathcal{A}$.[19]

We argue that doing so would confuse more than it would help. Picking notation for these more general framings would hide the essence of the idea which is rather straightforward. We emphasize that the way this section comes together with nary a technical detail beyond writing definitions is a positive result of the way our different models of group computation were captured via algorithms within the same computational framework, just by increasing syntactic restrictions on what operations are allowed.

That said, it is important that when applying such a lifting result to watch out for scope creep. A sufficiently large change in the setting (e.g. to simulation-based definitions where there are simulators that also need to be quantified over or to multi-stage games where one needs to be careful whether $\mathcal{X}[\mathcal{A}]$ can actually be run for a given $\mathcal{A}$) should invite re-examination.

Applicability to prior results. We looked at some prior AGM reductions in light of the above. We present our analysis of the reductions behind these hardness results when they are expressed in our framework in Table 1. Column 1 lists the property proven by a reduction to the assumption in column 2. By examination and assigning types to variables, we can note that the reductions for each of these satisfy the properties required for versions of our lifting result.

6.3 Example Reduction: Schnorr Signatures

As an example of the assessment we used in creating Table 1, we consider the Fuchsbauer, Plouviez, and Seurin [FPS20] proof that discrete log security implies Schnorr signatures are EUF-CMA secure.

[19] Asymptotically, if the Δ's are polynomial and the ε is negligible whenever $\mathcal{A}$ is polynomially efficient.

$\mathsf{Sch.Sign}(\langle\mathbf{int}\rangle\ x, \langle\mathbf{bits}\rangle\ m)$	$\mathsf{Sch.Verify}(\langle\mathbf{elem}\rangle\ X, R, \langle\mathbf{int}\rangle\ s, \langle\mathbf{bits}\rangle\ m)$
$\langle\mathbf{int}\rangle\ r \leftarrow\!\!\$\ \mathbb{Z}_p; \langle\mathbf{elem}\rangle\ R \leftarrow g_{\mathbb{G}}^r$	$\langle\mathbf{int}\rangle\ c \leftarrow H(\mathrm{encode}_\sigma(R)\|\|m)$
$\langle\mathbf{int}\rangle\ c \leftarrow H(\mathrm{encode}_\sigma(R)\|\|m)$	$\mathbf{return}\ g_{\mathbb{G}}^s = R \cdot X^c$
$\langle\mathbf{int}\rangle\ s \leftarrow r + cx \bmod p$	
$\mathbf{return}\ (R, s)$	

Fig. 12: Schnorr signature scheme expressed in our pseudocode framework, with signing key $x \leftarrow\!\!\$\ \mathbb{Z}_p$ and verification key $X \leftarrow g_{\mathbb{G}}^x$

Game $\mathsf{G}^{\mathsf{euf\text{-}cma}}_{U,\mathsf{Sch}}(\mathcal{A})$	Oracle $\mathsf{RO}(R, m)$
$\langle\mathbf{bits}\rangle\ m, Q_1, \ldots, Q_{q_{\mathsf{Sign}}}$ // Message and message list	$\mathbf{if}\ T(R, m) = \bot\ \mathbf{then}$
$\langle\mathbf{int}\rangle\ j \leftarrow 0$ // Signing counter	$\quad T(R, m) \leftarrow\!\!\$\ \mathbb{Z}_p$
$\langle\mathbf{int}\rangle\ \{T(R, m)\}_{R,m}$ // Random Oracle table	$\mathbf{return}\ T(R, m)$
$\langle\mathbf{int}\rangle\ x \leftarrow\!\!\$\ \mathbb{Z}_p; \langle\mathbf{elem}\rangle\ X \leftarrow g_{\mathbb{G}}^x$ // Keys	Oracle $\mathsf{Sign}(\langle\mathbf{int}\rangle\ m)$
$\langle\mathbf{elem}\rangle\ R; \langle\mathbf{int}\rangle\ s, r, c$	$j \leftarrow j + 1$
$(m, R, s) \leftarrow\!\!\$\ \mathcal{A}^{\mathsf{RO},\mathsf{Sign}}(X)$	$r \leftarrow\!\!\$\ \mathbb{Z}_p; R \leftarrow g_{\mathbb{G}}^r$
$\langle\mathbf{int}\rangle\ c \leftarrow \mathsf{RO}(R, m)$	$c \leftarrow \mathsf{RO}(R, m)$
$\mathbf{return}\ (g_{\mathbb{G}}^s = R \cdot X^c) \wedge (m \notin \{Q_1, \ldots, Q_{q_{\mathsf{Sign}}}\})$	$s \leftarrow r + cx \mod p$
	$Q_j \leftarrow m$
	$\mathbf{return}\ (R, s)$

Fig. 13: Game defining EUF-CMA security of Schnorr signature scheme Sch

We try largely to follow their notation, making minor modifications to match our pseudocode style (e.g., labelling variables with types, writing the group multiplicatively).

We start by recalling the algorithms of the scheme and the security game briefly for context, then look at the reduction, though for creating the table, one could simply have immediately jumped to examining the reduction.

The algorithms of the Schnorr signature scheme Sch is defined in Fig. 12. In practice, it is based on a hash function $H : \{0,1\}^* \to \mathbb{Z}_p$ which takes as input the encoding of a group element concatenated with the message to be signed. This in general makes the scheme not type safe, but in the analysis, we model H as a random oracle RO, which we define as taking an element and a bitstring as input, making it type safe.

This can be seen in the EUF-CMA security game for Schnorr signatures given in Fig. 13. We do not formally require that the game be type safe for the lifting result, but it is nice to verify this to follow Zhandry's [Zha22] recommendation that it is best to restrict the AGM to security games that are type safe. The operations on element variables in the main procedure and signing oracle can clearly be implemented using operations from any standard universe U for a group $\mathbb{G}$.

There is a little subtlety with the random oracle. We treated it as if we can simply implement a table T efficiently indexed by a group element R (and bitstring m). However, the typical group operations do not provide any sort of indexing operation. This is nonetheless not an issue. We could instantiate what is written here by having lists $(R_1, \ldots, R_n)$, $(m_1, \ldots, m_n)$, and $(c_1, \ldots, c_n)$ of elements, bitstrings, and integers together with a counter i initialized to zero. At any point in time, these lists represent that the table T is defined by $T(R_l, m_l) = c_l$ for $l = 1, \ldots, i$. Accessing

Reduction $\mathsf{R}[\mathcal{A}](X)$

$\langle\textbf{bits}\rangle\ m, Q_1, \ldots, Q_{q_{\mathsf{Sign}}}$ // Message and message list
$\langle\textbf{int}\rangle\ j \leftarrow 0$ // Signing counter
$\langle\textbf{int}\rangle\ \{T(R,m)\}_{R,m}$ // Random Oracle table
$\langle\textbf{int}\rangle \times \langle\textbf{int}\rangle \times \langle\textbf{ints}\rangle\ \{U(R,m)\}_{R,m}$ // Explanations table
$\langle\textbf{elem}\rangle\ R; \langle\textbf{int}\rangle\ s, c$
$\langle\textbf{int}\rangle\ \gamma, \xi; \langle\textbf{ints}\rangle\ \overrightarrow{\rho}$ // Explanations
$(m, R_{[\gamma,\xi,\overrightarrow{\rho}]}, s) \leftarrow\!\$\ \mathcal{X}^*[\mathcal{A}]^{\mathsf{RO},\mathsf{Sign}}(X)$
// $R = g_{\mathbb{G}}{}^{\gamma} \cdot X^{\xi} \cdot \prod_{i=1}^{j} R^{\rho_i}$
$\langle\textbf{int}\rangle\ c \leftarrow \mathsf{RO}(R_{[\gamma,\xi,\overrightarrow{\rho}]}, m)$
$(\gamma, \xi, \overrightarrow{\rho}) \leftarrow U(R,m)$
// $R = g_{\mathbb{G}}{}^{\gamma} \cdot X^{\xi} \cdot \prod_{i=1}^{j} R^{\rho_i}$
$\langle\textbf{int}\rangle\ x \leftarrow (s - \gamma - \sum_{i=1}^{j} \rho_i s_i)(\xi + c - \sum_{i=1}^{j} \rho_i c_i)^{-1} \bmod p$
return x

Oracle $\mathsf{RO}(R_{[\gamma,\xi,\overrightarrow{\rho}]}, m)$

// $R = g_{\mathbb{G}}^{\gamma} \cdot X^{\xi} \cdot \prod_{i=1}^{j} R_j^{\rho_j}$
if $T(R,m) = \bot$ **then**
  $T(R,m) \leftarrow\!\$\ \mathbb{Z}_p$
  $U(R,m) \leftarrow (\gamma, \xi, \overrightarrow{\rho})$
return $T(R,m)$

Oracle $\mathsf{Sign}(m)$

$j \leftarrow j + 1$
$\langle\textbf{int}\rangle\ c_j, s_j \leftarrow\!\$\ \mathbb{Z}_p$
$\langle\textbf{elem}\rangle\ R_j \leftarrow g_{\mathbb{G}}^{s_j} / X^{c_j}$
if $T(R_j, m) = \bot$ **then** $T(R_j, m) \leftarrow c_j$
else return $\bot$
$Q_j \leftarrow m$
return (R_j, s_j)

Fig. 14: Model-preserving reduction R showing security of Sch

or adding to the table can be implemented by linearly scanning the lists and incrementing i when appropriate.

Finally, the reduction we consider is given in Fig. 14. Here $\mathcal{X}^*[\mathcal{A}]$ is the algorithm that runs just like $\mathcal{A}$ except when $\mathcal{A}$ would output a group element R (to RO or as a final output, it instead outputs $(R, (\gamma, \xi, \overrightarrow{\rho}))$ where $(\gamma, \xi, \overrightarrow{\rho})$ is an explanation of how R is derived. We denote this as $R_{[\gamma,\xi,\overrightarrow{\rho}]}$ for compactness.

Of this reduction, Fuchsbauer, Plouviez, and Seurin [FPS20] prove that if $\mathcal{A}$ is algebraic, then

$$\mathsf{Adv}^{\text{euf-cma}}_{U,\mathsf{Sch}}(\mathcal{A}) \leq \mathsf{Adv}^{\text{dlog}}_{U}(\mathsf{R}[\mathcal{A}]) + \frac{q_s(q_s + q_h) + q_h + 1}{p}.$$

Examining its code similarly to $\mathsf{G}^{\mathsf{euf\text{-}cma}}$ we can see that R only uses type-safe operation on top of whatever $\mathcal{A}$ does, so R is model-preserving as expected. Hence if $\mathcal{A}$ is generic, then $\mathsf{R}[\mathcal{A}]$ will be and the claimed bound still holds.

7 Variant AGM Settings

Our CRYPTO 2024 reviewers recommended that we discuss how/whether our AGM model can provide insights into variants of the AGM introduced to port the AGM over to other settings such as the algebraic group model with oblivious sampling [LPS23], algebraic distinguishers [RS20], and algebraic attackers in the universal composability framework [ABK+21]. We do so briefly below.

7.1 AGM with Oblivious Sampling

The algebraic group model with oblivious sampling [LPS23] (AGMOS) was introduced by Lipmaa, Parisella, and Siim to model a natural ability of standard model attackers that is disallowed for algebraic attacks—the ability to obliviously sample group elements.

As an example, suppose the bitstring representations of group elements (the output of σ in our frameworks) are in a dense subset of $\{0,1\}^{\sigma.n}$. Then an attacker could pick a random string of

length $\sigma.n$ to get a uniform element A of the group (up to some small error probability) about whose discrete logarithm it has no knowledge. It could then output A or perform some group operations using it and output a related group element A'. Note that an FKL algebraic attacker could not *directly* behave in the same way because (assuming hardness of computing discrete logarithms) it would not be able to explain how A or A' can be derived from the algorithm's initial input. (In our framing of the AGM, the attacker would not be able to convert the bitstring back to a group element.)

This simple example may make the problem seem unimportant. After all, an algebraic attacker could emulate this obvious sampling of A by non-obliviously setting $A = g_{\mathbb{G}}^{a}$ for $a \leftarrow\!\!\$\ \mathbb{Z}_p$. Thereby, for simple security games it is provably the case that this simple oblivious sampling does not help. However, this straightforward proof could fail when considering more complex ways of oblivious sampling or more complex notions of security.

In essence, one can view the use of the plain AGM as implying a conjecture that relevant attacks against the cryptosystem in question will not profitably make use of obliviously sampled elements. The AGMOS serves as a way of formalizing this and making it explicit for a given class of oblivious sampling distributions, by explicitly giving AGM attackers the ability to oblivious sample according to these distributions. Typical AGMOS proofs rely on the TOFR (Tensor Oracle Find Representation) assumption [LPS23] which roughly requires that an attacker cannot find a non-trivial linear relationship between elements sampled from the distributions (or a quadratic relationship if the group has pairings).

Lipmaa, Parisella, and Siim [LPS25] showed how to express the AGMOS in our framework. Let $\mathcal{F}$ denote a set of distributions F over (A, s) tuples where A is a group element and s is a bitstring. Then to the universe of computation U we add an operation that takes as input $F \in \mathcal{F}$, samples $(A, s) \leftarrow\!\!\$\ F$, and returns (A, s).[20] By framing AGMOS in this way, our lifting result gives us conditions under which security proofs can automatically lift to a type-safe generic group model with oblivious sampling. For an all-representation/random-representation generic group model with oblivious sampling [LPS23], one would need to analyze under what conditions the Jager-Schwenk equivalence between generic group models [JS08, Zha22] extends to this setting.

7.2 AGM for Decisional Security Notions

In decisional security notions, the adversary is tasked with outputting a bit to guess which of two "worlds" it is in. Thus, by the known equivalence between the AGM and standard model for adversaries with no group element outputs (including no group elements given as input to oracle queries) discussed in Sec. 4, the AGM is not useful for analyzing such security notions. Motivated by this, Rotem and Segev [RS20] proposed a variant of FKL's AGM for decisional problems.

Definition. For distinguishing games, we consider an algorithm A attempting to distinguish between games $\mathsf{G}_1, \mathsf{G}_0$. Its advantage metric is $\mathsf{Adv}_U(\mathsf{A}) = |\Pr[\mathsf{G}_1(\mathsf{A})] - \Pr[\mathsf{G}_0(\mathsf{A})]|$. The view of A in game G_b, denoted $\mathsf{View}(\mathsf{A}, b)$, is the distribution obtained by running $\mathsf{G}_b(\mathsf{A})$ and creating a vector of all of A's inputs throughout the game (including its randomness and $g_{\mathbb{G}}$). If $\mathbf{w} \in \mathbb{Z}_p^*$ is a vector, then the adversary's view restricted to $\mathbf{w}$, denoted $\mathsf{View}_{\mathbf{w}}(\mathsf{A}, b)$, samples a view $\mathbf{v}$ from $\mathsf{View}(\mathsf{A}, b)$ then replaces the i-th group element of $\mathbf{v}$ with $\perp$ for each i satisfying $\mathbf{w}_i = 0$.

Rotem and Segev require that a decisional algebraic algorithm A be algebraic in the sense of FKL (namely, providing explanations of how to derive any group elements output to oracles) and

[20] As mentioned briefly in Sec. 3.1, this is not formally allowed in our definition of universes of computation because the operation is randomized, but all of our results extend naturally to allow it.

Game $\mathsf{G}_b^{\mathsf{con}}(\mathcal{A})$
$\langle\mathbf{bit}\rangle\ d \leftarrow\!\!\$\ \{0,1\}$
$\langle\mathbf{int}\rangle\ a' \leftarrow\!\!\$\ \mathcal{A}(g_{\mathbb{G}}^{1}, g_{\mathbb{G}}^{2-d}, g_{\mathbb{G}}^{3}, g_{\mathbb{G}}^{4-(d\oplus b)})$
return $(a = 1)$

Game $\mathsf{G}_b^{\mathsf{ddh}}(\mathcal{A})$
$\langle\mathbf{int}\rangle\ x, y, z \leftarrow\!\!\$\ \mathbb{Z}_p^*$
$a \leftarrow\!\!\$\ \mathcal{A}(g_{\mathbb{G}}, g_{\mathbb{G}}^{x}, g_{\mathbb{G}}^{y}, g_{\mathbb{G}}^{xyz^b})$
return $(a = 1)$

Fig. 15: Counterexample games to Proposition 3.2 in [RS20, Sec. 3.3]

that together with its output bit it must also output an explanation vector $\mathbf{w}$. Two restrictions are placed on this explanation.

1. It must explain how to derive the identity element, meaning $g_{\mathbb{G}}^{0} = \prod_i \mathbf{L}_i^{w_i}$ where $\overrightarrow{\mathbf{L}}$ is the list of group elements A has received so far.
2. For at least one choice of $\mathsf{H} \in \{\mathsf{G}_1, \mathsf{G}_0\}$ it must hold that with probability at least $\mathsf{Adv}_U(\mathsf{A})/t^2$, where t is the runtime of $\mathsf{H}(\mathsf{A})$, the vector $\mathbf{w}$ output by A gives different views in G_1 and G_0. By this, we formally mean that $\mathsf{View}_{\mathbf{w}}(\mathsf{A}, 0)$ and $\mathsf{View}_{\mathbf{w}}(\mathsf{A}, 1)$ are different distributions.

Lifting Result? Let us try to use our formalism to formalize a restricted lifting result. We need to start by describing the notion of being algebraic in our language. We cannot capture it as elegantly as the FKL notion of algebraic, because being decisional algebraic requires both a syntactic and a probabilistic property of the algorithm. To capture restriction (1), we can require an attacker to be algebraic in our sense and output the identity element $g_{\mathbb{G}}^{0}$. To capture restriction (2), we can determine $\mathbf{w}$ with the extractor $\mathcal{X}^*[\mathsf{A}]$ and place the same restriction on it.

The ideas from our lifting theorem apply to lift hardness from algebraic adversaries satisfying the above restrictions to hardness for generic type-safe adversaries *satisfying the same restrictions*. A "true" lifting result should imply hardness for all type-safe adversaries (not necessarily satisfying the restrictions). If any type-safe adversary can be converted to one satisfying the restriction, then we would be done. Rotem and Segev claims such a result in their "informal proposition" Proposition 3.2, but we will provide examples below that break the given proof for this proposition.

Roughly and stated in our language, Proposition 3.2 intends to maps a type-safe generic algorithm $\mathsf{A}_{\mathsf{gen}}$ to a type-safe generic algorithm $\mathsf{A}_{\mathsf{alg}}$ with the same advantage that also satisfies the restrictions. Algorithm $\mathsf{A}_{\mathsf{alg}}$ will run $\mathsf{A}_{\mathsf{gen}}$ internally. It tracks all group elements $\mathsf{A}_{\mathsf{gen}}$ computes throughout the game and randomly selects a pair X, Y of these group elements which are equal to each other and then outputs the identity element as $\mathtt{op}(X, \mathtt{inv}(Y))$ along with whatever bit $\mathsf{A}_{\mathsf{gen}}$ outputs. Note that $\mathsf{A}_{\mathsf{gen}}$ computes at most t group elements and so any pair of equal group elements is selected with probability at least $1/\binom{t}{2} > 1/t^2$. Thus the crux of proof depends on whether, with probability at least $\mathsf{Adv}_U(\mathsf{A})$, the exists such a pair for which the explanation of $\mathtt{op}(X, \mathtt{inv}(Y))$ gives different views in the two games. Rotem and Segev explain this as follows:

> 2. Since the access that $\mathsf{A}_{\mathsf{gen}}$ has to the group is representation independent, the only useful information it acquires throughout the game is the equality pattern among the group elements that it receives or produces during the game. Hence, in order to distinguish between G_1 and G_0 with advantage ϵ, then with probability at least ϵ there must exist an equality relation which occurs in one game with different probability than in the other game. [RS20, Sec. 3.3]

In Fig. 15, we give a contrived game for which this proof fails. In $\mathsf{G}^{\mathsf{con}}$, an adversary can obtain advantage 1 just by outputting $\mathtt{eq}(X, X') \oplus \mathtt{eq}(Y, Y')$ on input (X, X', Y, Y'). The proposition's

proof fails because the only pairs of group elements that can be equal are (X, X') and (Y, Y') (or any element with itself), but restricting to either of these pairs gives a view which is identical in either game. In detail, either $\mathbf{w} \in \{(0,0,0,0), (\pm 1, \mp 1, 0, 0), (0, 0, \pm 1, \mp 1)\}$. Then the restricted view $\mathsf{View}_{\mathbf{w}}(\mathsf{A}, b)$ is either $(\bot, \bot, \bot, \bot)$, $(g_{\mathbb{G}}^{1}, g_{\mathbb{G}}^{2-d}, \bot, \bot)$, or $(\bot, \bot, g_{\mathbb{G}}^{3}, g_{\mathbb{G}}^{4-(d \oplus b)})$ for $d \leftarrow\!\!\$\ \{0,1\}$. Each of these can be verified to be independent of b and so $\mathsf{View}_{\mathbf{w}}(\mathsf{A}, 0) \equiv \mathsf{View}_{\mathbf{w}}(\mathsf{A}, 1)$. Informally, the proof assumes a single one of the adversary's equality queries will "solve" the game, but this adversary only succeeds from the combination of both equalities it checks.

This proof also fails for the Decisional Diffie-Hellman assumption captured by game $\mathsf{G}^{\mathsf{ddh}}$ in Fig. 15. Consider the adversary that on input $g_{\mathbb{G}}, X, Y, Z$ outputs $\mathtt{eq}(g_{\mathbb{G}}, X) \cdot \mathtt{eq}(X, Y) \cdot \mathtt{eq}(Y, Z)$. It has advantage $\mathsf{Adv}^{\mathrm{ddh}}_{U}(\mathsf{A}) = 1/(p-1)^2 - 1/(p-1)^3 > 0$ because it outputs 1 in $\mathsf{G}^{\mathsf{ddh}}_{0}$ iff $1 = x = y$ and it outputs 1 in $\mathsf{G}^{\mathsf{ddh}}_{1}$ iff $1 = x = y = z$. Each group element has an explanation with one non-zero element, so $\mathbf{w}$ can have at most two. Clearly $\mathsf{View}_{\mathbf{w}}(\mathsf{A}, b)$ is independent of b if the last group element is excluded. But in any of $(g_{\mathbb{G}}, \bot, \bot, g_{\mathbb{G}}^{xyz^b})$, $(\bot, g_{\mathbb{G}}^{x}, \bot, g_{\mathbb{G}}^{xyz^b})$, or $(\bot, \bot, g_{\mathbb{G}}^{y}, g_{\mathbb{G}}^{xyz^b})$ one of x, y is independent of the first group element and so the last group element remains independent of b in this view. Hence $\mathsf{View}_{\mathbf{w}}(\mathsf{A}, 0) \equiv \mathsf{View}_{\mathbf{w}}(\mathsf{A}, 1)$. Again, this provided a counterexample because the adversary's advantage comes only from the combination of multiple equalities.

Similarly, consider an adversary that brute-force computes $\mathtt{dlog}_{\mathbb{G}}(X)$ by computing $X_i = \mathtt{exp}(\mathtt{gen}, i)$ for $i = 1, \ldots, p-1$ and setting $x = i$ if $\mathtt{eq}(X_i, X)$. Then it computes $Z' = \mathtt{exp}(Y, x)$ and returns $\mathtt{eq}(Z', Z)$. It has advantage $\epsilon = 1 - 1/(p-1)$. The explanation of every group element $g_{\mathbb{G}}, X, Y, Z, X_1, \ldots, X_{p-1}, Z'$ refers to only a single input group element. So by the same argument, $\mathsf{View}_{\mathbf{w}}(\mathsf{A}, b)$ will be independent of b. Again, this provided a counterexample because the adversary's advantage comes only from the combination of multiple equalities.

Consequently, we cannot claim a true lifting result for algorithms that are algebraic in the sense of Lior and Rotem. An interesting question for future work is whether a version of Proposition 3.2 holds. Their definition of algebraic is motivated by Proposition 3.2. "[The second restriction] is *descriptive* of generic group algorithms (see [Proposition 3.2] for further details; this also sheds light as to where the term t^2 comes from)" [RS20], so might need to be modified in the process.

7.3 AGM for Simulation-based Security Notions

The algebraic group model has regularly been used for analysis in simulation-based security notions. Formally, the lifting result from Sec. 6 does not apply, as it only considered single quantifier definitions. We sketch the ideas required for a simulation-based lifting result and observe that it applies to results in the literature.

Abstractly. Starting abstractly, security requires $\forall \mathcal{A}, \exists \mathcal{S} : \mathsf{Adv}^{\mathrm{met}}_{U}(\mathcal{A}, \mathcal{S}) = \mathsf{negl}$ or $\exists \mathcal{S}, \forall \mathcal{A} : \mathsf{Adv}^{\mathrm{met}}_{U}(\mathcal{A}, \mathcal{S}) = \mathsf{negl}$ or $\forall \mathcal{A}_1, \exists \mathcal{S}, \forall \mathcal{A}_2 : \mathsf{Adv}^{\mathrm{met}}_{U}(\mathcal{A}_1, \mathcal{A}_2, \mathcal{S}) = \mathsf{negl}$. Here $\mathcal{S}$ is often called a simulator, but in some contexts other terms are used such as an extractor or emulator. We can discuss these three basic quantifications jointly by viewing $\mathcal{S}[\cdot]$ as a map $\mathcal{A} \mapsto \mathcal{S}[\mathcal{A}]$ with the quantification $\exists \mathcal{S}, \forall \mathcal{A} : \mathsf{Adv}^{\mathrm{met}}_{U}(\mathcal{A}, \mathcal{S}[\mathcal{A}]) = \mathsf{negl}$. The first quantification is captured naturally, the second is captured by requiring $\mathcal{S}[\mathcal{A}]$ be independent of $\mathcal{A}$, and the third is captured by interpreting $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ and requiring $\mathcal{S}[\mathcal{A}]$ be independent of $\mathcal{A}_2$. Quantifying $\mathcal{A}$ over $\mathrm{TSA}_{\mathrm{P}}[U]$, $\mathrm{AA}_{\mathrm{P}}[U]$, or $\mathrm{SA}_{\mathrm{P}}[U]$ naturally defines type-safe generic, algebraic, or standard model security.

Suppose $\mathsf{Adv}^{\mathrm{met1}}_{U}$ is a simulation-based advantage metric and $\mathsf{Adv}^{\mathrm{met2}}_{U}$ is a non-simulation advantage metric. A typical security proof for the simulation metric will establish a result that boils down to the the form $\exists R, \mathcal{S}, \forall \mathcal{A} : \mathsf{Adv}^{\mathrm{met1}}_{U}(\mathcal{A}, \mathcal{S}[\mathcal{A}]) \leq \Delta \cdot \mathsf{Adv}^{\mathrm{met2}}_{U}(R[\mathcal{A}])$. Naturally, if $R[\cdot] : \mathrm{AA}_{\mathrm{P}}[U] \rightarrow \mathrm{AA}_{\mathrm{P}}[U]$ is model-preserving the lifting result will apply for such a proof. In the

other direction, a typical security proof for the non-simulation metric will establish something like $\exists R, \forall \mathcal{S}, \mathcal{A} : \mathsf{Adv}^{\mathrm{met2}}_{U}(\mathcal{A}) \leq \Delta \cdot \mathsf{Adv}^{\mathrm{met1}}_{U}(R[\mathcal{A}, \mathcal{S}], \mathcal{S}[R[\mathcal{A}, \mathcal{S}]])$. Note that R and $\mathcal{S}$ are not circular because R depends on the map $\mathcal{S}[\cdot]$ while $\mathcal{S}$ depends on the particular instantiation $R[\mathcal{A}, \mathcal{S}]$. Depending on the underlying basic quantifications, it might be natural for $R[\mathcal{A}, \mathcal{S}]$ to be independent of $\mathcal{S}$ or $\mathcal{S}[R[\mathcal{A}, \mathcal{S}]]$ to be independent of $R[\mathcal{A}, \mathcal{S}]$. Now for a lifting result, note that $R[\mathcal{A}, \mathcal{S}]$ depends on both $\mathcal{A}$ and $\mathcal{S}$. We need $R[\cdot, \mathcal{S}]$ to be model-preserving. Natural ways to achieve this are for R to be independent of $\mathcal{S}$ or for $\mathcal{S}$ itself to be model preserving. This second case is part of a more general phenomena.

Remark 2. In multi-step proofs, a simulator introduced in one step of the proof often is internally run by later reduction adversaries in the proof. As such, it is desirable for simulators to be model-preserving.

If both metrics are simulation-based, then a security proof will typically establish something like $\exists R, \mathcal{S}_1 \forall \mathcal{S}_2, \mathcal{A} : \mathsf{Adv}^{\mathrm{met1}}_{U}(\mathcal{A}, \mathcal{S}_1[\mathcal{A}, \mathcal{S}_2]) \leq \Delta \cdot \mathsf{Adv}^{\mathrm{met2}}_{U}(R[\mathcal{A}, \mathcal{S}_2], \mathcal{S}_2[\mathcal{A}, R[\mathcal{A}, \mathcal{S}_2]])$. Similar observations to the prior case apply for deciding whether R is model-preserving.

Examples. The most common use of simulation-based definitions in the AGM consider extraction-based definitions. This is exemplified by the FKL result [FKL18, Thms. 7.2 , ePrint] proving knowledge soundness of a SNARK. Security requires that for all $\mathcal{A}$ there exists an extractor $\mathcal{X}$ such that if $\mathcal{A}$ outputs a valid proof for a statement ϕ, then $\mathcal{X}$, given the same inputs as $\mathcal{A}$, will output a valid witness for ϕ. In the proof, the extractor and reduction have simple blackbox forms (that can be expressed akin to Fig. 11) where they run $\mathcal{A}$ and use the explanations of $\mathcal{A}$'s outputs to produce their outputs. Consequently, they are both model-preserving and the lifting result will apply.

Formally, FKL were considering the plain AGM without bilinear maps. In this view, the lifting result would lift to a GGM without bilinear maps, which is unsatisfying because the proof system considered requires a bilinear map. Conveniently, the proof works essentially unchanged to an AGM with bilinear maps (as in Sec. 5.5) and can thus be lifted to a GGM with bilinear maps.

Similar extraction proofs have also been considered in the AGMOS [LPS23, LPS25]. There, $\mathcal{X}$ being given the same inputs as $\mathcal{A}$ should include the oblivious samples $\mathcal{A}$ obtains. The reductions and extractors used are similarly blackbox so the lifting result applies.

7.4 AGM for Universal Composability

Abdalla, Barbosa, Katz, Loss, and Xu [ABK+21] (ABKLX) introduced a universal composability (UC) framework for analyzing algebraic adversaries. Formalizing a UC framework based on our formalism of algebraic/generic algorithms and rigorously comparing to the ABKLX framework is well beyond the scope of this paper. Instead, we will approach this discussion intuitively and pictorially (Fig. 16), aiming to understand high-level takeaways about the application of algebraic/generic algorithms to UC frameworks. We elide significant details.

UC Background. UC security considers a execution model (represented pictorially on the left of Fig. 16a) wherein an environment adversary $\mathcal{E}$ can provide inputs to and receive outputs from a protocol (π), while hampering with the execution of the protocol via calls to a local adversary $\mathcal{A}$ (which may, e.g., read and tamper with communication between protocol parties $\pi_1, \pi_2, \ldots$ sent over the network). Let $\mathrm{EXEC}^{\mathcal{E}}_{\mathcal{A},\pi}$ denotes the probability $\mathcal{E}$ outputs 1 in this execution model. We say π *emulates* μ (intuitively meaning π is at least as secure as μ) if $\exists \mathcal{S}, \forall \mathcal{A}, \mathcal{E} : \mathrm{EXEC}^{\mathcal{E}}_{\mathcal{A},\pi} \approx \mathrm{EXEC}^{\mathcal{E}}_{\mathcal{S}[\mathcal{A}],\mu}$.

Fig. 16a represents this definition pictorially. Note that $\mathcal{S}[\mathcal{A}]$ may not depend on $\mathcal{E}$; the definition is more typically written in the triple-quantifier style $\forall\mathcal{A}, \exists\mathcal{S}, \forall\mathcal{E}$.

An important variant of this (represented pictorially on the left of Fig. 16b) when $\mathcal{E}$ is given direct access to the "backdoor" interface of π machines via a "transparent channel". This is typically formalized as a special case of the first execution model by defining a "dummy adversary" that simply ferries inputs from $\mathcal{E}$ to π and vice versa. Let $\text{EXEC}_{\pi}^{\mathcal{E}}$ denotes the probability $\mathcal{E}$ outputs 1 in this execution. We say π *with a transparent channel emulates* μ if $\exists\mathcal{S}, \forall\mathcal{E} : \text{EXEC}_{\pi}^{\mathcal{E}} \approx \text{EXEC}_{\mathcal{S},\mu}^{\mathcal{E}}$. Fig. 16b represents this definition pictorially.

We now state some of the "canonical" results that are desirable to have hold in a UC model.

Theorem 4 (Informal). *The following hold:*

- *(Completeness [of Transparent Channel Emulation]) If protocol π with a transparent channel emulates μ, then π emulates μ.*
- *(Transitivity) If π with a transparent channel emulates μ and μ emulates υ, then π emulates υ.*
- *(Universal Composability) If protocol π with a transparent channel emulates μ, then ρ^{π} (a protocol wherein π is a sub-protocol) with a transparent channel emulates ρ^{μ}.*

Proof (Pictorial). These are proven in Fig. 16c, Fig. 16d, and Fig. 16e respectively. Here $\equiv$ denotes equivalence and $\approx$ denotes closeness from an emulation assumption. Dotted boxes show how given environments (resp. simulators) are combined with other machines to construct required environments (resp. simulators). □

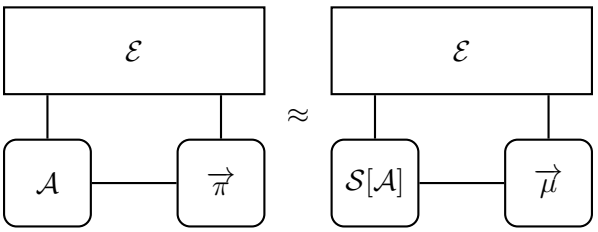
We refer the reader to [Can01, ABK+21] for more rigorous treatment of such results.

A Strict Algebraic Model. The most basic approach to defining a UC framework with our formalism is to take a typical UC framework [Can01] while assuming the code of machines is, say, written using pseudocode matching our framework. Then we capture algebraic or generic security by requiring $\mathcal{E}$ and $\mathcal{A}$ be in $\text{AA}_{\text{P}}[U]$ or $\text{TSA}_{\text{P}}[U]$. Recall that while $\mathcal{E}$ is commonly called "the environment" and $\mathcal{A}$ "the adversary", both of these are adversaries in the non-UC sense. A model where only $\mathcal{A}$ is restricted would make little sense—formally, completeness of transparent channel emulation shows restricting only $\mathcal{A}$ is no restriction at all. We examine the effect of this on Thm. 4.

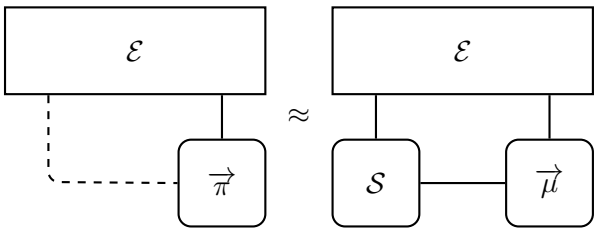
Completeness of transparent channel emulation (Fig. 16c) holds without restriction. The reduction map $\mathcal{E}'[\cdot,\cdot]$ combines a given $\mathcal{E}$ and $\mathcal{A}$ in a clearly model-preserving manner. Thus the application of the assumption that π with a transparent channel emulates μ is valid. Moreover, a formalization of the proof would only need to verify it holds for algebraic adversaries because then the lifting result will apply to imply it for type-safe ones.

The simulator map $\mathcal{S}'[\mathcal{S},\mathcal{A}]$ may not be model-preserving as we have not restricted the given $\mathcal{S}$. If $\mathcal{S}$ is restricted to being type safe, then $\mathcal{S}'$ will be model-preserving. As observed in Remark 2, simulators should be model preserving because in multi-step proofs they often become part of later adversaries. Indeed, we can see exactly this in proving transitivity (Fig. 16d). When we use the first assumption that π with a transparent channel emulates μ, we introduce a simulator $\mathcal{S}$ which is then interpreted as an adversary to use the second assumption that μ emulates υ. Thus, the proof only works if we restrict $\mathcal{S}$ analogously to $\mathcal{E}$ and $\mathcal{A}$. If $\mathcal{S}'[\cdot]$ is restricted to being model-preserving for $\mathcal{S}$, then so too is $\mathcal{S}''[\cdot,\cdot]$.

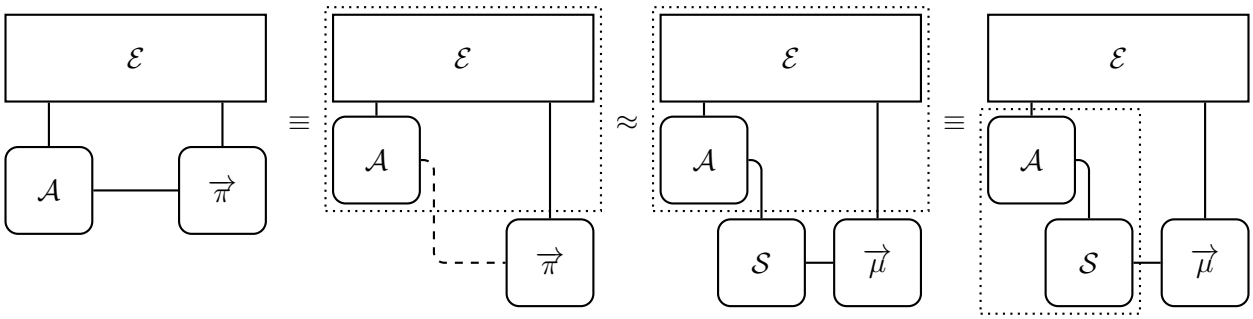
Finally, consider universal composability (Fig. 16e). The reduction map $\mathcal{E}'[\cdot]$ combines a gives $\mathcal{E}$ with the parties of protocol ρ. The application of the assumption that protocol π with a transparent channel emulates μ requires $\mathcal{E}'[\mathcal{E}]$ be of the correct type, thus requiring that the parties of protocol ρ be in the same model as $\mathcal{E}$ and $\mathcal{A}$. If ρ is specifically type safe, then $\mathcal{E}'[\cdot]$ will be model-preserving



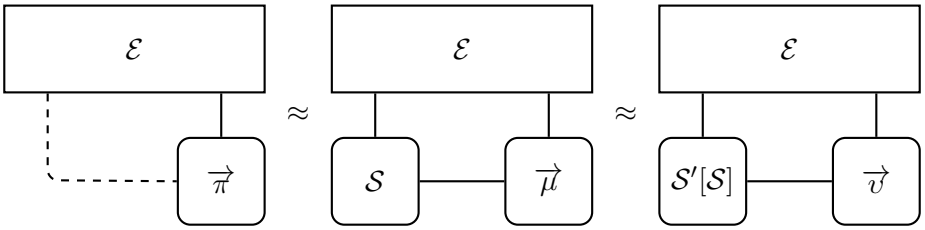
(a) If $\exists \mathcal{S}, \forall \mathcal{A}, \mathcal{E} : \text{EXEC}^{\mathcal{E}}_{\mathcal{A},\pi} \approx \text{EXEC}^{\mathcal{E}}_{\mathcal{S}[\mathcal{A}],\mu}$, we say that the protocol π emulates μ.



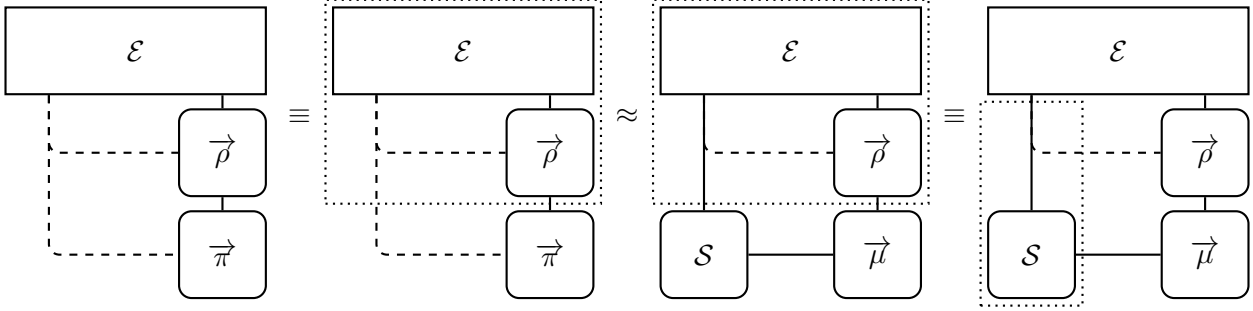
(b) If $\exists \mathcal{S}, \forall \mathcal{E} : \text{EXEC}^{\mathcal{E}}_{\pi} \approx \text{EXEC}^{\mathcal{E}}_{\mathcal{S},\mu}$, we say that the protocol π with a transparent channel emulates μ.



(c) If protocol π with a transparent channel emulates μ, then π emulates μ. Define $\mathcal{E}'[\mathcal{E}, \mathcal{A}]$ and $\mathcal{S}'[\mathcal{S}, \mathcal{A}]$ as shown by the dotted boxes. Then $\forall \mathcal{S}, \mathcal{A}, \mathcal{E} : \text{EXEC}^{\mathcal{E}'[\mathcal{E},\mathcal{A}]}_{\pi} \approx \text{EXEC}^{\mathcal{E}'[\mathcal{E},\mathcal{A}]}_{\mathcal{S},\mu} \Rightarrow \text{EXEC}^{\mathcal{E}}_{\mathcal{A},\pi} \approx \text{EXEC}^{\mathcal{E}}_{\mathcal{S}'[\mathcal{S},\mathcal{A}],\mu}$.



(d) If π with a transparent channel emulates μ and μ emulates υ, then π emulates υ. Define $\mathcal{S}''[\mathcal{S}, \mathcal{S}'] = \mathcal{S}'[\mathcal{S}]$. Then $\forall \mathcal{S}, \mathcal{S}', \mathcal{E} : (\text{EXEC}^{\mathcal{E}}_{\pi} \approx \text{EXEC}^{\mathcal{E}}_{\mathcal{S},\mu} \wedge \text{EXEC}^{\mathcal{E}}_{\mathcal{S},\mu} \approx \text{EXEC}^{\mathcal{E}}_{\mathcal{S}'[\mathcal{S}],\upsilon}) \Rightarrow \text{EXEC}^{\mathcal{E}}_{\pi} \approx \text{EXEC}^{\mathcal{E}}_{\mathcal{S}''[\mathcal{S},\mathcal{S}'],\upsilon}$.



(e) If protocol π with a transparent channel emulates μ, then ρ^{π} with a transparent channel emulates ρ^{μ}. Define $\mathcal{E}'[\mathcal{E}]$ and $\mathcal{S}'[\mathcal{S}]$ as shown by the dotted boxed. Then $\forall \mathcal{E} : \text{EXEC}^{\mathcal{E}'[\mathcal{E}]}_{\pi} \approx \text{EXEC}^{\mathcal{E}'[\mathcal{E}]}_{\mathcal{S},\mu} \Rightarrow \text{EXEC}^{\mathcal{E}}_{\rho^{\pi}} \approx \text{EXEC}^{\mathcal{E}}_{\mathcal{S}'[\mathcal{S}],\rho^{\mu}}$.

Fig. 16: Informal pictorial representations of security definitions and fundamental proofs for universal composability. Solid lines denote communication channels for inputs/outputs (top/bottom of machines) and backdoors (sides). Dashed lines indicate a directly connecting "transparent channel" (typically formalized by a "dummy adversary"). Dotted boxes show how to combine an environment (resp. simulator) with other machines to create a new environment (resp. simulator). By $\vec{\mu}, \vec{\pi}, \vec{\rho}$ we denote multiple machines for protocol μ, π, ρ. The left of 16a shows $\text{EXEC}^{\mathcal{E}}_{\mathcal{A},\pi}$ while the left of 16b shows $\text{EXEC}^{\mathcal{E}}_{\pi}$.

so a formalization of the proof would only need to verify it holds for algebraic adversaries because then the lifting result will apply to imply it for type-safe ones. It is worth noting here that protocols parties are in essence part of the security game, so restricting attention to type-safe protocols can serve as a way to ensure that non-group-elements inputs must not depend on group elements as per FKL by following Zhandry's recommendation of restricting attention to type-safe games.

The model as discussed so far is quite distinct from the framework of ABKLX. In, say, a low-level proof that π emulates μ, a reduction $R[\mathcal{E}, \mathcal{A}]$ could exploit the algebraicness/type safety of $\mathcal{E}$ and $\mathcal{A}$ to extract explanations of group elements they output. However, the simulator has not been given this extra power. This makes this framework analogous to the strict (non-observable, non-programmable) global random oracle model [CDG+18] which gives reductions, but not simulators, extra power over the adversaries. The ABKLX model is more akin to an observable random oracle model [CDG+18, CJS14].

An Observable Algebraic Model. The algebraic UC framework of ABKLX explicitly surfaces the explanations of some group elements output by $\mathcal{E}$ or $\mathcal{A}$. In particular, $\mathcal{A}$ is required to provide explanations of group elements (in terms of group element inputs $\mathcal{E}$ or $\mathcal{A}$ has received) when sending "backdoor" messages to protocol machines. When $\mathcal{E}$ is using a transparent channel to send backdoor messages, it must provide explanations. These explanations are then visible to simulators. We could try to mirror this directly by combining $\mathcal{E}$ and $\mathcal{A}$ with appropriate versions of our extractor $\mathcal{X}^*[\cdot]$, but we will instead consider a version more natural to our framework, mirroring how one might expect an (observable, non-programmable) generic group model to work.

Assume the code of machines is written our oracle framework. Then we capture algebraic or generic security by requiring $\mathcal{E}$ and $\mathcal{A}$ be in $\mathrm{AA}_{\mathsf{O}}[U]$ or $\mathrm{TSA}_{\mathsf{O}}[U]$. Moreover, we endow simulators with the "wiretap" ability to see the oracle queries made by its adversaries $\mathcal{E}$ and $\mathcal{A}$ to O_f for $f \in U.\Pi$.[21] The restrictions on Thm. 4 from the strict model still apply. We examine additional subtleties.

Completeness of transparent channel emulation (Fig. 16c) holds without additional restriction. Note that $\mathcal{S}$ expects to wiretap its environment $\mathcal{E}'[\mathcal{E}, \mathcal{A}]$. Then in the final step, $\mathcal{S}'[\mathcal{S}, \mathcal{A}]$ must emulate this. This causes no issues as $\mathcal{S}'$ has wiretap access to $\mathcal{E}$ and is internally running $\mathcal{A}$, so can wiretap it.

Now consider transitivity (Fig. 16d). The simulator $\mathcal{S}$ introduced by applying the assumption that π with a transparent channel emulates μ expects to wiretap its environment $\mathcal{E}$. However, for applying the second assumption that μ emulates υ we want to interpret $\mathcal{S}$ as an adversary *which would not naturally have a wiretap*. Fortunately, if μ emulates υ, then we can prove that μ emulates υ for adversaries that wiretap the environment.[22] Consequently, we can complete the transitivity proof without additional restrictions.

Finally, consider universal composability (Fig. 16e). The simulator $\mathcal{S}$ expects to wiretap its environment $\mathcal{E}'[\mathcal{E}]$. This environment internally runs both $\mathcal{E}$ and the parties of ρ; however, the final simulator $\mathcal{S}'[S]$ only has wiretap access to $\mathcal{E}$. Seemingly, the natural, reasonable fix for this is for ρ to not just be type safe, but instead to not operate on group elements at all. If we are willing to give up the lifting result and algebraic ρ never outputs group elements, then $\mathcal{E}'[\mathcal{E}]$ can simulate ρ's queries by querying Encode_σ on all group elements (which is not captured by the wiretapping) performing

[21] One might worry that seeing all queries of an algebraic adversary gives the simulator more power than just seeing the explanation of the final output. However, for efficient $U.\sigma$, our equivalence argument in Sec. 3.2 shows the adversary can refrain from making any queries other than those leaking its chosen explanation.

[22] If μ emulates υ, then μ with a transparent channel emulates υ. Then the proof of completeness of transparent channel emulation (Fig. 16c) still works even if $\mathcal{A}$ wiretaps $\mathcal{E}$. Environment $\mathcal{E}'[\mathcal{E}, \mathcal{A}]$ runs $\mathcal{E}$ internally, so can wire tap it for $\mathcal{A}$. Similarly, $\mathcal{S}'$ has wiretap access to $\mathcal{E}$ so can provide this access to $\mathcal{A}$.

the operation ρ requires on the bit representations of elements. Note that it calls Encode_σ even if ρ is generic, so $\mathcal{E}'$ will not be model preserving unless ρ does not interact with group elements.

From the restriction required for universal composability, if we want to consider the composition of many protocols, we can only use a group modeled algebraically/generically for one of the protocols. Analogous restrictions hold for the observable random oracle model and ABKLX's algebraic UC framework. Related subtleties of composing AGM proofs have also been observed in [GT21, KKK21].

Acknowledgments

We thank our anonymous EUROCRYPT and CRYPTO reviewers. Their feedback and recommendations helped us better organize and clarify the results in the paper. We thank Mark Zhandry for detailed insight into how he thinks about his paper [Zha22] and earlier related work.

References

ABK+21. Michel Abdalla, Manuel Barbosa, Jonathan Katz, Julian Loss, and Jiayu Xu. Algebraic adversaries in the universal composability framework. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part III*, volume 13092 of *LNCS*, pages 311–341, Singapore, December 6–10, 2021. Springer, Cham, Switzerland. doi:10.1007/978-3-030-92078-4_11. 30, 34, 35

ADMP20. Navid Alamati, Luca De Feo, Hart Montgomery, and Sikhar Patranabis. Cryptographic group actions and applications. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part II*, volume 12492 of *LNCS*, pages 411–439, Daejeon, South Korea, December 7–11, 2020. Springer, Cham, Switzerland. doi:10.1007/978-3-030-64834-3_14. 23

AHK20. Thomas Agrikola, Dennis Hofheinz, and Julia Kastner. On instantiating the algebraic group model from falsifiable assumptions. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 96–126, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland. doi:10.1007/978-3-030-45724-2_4. 28

BFL20. Balthazar Bauer, Georg Fuchsbauer, and Julian Loss. A classification of computational assumptions in the algebraic group model. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 121–151, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland. doi:10.1007/978-3-030-56880-1_5. 22, 28

BL22. Renas Bacho and Julian Loss. On the adaptive security of the threshold BLS signature scheme. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 193–207, Los Angeles, CA, USA, November 7–11, 2022. ACM Press. doi:10.1145/3548606.3560656. 28

BR06. Mihir Bellare and Phillip Rogaway. The security of triple encryption and a framework for code-based game-playing proofs. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 409–426, St. Petersburg, Russia, May 28 – June 1, 2006. Springer Berlin Heidelberg, Germany. doi:10.1007/11761679_25. 10

BVHKX23. Willow Barkan-Vered, Franklin Harding, Jonathan Keller, and Jiayu Xu. On the non-malleability of ECVRF in the algebraic group model. Cryptology ePrint Archive, Report 2023/1004, 2023. URL: https://eprint.iacr.org/2023/1004. 28

Can01. Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145, Las Vegas, NV, USA, October 14–17, 2001. IEEE Computer Society Press. doi:10.1109/SFCS.2001.959888. 35

CDG+18. Jan Camenisch, Manu Drijvers, Tommaso Gagliardoni, Anja Lehmann, and Gregory Neven. The wonderful world of global random oracles. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part I*, volume 10820 of *LNCS*, pages 280–312, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland. doi:10.1007/978-3-319-78381-9_11. 37

CGH98. Ran Canetti, Oded Goldreich, and Shai Halevi. The random oracle methodology, revisited (preliminary version). In *30th ACM STOC*, pages 209–218, Dallas, TX, USA, May 23–26, 1998. ACM Press. doi:10.1145/276698.276741. 16

CH20. Geoffroy Couteau and Dominik Hartmann. Shorter non-interactive zero-knowledge arguments and ZAPs for algebraic languages. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part III*, volume 12172 of *LNCS*, pages 768–798, Santa Barbara, CA, USA, August 17–21, 2020. Springer, Cham, Switzerland. doi:10.1007/978-3-030-56877-1_27. 9, 22

CJS14. Ran Canetti, Abhishek Jain, and Alessandra Scafuro. Practical UC security with a global random oracle. In Gail-Joon Ahn, Moti Yung, and Ninghui Li, editors, *ACM CCS 2014*, pages 597–608, Scottsdale, AZ, USA, November 3–7, 2014. ACM Press. doi:10.1145/2660267.2660374. 37

Den02. Alexander W. Dent. Adapting the weaknesses of the random oracle model to the generic group model. In Yuliang Zheng, editor, *ASIACRYPT 2002*, volume 2501 of *LNCS*, pages 100–109, Queenstown, New Zealand, December 1–5, 2002. Springer Berlin Heidelberg, Germany. doi:10.1007/3-540-36178-2_6. 16

DH76. Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976. doi:10.1109/TIT.1976.1055638. 2

DHH+21. Nico Döttling, Dominik Hartmann, Dennis Hofheinz, Eike Kiltz, Sven Schäge, and Bogdan Ursu. On the impossibility of purely algebraic signatures. In Kobbi Nissim and Brent Waters, editors, *TCC 2021, Part III*, volume 13044 of *LNCS*, pages 317–349, Raleigh, NC, USA, November 8–11, 2021. Springer, Cham, Switzerland. doi:10.1007/978-3-030-90456-2_11. 11

DHK+23. Julien Duman, Dominik Hartmann, Eike Kiltz, Sabrina Kunzweiler, Jonas Lehmann, and Doreen Riepel. Generic models for group actions. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part I*, volume 13940 of *LNCS*, pages 406–435, Atlanta, GA, USA, May 7–10, 2023. Springer, Cham, Switzerland. doi:10.1007/978-3-031-31368-4_15. 9, 23, 24

Duc09. Léo Ducas. Conception of a language for cryptographic reductions (master thesis, in french), 2009. URL: https://homepages.cwi.nl/~ducas/stage2009/. 11

FKL18. Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 33–62, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland. doi:10.1007/978-3-319-96881-0_2. 2, 6, 7, 8, 11, 13, 14, 15, 28, 34

FPS20. Georg Fuchsbauer, Antoine Plouviez, and Yannick Seurin. Blind Schnorr signatures and signed ElGamal encryption in the algebraic group model. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 63–95, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland. doi:10.1007/978-3-030-45724-2_3. 28, 30

GT21. Ashrujit Ghoshal and Stefano Tessaro. Tight state-restoration soundness in the algebraic group model. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part III*, volume 12827 of *LNCS*, pages 64–93, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland. doi:10.1007/978-3-030-84252-9_3. 38

JM24. Joseph Jaeger and Deep Inder Mohan. Generic and algebraic computation models: When AGM proofs transfer to the GGM. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part V*, volume 14924 of *LNCS*, pages 14–45, Santa Barbara, CA, USA, August 18–22, 2024. Springer, Cham, Switzerland. doi:10.1007/978-3-031-68388-6_2. 1

JS08. Tibor Jager and Jörg Schwenk. On the equivalence of generic group models. In Joonsang Baek, Feng Bao, Kefei Chen, and Xuejia Lai, editors, *ProvSec 2008*, volume 5324 of *LNCS*, pages 200–209, Shanghai, China, October 31 – November 1, 2008. Springer Berlin Heidelberg, Germany. doi:10.1007/978-3-540-88733-1_14. 6, 22, 31

KKK21. Thomas Kerber, Aggelos Kiayias, and Markulf Kohlweiss. Composition with knowledge assumptions. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part IV*, volume 12828 of *LNCS*, pages 364–393, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland. doi:10.1007/978-3-030-84259-8_13. 38

KLX22. Julia Kastner, Julian Loss, and Jiayu Xu. On pairing-free blind signature schemes in the algebraic group model. In Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe, editors, *PKC 2022, Part II*, volume 13178 of *LNCS*, pages 468–497, Virtual Event, March 8–11, 2022. Springer, Cham, Switzerland. doi:10.1007/978-3-030-97131-1_16. 28

LPS23. Helger Lipmaa, Roberto Parisella, and Janno Siim. Algebraic group model with oblivious sampling. In Guy N. Rothblum and Hoeteck Wee, editors, *TCC 2023, Part IV*, volume 14372 of *LNCS*, pages 363–392, Taipei, Taiwan, November 29 – December 2, 2023. Springer, Cham, Switzerland. doi:10.1007/978-3-031-48624-1_14. 30, 31, 34

LPS25. Helger Lipmaa, Roberto Parisella, and Janno Siim. On knowledge-soundness of plonk in ROM from falsifiable assumptions. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part VII*, volume 16006 of *LNCS*, pages 362–395, Santa Barbara, CA, USA, August 17–21, 2025. Springer, Cham, Switzerland. doi:10.1007/978-3-032-01907-3_12. 9, 22, 31, 34

Mau05. Ueli M. Maurer. Abstract models of computation in cryptography (invited paper). In Nigel P. Smart, editor, *10th IMA International Conference on Cryptography and Coding*, volume 3796 of *LNCS*, pages 1–12, Cirencester, UK, December 19–21, 2005. Springer Berlin Heidelberg, Germany. doi:10.1007/11586821_1. 2, 3, 4, 8, 9, 10, 17, 18, 25

Mei23. Lúcás Meier. Idealized models for free (podcast), 2023. URL: https://cronokirby.substack.com/p/idealized-models-for-free. 11

MTT19. Taiga Mizuide, Atsushi Takayasu, and Tsuyoshi Takagi. Tight reductions for Diffie-Hellman variants in the algebraic group model. In Mitsuru Matsui, editor, *CT-RSA 2019*, volume 11405 of *LNCS*, pages 169–188, San Francisco, CA, USA, March 4–8, 2019. Springer, Cham, Switzerland. doi:10.1007/978-3-030-12612-4_9. 9, 22

MZ22. Hart Montgomery and Mark Zhandry. Full quantum equivalence of group action DLog and CDH, and more. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part I*, volume 13791 of *LNCS*, pages 3–32, Taipei, Taiwan, December 5–9, 2022. Springer, Cham, Switzerland. doi:10.1007/978-3-031-22963-3_1. 24

PV05. Pascal Paillier and Damien Vergnaud. Discrete-log-based signatures may not be equivalent to discrete log. In Bimal K. Roy, editor, *ASIACRYPT 2005*, volume 3788 of *LNCS*, pages 1–20, Chennai, India, December 4–8, 2005. Springer Berlin Heidelberg, Germany. doi:10.1007/11593447_1. 14

RS20. Lior Rotem and Gil Segev. Algebraic distinguishers: From discrete logarithms to decisional uber assumptions. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part III*, volume 12552 of *LNCS*, pages 366–389, Durham, NC, USA, November 16–19, 2020. Springer, Cham, Switzerland. doi:10.1007/978-3-030-64381-2_13. 30, 31, 32, 33

RSS11. Thomas Ristenpart, Hovav Shacham, and Thomas Shrimpton. Careful with composition: Limitations of the indifferentiability framework. In Kenneth G. Paterson, editor, *EUROCRYPT 2011*, volume 6632 of *LNCS*, pages 487–506, Tallinn, Estonia, May 15–19, 2011. Springer Berlin Heidelberg, Germany. doi:10.1007/978-3-642-20465-4_27. 22

RSS20. Lior Rotem, Gil Segev, and Ido Shahaf. Generic-group delay functions require hidden-order groups. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part III*, volume 12107 of *LNCS*, pages 155–180, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland. doi:10.1007/978-3-030-45727-3_6. 11

SGS20. Gili Schul-Ganz and Gil Segev. Accumulators in (and beyond) generic groups: Non-trivial batch verification requires interaction. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part II*, volume 12551 of *LNCS*, pages 77–107, Durham, NC, USA, November 16–19, 2020. Springer, Cham, Switzerland. doi:10.1007/978-3-030-64378-2_4. 6, 11

SGS21. Gili Schul-Ganz and Gil Segev. Generic-Group Identity-Based Encryption: A Tight Impossibility Result. In Stefano Tessaro, editor, *2nd Conference on Information-Theoretic Cryptography (ITC 2021)*, volume 199 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 26:1–26:23, Dagstuhl, Germany, 2021. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.ITC.2021.26. 11

Sho97. Victor Shoup. Lower bounds for discrete logarithms and related problems. In Walter Fumy, editor, *EUROCRYPT'97*, volume 1233 of *LNCS*, pages 256–266, Konstanz, Germany, May 11–15, 1997. Springer Berlin Heidelberg, Germany. doi:10.1007/3-540-69053-0_18. 2, 12, 13

Zha22. Mark Zhandry. To label, or not to label (in generic groups). In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part III*, volume 13509 of *LNCS*, pages 66–96, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland. doi:10.1007/978-3-031-15982-4_3. 2, 4, 5, 6, 10, 11, 13, 16, 17, 18, 22, 29, 31, 38, 42

ZZK22. Cong Zhang, Hong-Sheng Zhou, and Jonathan Katz. An analysis of the algebraic group model. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part IV*, volume 13794 of *LNCS*, pages 310–322, Taipei, Taiwan, December 5–9, 2022. Springer, Cham, Switzerland. doi:10.1007/978-3-031-22972-5_11. 2, 5, 6, 8, 13

A (In)equivalence of the Three Frameworks

We continue our discussion from Sec. 5.3 about the equivalence between our three frameworks.

Algorithms on bits. For algorithms whose inputs and outputs are strictly bits, we assume the existence of a compiler C. We assume for each $\mathcal{X}, \mathcal{Y} \in \{\mathcal{P}, \mathcal{M}, \mathcal{C}\}$ it defines a function $\mathsf{C} : \mathcal{X} \to \mathcal{Y}$ so that if $\mathcal{A}$ is an algorithm in $\mathcal{X}$ then $\mathsf{C}(\mathcal{A})$ is an equivalent algorithm in $\mathcal{Y}$ that uses a "comparable" amount of resources.[23] Note that our assumption for the existence of a compiler is only for algorithms

[23] There is some notational ambiguity because we use the same symbol C for the maps between different pairs of $\mathcal{X}$ and $\mathcal{Y}$. In our use of it, which $\mathcal{X}$ and $\mathcal{Y}$ are intended will always be clear.

that operate only on bits and do not interact with U. We will later use this as a tool for comparing algorithms in the frameworks that do interact with U.

A.1 Framework Inequivalence

As stated before, we need to make mild assumptions about the universe U as well as the underlying computation notions $\mathcal{P}$, $\mathcal{M}$, and $\mathcal{C}$ to prove the equivalence between our three frameworks. Here, we justify the need for assumptions on U by giving a natural U for which equivalence does not hold.

Framework Inequivalence Result. This result is based on the following two natural functions, which can be computed "for free" in the pseudocode framework and oracle framework (with the permissive convention), but are not free with the circuit framework or when using the strict convention with the oracle framework.

Definition A.11 (Multiplexer and Identity) *For a given S, we define the* multiplexer function $\mathtt{mux} : S_\perp^2 \times \{0,1\} \to S_\perp$ *by* $\mathtt{mux}(X_0, X_1, b) = X_b$. *We define the* identity function $\mathtt{id} : S_\perp \to S_\perp$ *by* $\mathtt{id}(X) = X$. *These functions with $\perp$ excluded from the domain are denoted* $\mathtt{mux}_S$ *and* $\mathtt{id}_S$.

The following theorem captures our result. Later, when we give our positive result, we will indeed show that the inability to compute `mux` for free is the only weakness of the circuit mode,l and the inability to compute `id` is the only weakness of the oracle model with the strict convention.

Theorem 5. *For any partial universe of computation U, the function* `mux` *can be computed by algorithms in* $\mathrm{TSA}_\mathrm{O}[U]$ *and* $\mathrm{TSA}_\mathrm{P}[U]$. *There exists a U_1 for which* $\mathtt{mux}_S$ *is not computable by any algorithm in* $\mathrm{TSA}_\mathrm{C}[U_1]$. *If we instead used the strict output convention for the oracle framework, then* $\mathtt{mux}_S$ *is computable by an algorithm in* $\mathrm{TSA}_\mathrm{SO}[U_1]$, *but there exists a U_2 for which* $\mathtt{mux}_S$ *is not computable by any algorithm in* $\mathrm{TSA}_\mathrm{O}[U_1]$.

Proof. First note that `mux` is trivial to compute in the pseudocode framework. Consider the algorithm $A_\mathrm{P} \in \mathrm{TSA}_\mathrm{P}[U]$ for any U that on input $\langle\textbf{elem}\rangle$ X_0, X_1 and $\langle\textbf{bit}\rangle$ b simply computes "**if** $(b = 1)$ **return** X_1 **else return** X_2".

It is similarly trivial in the oracle framework with the permissive convention. Consider $A_\mathrm{O} \in \mathrm{TSA}_\mathrm{O}[U]$ for any U that is given input b while its table is initialized to store X_0 and X_1 at some labels ℓ_0 and ℓ_1. Its goal is to output a label ℓ_output at which X_b is stored. Consequently, it can simply output the bitstring ℓ_b because the function $b \mapsto \ell_b$ is a function on bitstrings which is trivially Turing-computable.

For a given p-group $\mathbb{G}$ (with $p > 1$), we will define U_1 by $U_1.S = \mathbb{G}.S$, $U_1.\Pi = \{\mathtt{gen}, \mathtt{op}, \mathtt{inv}\}$, and $U_1.\Sigma = \{\mathtt{eq}\}$. Pick an arbitrary U_2 with $|U_2.S| > 1$ and $U_2.\Pi = U_2.\Sigma = \emptyset$. In the circuit framework $\mathrm{TSA}_\mathrm{C}[U_1]$, the inputs X_0 and X_1 are written onto element wires, and the input b is written on a bit wire. One of the circuit's element wires is specified as being its output. This element wire must either be an input wire or must be the output wire of a $\mathtt{G}_\mathtt{gen}$, $\mathtt{G}_\mathtt{op}$, or $\mathtt{G}_\mathtt{inv}$. The first takes no inputs, and the latter only takes element wires as input. A simple inductive argument thus shows that elements output by a circuit in $\mathrm{TSA}_\mathrm{C}[U]$ can only depend on element inputs; they cannot depend on bit inputs. So in our case, the output of a circuit cannot depend on b, and consequently, a circuit cannot compute $\mathtt{mux}_S$.

When using the strict convention in the oracle framework, the output label ℓ_output is fixed a priori, and the goal of A_O is to cause X_b to be stored at this label. Then it is still possible with U_1, but slightly more complicated for A_O to succeed with the permissive convention. Suppose we added the identity function `id` to U_1. Then A_O could simply query $\mathsf{O}_\mathtt{id}(\ell_b, \ell_\mathrm{output})$. So we only need to

show A_{O} can emulate $\mathsf{O}_{\mathtt{id}}$ using its oracle. To emulate $\mathsf{O}_{\mathtt{id}}(\ell_1, \ell_{\mathrm{out}})$ it can first query $\mathsf{O}_{\mathtt{inv}}(\ell_b, \ell_{\mathrm{output}})$ to store X_b^{-1} at ℓ_{output}. Then it can query the $\mathsf{O}_{\mathtt{op}}(\ell_b, \ell_{\mathrm{output}}, \ell_{\mathrm{output}})$ twice to store $X_b X_b X_b^{-1} = X_b$ at ℓ_{output}. With U_2, the algorithm cannot write into new table entries (because $U_2.\Pi$ is empty), so trivially it cannot write the correct value to ℓ_{output}.[24] □

A.2 Framework Equivalence

Now we prove that the different models are equivalent under appropriate conventions. A challenge in doing so is finding the right level of abstraction to work at.

At one extreme, we could have explicitly defined the syntax and semantics of pseudocode language $\mathcal{P}$, our Turing machines underlying $\mathcal{M}$, and precisely specify how a bit circuit in $\mathcal{C}$ can be built from `nand` gates. Then it would in principle be possible to give a thorough, literal mapping between our computation models. However, this level of detail would make the proof effectively impossible to read, hiding the essential ideas behind layers of excessive pedantry, and would not be robust to changes in $\mathcal{P}$, $\mathcal{M}$, and $\mathcal{C}$.

At the other extreme, a complete lack of detail in the proof could easily result in missing subtle, but important details. Indeed, in earlier drafts of our proof we erroneously believed we had shown an equivalence between the models without requiring any modifications to U. Our corrected result requires the inclusion of `mux` in the universe for the circuit model. Recall by our earlier result that $\Pi = \{\mathtt{gen}, \mathtt{op}, \mathtt{inv}\}$ for a group-based universe does not suffice for a circuit to compute `mux`. Zhandry's [Zha22] circuit-based model for generic groups uses $\{\mathtt{expop}\}$ which is sufficient to emulate `mux`, but there is no indication that this decision was made out of necessity as opposed to, say, an aesthetic desire to only need one gate type for group operations. We only identified the `mux`-based non-equivalence result in Thm. 5 by fleshing out the details of this proof.

Note: We will briefly find it useful to think of the pseudo-code conventions allowing pointers. We write $\langle\mathbf{type}\rangle*$ as the type for pointers to data of type $\langle\mathbf{type}\rangle$. If P has type $\langle\mathbf{type}\rangle*$, then $*P$ denotes the data (of type $\langle\mathbf{type}\rangle$) pointed at by P. If T has type $\langle\mathbf{type}\rangle$, then $\&T$ denotes the address (with type $\langle\mathbf{type}\rangle*$) of T. A variable $\langle\mathbf{type}\rangle*\ P$ will, in fact, be a restricted case of a pointer type $\langle\mathbf{ptr}\rangle$, with an added restriction placed on how $*P$ may be interpreted. In particular, there will be a type $\langle\mathbf{elem}\rangle*$ which is a restriction on $\langle\mathbf{ptr}\rangle$ and has an underlying representation in terms of $\langle\mathbf{bits}\rangle$.

Theorem 6. *Fix a universe of computation U. Then each of* $\mathrm{XA}_{\mathrm{P}}[U]$, $\mathrm{XA}_{\mathrm{O}}[U]$, $\mathrm{XA}_{\mathrm{C}}[U_{\mathtt{mux}}]$, *and* $\mathrm{XA}_{\mathrm{SO}}[U_{\mathtt{id}}]$ *are equivalent, where the subscripts indicate universes where the indicated functions have been added to $U.\Pi$.*

Our proof works by showing how to take an algorithm in any of these models and convert it to an algorithm in another of the models with up to a polynomial loss in efficiency.

Note that if `mux` or `id` can be emulated by other operations available in U by an algorithm in XA_{C} or $\mathrm{XA}_{\mathrm{SO}}$, then one can drop the respective subscript from U for the circuit or strict oracle frameworks. Based on this result, in the rest of the paper, we will view individual algorithms as existing in whichever framework is most notationally convenient. Should we use XA_{C} or $\mathrm{XA}_{\mathrm{SO}}$, we implicitly assume that `mux` and `id` are added to the universe if necessary. This theorem is stronger than our inter-equivalence claim in Sec 5.3 as $\mathrm{XA}_{\mathrm{SO}}$ only needs `id` and not `mux`.

Proof. We will use $\mathrm{XA}_{\mathrm{P}}[U]$ as the hub of our proof by showing how to convert to and from it to the other frameworks. Thus, informally, the mappings we establish give us $\mathrm{XA}_{\mathrm{O}}[U] \rightleftarrows \mathrm{XA}_{\mathrm{P}}[U] \rightleftarrows$

[24] Technically, it could be that $\ell_{\mathrm{output}} = \ell_0$. In that case, we would redefine the identity function to $\mathtt{id}(X, Y) = Y$ to build our counterexample.

Algorithm $\mathcal{A}_{\mathrm{P}\leftarrow\mathrm{C}}(\langle\mathbf{bits}\rangle\ x, \langle\mathbf{elems}\rangle\ X)$

$\langle\mathbf{bit}\rangle$ wire$[w]$ **for** $w \in \mathcal{W}_{\langle\mathbf{bit}\rangle}$
$\langle\mathbf{elem}\rangle$ wire$[w]$ **for** $w \in \mathcal{W}_{\langle\mathbf{elem}\rangle}$
for $i \in [|\mathbf{w}_{\mathrm{in}}|]$ **do** wire$[\mathbf{w}_{\mathrm{in}}[i]] \leftarrow x_i$
for $i \in [|\mathbf{W}_{\mathrm{in}}|]$ **do** wire$[\mathbf{W}_{\mathrm{in}}[i]] \leftarrow X_i$
for $i \in [g]$ **do**
  if $\mathcal{G}_i = \mathtt{nand}$ **do**
    wire$[\mathcal{G}_i.\mathbf{w}_{\mathrm{out}}] \leftarrow \neg(\mathrm{wire}[\mathcal{G}_i.\mathbf{w}_{\mathrm{in}}[1]] \wedge \mathrm{wire}[\mathcal{G}_i.\mathbf{w}_{\mathrm{in}}[2]])$
  elseif $\mathcal{G}_i = \mathcal{G}_{\mathtt{mux}}$ **do**
    $\langle\mathbf{bit}\rangle\ b \leftarrow \mathrm{wire}[\mathcal{G}_i.\mathbf{w}_{\mathrm{in}}[3]]$ // $\mathcal{G}_i.\mathbf{w}_{\mathrm{in}} = [A, B, b]$
    wire$[\mathcal{G}_i.\mathbf{w}_{\mathrm{out}}] \leftarrow \mathrm{wire}[\mathcal{G}_i.\mathbf{w}_{\mathrm{in}}[b+1]]$
  elseif $\mathcal{G}_i = \mathtt{encode}_\sigma$ **do**
    wire$[\mathcal{G}_i.\mathbf{w}_{\mathrm{out}}] \leftarrow \mathrm{encode}_\sigma(\mathrm{wire}[\mathcal{G}_i.\mathbf{w}_{\mathrm{in}}])$
  elseif $\mathcal{G}_i = \mathtt{decode}_\sigma$ **do**
    wire$[\mathcal{G}_i.\mathbf{w}_{\mathrm{out}}[1]] \leftarrow \mathrm{decode}_\sigma(\mathrm{wire}[\mathcal{G}_i.\mathbf{w}_{\mathrm{in}}])$
    wire$[\mathcal{G}_i.\mathbf{w}_{\mathrm{out}}[2]] \leftarrow (\mathrm{wire}[\mathcal{G}_i.\mathbf{w}_{\mathrm{out}}[1]] = \perp)$
  elseif $\mathcal{G}_i = \mathcal{G}_f$ **do** // $f \in U.\Pi$
    wire$[\mathcal{G}_i.\mathbf{w}_{\mathrm{out}}] \leftarrow \mathrm{func}_f(\mathrm{wires}[\mathcal{G}_i.\mathbf{w}_{\mathrm{in}}])$
  elseif $\mathcal{G}_i = \mathcal{G}_\rho$ **do** // $\rho \in U.\Sigma$
    wire$[\mathcal{G}_i.\mathbf{w}_{\mathrm{out}}] \leftarrow \mathrm{rel}_f(\mathrm{wires}[\mathcal{G}_i.\mathbf{w}_{\mathrm{in}}])$
return $(\mathrm{wire}[\mathbf{w}_{\mathrm{out}}], \mathrm{wire}[\mathbf{W}_{\mathrm{out}}])$

Fig. 17: Pseudocode algorithm $\mathcal{A}_{\mathrm{P}\leftarrow\mathrm{C}}$ which emulates a given circuit algorithm

$\mathrm{XA}_{\mathrm{C}}[U_{\mathtt{mux}}]$. Equivalence of the strict oracle based framework is deferred to Appendix A.3 where we show the mapping $\mathrm{XA}_{\mathrm{SO}}[U_{\mathtt{id}}] \rightleftarrows \mathrm{XA}_{\mathrm{O}}[U]$. X is preserved in our mappings and will typically be clear from construction so we will not mention it explicitly. Let us start with the easiest case of converting to $\mathrm{XA}_{\mathrm{P}}[U]$ from the other frameworks.

Converting P $\leftarrow$ C: Let circuit algorithm $\mathcal{A}_{\mathrm{C}} \in \mathrm{XA}_{\mathrm{C}}[U_{\mathtt{mux}}]$ be given. At a high level, we build a pseudocode algorithm $\mathcal{A}_{\mathrm{P}\leftarrow\mathrm{C}}$ which associates a $\langle\mathbf{bit}\rangle$ variable with each bit wire and a $\langle\mathbf{elem}\rangle$ variable with each element wire, then one at a time emulates the gate of the circuit on these variables. To be slightly more detailed, let us introduce some notation for our circuit. Think of $\mathcal{A}_{\mathrm{C}}$ specifying a topologically ordered sequence of gates $\mathcal{G}_1, \ldots, \mathcal{G}_g$ and a set of wires $\mathcal{W}$. Each wire is either a bit wire or an element wire. Vectors $\mathbf{w}_{\mathrm{in}}$, $\mathbf{W}_{\mathrm{in}}$, $\mathbf{w}_{\mathrm{out}}$, and $\mathbf{W}_{\mathrm{out}}$ specify the input and output wires of the circuit (with lowercase denoting bit wires and uppercase denoting element wires). Each gate $\mathcal{G}_i$ is one of $\mathtt{nand}$, $\mathtt{encode}_\sigma$, $\mathtt{decode}_\sigma$, or $\mathcal{G}_\phi$ for $\phi \in U.\Pi \cup U.\Sigma \cup \{\mathtt{mux}\}$. It specifies its input wires $\mathcal{G}_i.\mathbf{w}_{\mathrm{in}}$ and output wires $\mathcal{G}_i.\mathbf{w}_{\mathrm{out}}$ that it acts on. Using all of this syntax, we formally specify $\mathcal{A}_{\mathrm{P}\leftarrow\mathrm{C}}$ in Fig. 17. Note that it only uses encode_σ or decode_σ if the circuit $\mathcal{A}_{\mathrm{C}}$ contains the corresponding gates, hence $\mathcal{A}_{\mathrm{P}\leftarrow\mathrm{C}} \in \mathrm{XA}_{\mathrm{P}}[U]$. The correctness of this algorithm follows from its construction.

Converting O $\rightarrow$ P: Next let oracle algorithm $\mathcal{A}_{\mathrm{O}} \in \mathrm{XA}_{\mathrm{O}}[U]$ be given. At a high level, we will build a pseudocode algorithm $\mathcal{A}_{\mathrm{O}\rightarrow\mathrm{P}}$ which runs $\mathcal{A}_{\mathrm{O}}$, locally storing its own table V which is used to emulate the oracles of $\mathcal{A}_{\mathrm{O}}$. Formally, this algorithm is shown in Fig. 18. Therein, we treat $\ell_{\mathrm{input},i}$ as a constant variable of type $\langle\mathbf{bits}\rangle$ which is hardcoded to store the label into which $\mathcal{A}_{\mathrm{O}}$ expects the i-th element input to be stored. For this algorithm to be in the pseudocode framework, we apply

Algorithm $\mathcal{A}_{\mathrm{O}\to\mathrm{P}}(\langle\mathbf{bits}\rangle\ x, \langle\mathbf{elems}\rangle\ X)$

$\langle\mathbf{elems}\rangle\ V$
$\langle\mathbf{bits}\rangle\ \ell, \ell_{\mathrm{input},i}, \alpha$ // $\ell_{\mathrm{input},i}$ are the input labels
for $i \in [|X|]$ **do** $V[\ell_{\mathrm{input},i}] \leftarrow X[i]$
$(\ell_1, \ldots, \ell_n, x) \leftarrow\!\$\ \mathsf{C}(\mathcal{A}_{\mathrm{O}})^{\mathsf{O}_f, \mathsf{O}_\rho, \mathsf{Encode}_\sigma, \mathsf{Decode}_\sigma}(x)$
return $(x, V[\ell_1], \ldots, V[\ell_n])$

Oracles $\mathsf{O}_f(\ell_1, \ldots, \ell_n, x, \ell_{\mathrm{out}})$

$V[\ell_{\mathrm{out}}] \leftarrow \mathrm{func}_f(V[\ell_1], \ldots, V[\ell_n], x)$
return

Oracles $\mathsf{O}_\rho(\ell_1, \ldots, \ell_n, x)$

return $\mathrm{rel}_\rho(V[\ell_1], \ldots, V[\ell_n], x)$

Oracle $\mathsf{Encode}_\sigma(\ell_1)$

return $\mathrm{encode}_\sigma(V[\ell_1])$

Oracle $\mathsf{Decode}_\sigma(\alpha, \ell_{\mathrm{out}})$

$V[\ell_{\mathrm{out}}] \leftarrow \mathrm{decode}_\sigma(\alpha)$
return

Fig. 18: Pseudocode algorithm $\mathcal{A}_{\mathrm{O}\to\mathrm{P}}$ which emulates oracle algorithm $\mathcal{A}_{\mathrm{O}}$

the compiler $\mathsf{C} : \mathcal{M} \to \mathcal{P}$ to rewrite the internal computations of $\mathcal{A}_{\mathrm{O}}$ as pseudocode computations on $\langle\mathbf{bit}\rangle$ variables. The correctness of this algorithm follows from its construction.

Converting O $\leftarrow$ P: Next, let pseudocode algorithm $\mathcal{A}_{\mathrm{P}} \in \mathrm{XA}_{\mathrm{P}}[U]$ be given which we want to convert to an oracle algorithm $\mathcal{A}_{\mathrm{O}\leftarrow\mathrm{P}}$. At a high level, we build oracle algorithm $\mathcal{A}_{\mathrm{O}\leftarrow\mathrm{P}}$ which runs $\mathcal{A}_{\mathrm{P}}$, replacing its operations on variables of type $\langle\mathbf{elem}\rangle$ with calls to the appropriate oracles, using the addresses of the original variables as the labels for the oracle. As a first step, we make modifications within the pseudocode framework. We will replace each operation that acts on element variables with a call to an oracle that internally uses that function. In short:

- Every operation $A_{\mathrm{out}} \leftarrow \mathrm{func}_f(\mathbf{A}, \mathbf{x})$ is replaced with an oracle call of the form $\mathsf{Func}_f(\&\mathbf{A}_1, \ldots, \&\mathbf{A}_n, \mathbf{x}, \&A_{\mathrm{out}})$. This oracle (given $\langle\mathbf{bits}\rangle\ \mathbf{x}$ and pointers $\langle\mathbf{elem}\rangle*\ P_i$) internally assigns the value $f(*P_1, \ldots, *P_n, \mathbf{x})$ to $*P_{\mathrm{out}}$.
- Every operation $b \leftarrow \mathrm{rel}_\rho(\mathbf{A}, \mathbf{x})$ is replaced with an oracle call of the form $b \leftarrow \mathsf{Rel}_\rho(\&\mathbf{A}_1, \ldots, \&\mathbf{A}_n, \mathbf{x}, \&A_{\mathrm{out}})$. The oracle (given $\langle\mathbf{bits}\rangle\ \mathbf{x}$ and pointers $\langle\mathbf{elem}\rangle*\ P_i$) outputs $\rho(*P_1, \ldots, *P_n, \mathbf{x})$.
- Every operation $\alpha \leftarrow \mathrm{encode}_\sigma(A)$ is replaced with an oracle call $\alpha \leftarrow \mathsf{Enc}(\&A)$. This oracle (given pointer $\langle\mathbf{elem}\rangle*\ P$) outputs $\mathrm{encode}_\sigma(A)$.
- Every operation $A \leftarrow \mathrm{decode}_\sigma(\alpha)$ is replaced with an oracle call $\mathsf{Dec}(\alpha, \&A)$. This oracle (given $\langle\mathbf{bits}\rangle\ \alpha$ and pointer $\langle\mathbf{elem}\rangle*\ P$) internally assigns the value $\mathrm{decode}_\sigma(\alpha)$ to $*P$.
- If the final output of $\mathcal{A}_{\mathrm{P}}$ is $(\langle\mathbf{bits}\rangle\ x, \langle\mathbf{elem}\rangle\ A_1, \ldots, A_n)$, then instead return $(x, \&A_1, \ldots, \&A_n)$.

Let $\mathcal{A}'_{\mathrm{P}}$ denote the algorithm obtained by applying these syntactic modifications to $\mathcal{A}_{\mathrm{P}}$. It can be viewed as an algorithm in $\mathcal{P}$ which expects access oracles named Func_f, Rel_ρ, Enc, and Dec with the specified syntax. Moreover, note that (casting from $\langle\mathbf{elem}\rangle*$ to $\langle\mathbf{bits}\rangle$), the syntax of these oracles exactly matches that of the oracle framework (in particular, the addresses of variables act like the ℓ labels). As such we can replace these new oracles with the appropriate ones from the oracle framework and apply the compiler $\mathsf{C} : \mathcal{M} \to \mathcal{P}$ to obtain $\mathcal{A}_{\mathrm{O}\leftarrow\mathrm{P}} = \mathsf{C}(\mathcal{A}'_{\mathrm{P}})$. The correctness of this algorithm follows from its construction.

Converting P $\to$ C: Consider pseudocode algorithm $\mathcal{A}_{\mathrm{P}} \in \mathrm{XA}_{\mathrm{P}}[U]$ which we want to convert to a circuit algorithm $\mathcal{A}_{\mathrm{P}\to\mathrm{C}} \in \mathrm{XA}_{\mathrm{C}}[U_{\mathtt{mux}}]$. At a high level, we will first **(1)** rewrite $\mathcal{A}_{\mathrm{P}}$ to use its element operations a fixed number of times q in a known order. Then we **(2)** apply the transform above to convert it into $\mathcal{A}'_{\mathrm{P}}$ which makes pointer-based oracle queries. We will **(3)** implement virtual addressing to rewrite our algorithm so that each $\langle\mathbf{elem}\rangle$ variable can only be written to once, but can be read from arbitrarily many times (this is necessary for associating $\langle\mathbf{elem}\rangle$ variables with element wires in a circuit). Having done that, we will **(4)** reconceptualize the algorithm as a sequence of $q+1$ algorithms, each of which maps from the current state of the algorithm and the response from the latest oracle query to the state of the algorithm at the next oracle query and the decision of which element addresses to input to the next oracle. Each individual algorithm can be viewed as being $\mathcal{P}$

algorithms, so we can **(5)** apply our compiler C to obtain a sequence of $q+1$ circuits. We will then **(6)** use `mux` to implement an oracle query as a subcircuit that applies the appropriate element gate. Our final circuit $\mathcal{A}_{\mathrm{P}\to\mathrm{C}}$ is obtained by **(7)** appropriately "soldering" together the sequence of $q+1$ circuits and oracle subcircuits.

Let us describe that in more detail. First we **(1)** picks a fixed order $\mathrm{op}_1, \ldots, \mathrm{op}_\kappa$ on all of the element operation func_f, rel_ρ, encode_σ, and decode_σ. Define $\mathcal{A}_1$ by syntactically modifying the code of $\mathcal{A}_{\mathrm{P}}$ so that whenever it uses one of the operations, it instead applies all of them in the fixed order. Note that this can increase the number of operations used by the algorithm by a multiplicative factor of κ. Now **(2)** apply that transformation described earlier (in $\mathrm{O} \leftarrow \mathrm{P}$) for creating $\mathcal{A}'_{\mathrm{P}}$ to our algorithm $\mathcal{A}_1$ to obtain algorithm $\mathcal{A}_2$. To implement virtual addressing we **(3)** have $\mathcal{A}_3$ adaptively create a virtual address table V which maps from $\&A$ (from $\mathcal{A}_b$'s perspective) to a new address in $1, \ldots, N$. Here N is an upper bound on the number of oracle queries it makes and the number of group elements it is given as input. In particular, the oracles will track a counter i and inside of Func_f or Dec (the oracles that write elements to a specified output address) we increment $i \leftarrow i+1$ and if P is the desired output address we then update $V[P] \leftarrow i$ and assign the derived element to $*i$ in place of $*P$. Inside of Func_f, Rel_ρ, and Enc (the oracles that read elements) if P is one of the specified input addresses we use $*V[P]$ in place of $*P$.

Next **(4)** we subdivide the algorithm, excising the code of the oracles and viewing our algorithm as being only the code that is run between oracle queries. Having done that, now rather than thinking of it as an algorithm that makes oracle queries, we view it as a sequence of $q+1$ algorithms $\mathcal{A}_{4,i}$ for $i = 0, \ldots, q$ where the i-th algorithm takes as input the output of the i-th oracle query (or the initial inputs when $i = 0$) as well as the current state of the program at the time it made said oracle query (including the current values of all variables *and* the current value of the program counter indicating where in the code of $\mathcal{A}_3$ the algorithm was at the time it make that query). Note that the output of the i-th oracle query is empty when it was a query to Func_f or Dec. The output of $\mathcal{A}_{4,i}$ consists of the current state of the program and the inputs to be used in the next oracle query (or the final outputs of the algorithm in the case that $i = q$). Note that the $\mathcal{A}_{4,i}$ does not perform computations on element variables, so all of its variables can be represented as bits. Thus **(5)** by using our compiler $\mathsf{C} : \mathcal{P} \to \mathcal{C}$, we obtain a sequence of circuit algorithms $\mathcal{A}_{5,i} = \mathsf{C}(\mathcal{A}_{4,i})$.

To implement the oracles as circuits **(6)**, we associate the write-once addresses $1, \ldots, N$ used by the oracles for storing element values with wires W_i. The first few of them will be assigned to any initial inputs. Then for each oracle query Rel_ρ or Dec that would write its output with the next address, we associate the next wire. The final of these wires are associated with the algorithm's final output. For each element input to the i-th oracle query, the circuit $\mathcal{A}_{5,i-1}$ will output bits representing the address j it wants to be used for this element. For each preceding element wire w_k (from the program's inputs or a previous query's output) we set up a circuit of `nand` gates to compute the bit $b_{i,k}$ which equals 1 iff $j = k$. Then we use a cascade of `mux` gates (introducing additional temporary element wires for this game) with these $b_{i,k}$ to select out the value of the wire j on a wire that is given as input to a copy of the gate associated with the current oracle query. The appropriate bit wires are also plugged into this gate so that its output wires will contain the desired wires. We obtain **(7)** our final circuit $\mathcal{A}_{\mathrm{P}\to\mathrm{C}}$ by plugging each $\mathcal{A}_{5,i-1}$'s query wires into the i-th oracle subcircuit and its state wires into $\mathcal{A}_{5,i}$. □

A.3 Equivalence Between Oracle Models

We complete the proof of Thm. 6 by showing the equivalence between the permissive and strict output conventions for the oracle-based framework, ie., $\mathrm{XA}_{\mathrm{SO}}[U_{\mathtt{id}}] \rightleftarrows \mathrm{XA}_{\mathrm{O}}[U]$. We will first do the $\to$ direction, showing how to convert a strict algorithm to a permissive one, then the $\leftarrow$ direction.

Algorithm $\mathcal{A}^{\mathsf{O}_f,\mathsf{O}_\rho,\mathsf{Encode}_\sigma,\mathsf{Decode}_\sigma}_{\mathrm{SO}\to\mathrm{O}}(x)$

foreach ℓ **do** $T[\ell] \leftarrow \ell'_0$
foreach i **do** $T[\ell_{\mathrm{input},i}] \leftarrow \ell_{\mathrm{input},i}$
$t \leftarrow 1$
$x \leftarrow\!\!\$\ \mathcal{A}^{\mathsf{SimO}_f,\mathsf{SimO}_\rho,\mathsf{SimEncode}_\sigma,\mathsf{SimDecode}_\sigma}_{\mathrm{SO}}(x)$
return $(x, T[\ell_{\mathrm{output},1}], T[\ell_{\mathrm{output},2}], \dots)$

Oracles $\mathsf{SimO}_f(\ell_1, \dots, \ell_n, x, \ell_{\mathrm{out}})$

$\mathsf{O}_f(T[\ell_1], \dots, T[\ell_n], x, \ell'_t)$
$T[\ell_{\mathrm{out}}] \leftarrow \ell'_t; t \leftarrow t + 1$
return

Oracles $\mathsf{SimO}_\rho(\ell_1, \dots, \ell_n, x)$

return $\mathsf{O}_\rho(T[\ell_1], \dots, T[\ell_n], x)$

Oracles $\mathsf{SimO}_{\mathtt{id}}(\ell_1, \ell_{\mathrm{out}})$

$T[\ell_{\mathrm{out}}] \leftarrow T[\ell_1]$
return

Oracle $\mathsf{SimEncode}_\sigma(\ell_1)$

return $\mathsf{Encode}_\sigma(T[\ell_1])$

Oracle $\mathsf{SimDecode}_\sigma(\alpha, \ell_{\mathrm{out}})$

$\mathsf{Decode}_\sigma(\alpha, \ell'_t)$
$T[\ell_{\mathrm{out}}] \leftarrow \ell'_t; t \leftarrow t + 1$
return

Fig. 19: Oracle algorithm $\mathcal{A}_{\mathrm{SO}\to\mathrm{O}}$ which emulates oracle algorithm $\mathcal{A}_{\mathrm{SO}}$

Let $\mathcal{A}_{\mathrm{SO}} \in \mathrm{XA}_{\mathrm{SO}}[U_{\mathtt{id}}]$ be given, which we want to convert to a permissive oracle algorithm $\mathcal{A}_{\mathrm{SO}\to\mathrm{O}}$. Converting from $\mathrm{XA}_{\mathrm{SO}}[U]$ (without $\mathtt{id}$) to $\mathrm{XA}_{\mathrm{O}}[U]$ would be completely trivial. We would have our new algorithm simply run $\mathcal{A}_{\mathrm{SO}}$, outputting $\ell_{\mathrm{output},i}$ (the label at which $\mathcal{A}_{\mathrm{SO}}$ stores its i-th output) for the i-th element output of the new algorithm. However, because $U_{\mathtt{id}}$ has the additional function $\mathtt{id}$ which may not be contained in U, we must spend a bit more effort to emulate it. In particular, we handle this by having $\mathcal{A}_{\mathrm{SO}\to\mathrm{O}}$ do a sort of virtual memory addressing to simulate the view of $\mathcal{A}_{\mathrm{O}}$. Our algorithm is shown in Fig. 19. It stores a table T which maps the labels at which $\mathcal{A}_{\mathrm{SO}}$ believes elements are written in its table V, to the labels at which $\mathcal{A}_{\mathrm{SO}\to\mathrm{O}}$ has actually written these elements in its own table. The table of $\mathcal{A}_{\mathrm{SO}\to\mathrm{O}}$ is used in a write-once manner, constantly picking new labels to write the output of $\mathsf{O}_{U.\Pi}$ or Decode_σ queries. For this, we let $\ell'_0, \ell'_1, \ell'_2, \dots$ be a sequence of distinct labels which are distinct from the $\ell_{\mathrm{input},i}$ labels. The algorithm writes into these labels, except ℓ'_0 which it always leaves having $\perp$ (and initiates $T[\ell]$ as storing ℓ'_0 for all non-input labels). With this label virtualization, the algorithm can simulate $\mathtt{id}$ queries by simply updating its table T. At the end of execution, the algorithm outputs $T[\ell_{\mathrm{output},i}]$ (the redirected label at which $\mathcal{A}_{\mathrm{SO}}$ stored its i-th output) for its i-th output.

Now let $\mathcal{A}_{\mathrm{O}} \in \mathrm{XA}_{\mathrm{O}}[U]$ be given, which we want to convert to a strict oracle algorithm $\mathcal{A}_{\mathrm{SO}\leftarrow\mathrm{O}}$. Our new algorithm first runs $\mathcal{A}_{\mathrm{O}}$ to completing, forwarding on all oracle queries (note that the oracles of $\mathcal{A}_{\mathrm{SO}\leftarrow\mathrm{O}}$ are a superset of those of $\mathcal{A}_{\mathrm{O}}$). Let $\ell'_0, \ell'_1, \ell'_2, \dots$ be a sequence of distinct labels which are distinct from the $\ell_{\mathrm{output},i}$ labels and any labels that $\mathcal{A}_{\mathrm{O}}$ would ever output. When $\mathcal{A}_{\mathrm{O}}$ outputs $(\ell_1, \dots, \ell_n, x)$ where the ℓ_i are the labels for its elements outputs, the new algorithm queries $\mathsf{O}_{\mathtt{id}}(\ell_i, \ell'_i)$ for $i = 1, \dots, n$ and then queries $\mathsf{O}_{\mathtt{id}}(\ell'_i, \ell'_{\mathrm{output},i})$ for $i = 1, \dots, n$. This process of copying around the labels in two passes is a technicality to handle the possibility that some ℓ_i outputs label coincide with some $\ell_{\mathrm{output},i}$ labels.